Конструирование графовой памяти Рин

Расширенный учебный курс по архитектуре памяти, динамике, эпистемике и инженерии

Расширенная учебная версия проектного мануала RIN

11 августа 2026 г.

Эта версия намеренно шире базового учебного дубля. Её задача — не просто перевести архитектурный замысел в более мягкую педагогическую форму, а развернуть его в курс, где рядом стоят интуиция, определения, формулы, инженерные ограничения, разборы сценариев и аппарат проверки. Базовый roadmap сохранён; расширения, которых не было в исходном тексте в явном виде, вводятся открыто и используются как развитие, а не как тихая замена исходных решений.

Оглавление

1 Как читать этот курс **1**
- 1.1 Не FAQ, а курс . . . 1
- 1.2 Три уровня утверждений . . . 1
- 1.3 Что именно расширено по сравнению с базовым дублем . . . 2
- 1.4 Три прохода чтения . . . 2
- 1.5 Главный критерий хорошего текста о памяти . . . 3

2 Введение: зачем памяти нужен субстрат **5**
- 2.1 Постановка задачи . . . 5
- 2.2 Почему здесь появляется граф . . . 5
- 2.3 Центральный тезис книги . . . 6
- 2.4 Четыре процесса, которые нельзя настраивать по отдельности . . . 6
- 2.5 Чего эта книга не утверждает . . . 7
- 2.6 Как читать этот курс . . . 7

3 Память как система: первый обзор **9**
- 3.1 Зачем нужна отдельная карта системы . . . 9
- 3.2 Что именно строится . . . 9
- 3.3 Четыре слоя системы . . . 9
- 3.4 Что течёт по системе . . . 11
- 3.5 Сквозной пример . . . 11
- 3.6 Что не входит в граф . . . 12
- 3.7 Роль архитектурной главы в остальной книге . . . 12

4 Основные объекты памяти **13**
- 4.1 Зачем вообще нужна отдельная глава понятий . . . 13
- 4.2 Объекты субстрата . . . 13
- 4.3 Поля состояния . . . 14
- 4.4 Операторы над памятью . . . 15
- 4.5 Сцепленные с памятью процессы . . . 17
- 4.6 Стартовое состояние и инженерная перспектива . . . 17
- 4.7 Что должно остаться после этой главы . . . 18

5 Субстрат: типизированный темпоральный граф **19**
- 5.1 Что такое узел и что такое ребро . . . 19

5.1.1 Схема узла 19
5.1.2 Схема ребра 20
5.2 Почему рёбра направленные, типизированные и знаковые 20
5.3 Виды узлов и что значит уверенность 21
5.3.1 От словаря к поведению: операционная матрица 23
5.4 Провенанс и обоснование 24
5.5 Что значит «коммит» и кто его гейтит 24
5.6 Кража схемы у Wikidata 24
5.7 Врождённое и приобретённое: не граница, а континуум 25

6 Обход графа: один движок, три политики 27
6.1 Случайное блуждание как универсальный механизм 27
6.2 Три политики как параметризация 27
6.3 Что walker делает с графом: читает, предлагает, но не коммитит . 28
6.4 Фоновая разведка: один процесс, а не «день/ночь» 29
6.4.1 Поля активации: быстрое внимание и медленный след — не путать 29
6.5 Точки входа: как query превращается в якоря 31
6.6 Семантический вентиль: дёшево и без протечек по омонимии . . . 33
6.6.1 Вентиль — условный throttle, а не плоский множитель 34
6.7 Единство и следствие 35

7 Настроение 37
7.1 Настроение — отдельный сцепленный модуль, а не поле графа . . . 37
7.2 Инерция и многослойность 37
7.3 Setpoint: медленный слой как аттрактор 38
7.4 Настроение зависит от графа, и граф — от настроения 38
7.5 Опасность петли и три тормоза 38
7.6 Метакогниция 39

8 Топология правит потоком 43
8.1 Тезис главы 43
8.2 Стационарное распределение $\neq$ заданные вероятности 43
8.3 Доказуемое свойство телепорта 44
8.4 Пять следствий для работы Рин с памятью 44
8.5 Операционные величины и осторожности 45

9 Математика retrieval: от локального шага к глобальному поведению 47
9.1 Зачем отдельная математическая глава 47
9.2 Переходный оператор 47
9.3 Температура и режим выбора 48
9.4 Якоря, рестарты и two-step retrieval 48
9.5 Hitting time и доступность далёких зон 48
9.6 Спектральная щель и смешивание 49
9.7 Проводимость и bottleneck starvation 49
9.8 Новая инженерная добавка: два бюджета контекст-пакера 49
9.9 Что именно надо мерить на стенде 50
9.10 Вывод 50

10 Fusion, вывод и консолидация 51
10.1 Что такое fusion, механически 51

10.2 Три вида fusion . . . 51
10.3 Fusion сцепляется: sub-fusion — это тоже fusion . . . 53
10.4 Консолидация как огрубление . . . 54
10.5 Кто пишет в узел и в каком виде . . . 54
10.6 Выход fusion — гипотеза, а не установленный факт . . . 56
10.7 Когда со-активация — повод для реплики, а не для узла-факта . . . 56

11 Многомасштабность, слои и забывание 59
11.1 Протекающий интегратор: точный смысл . . . 59
11.2 Два разных смысла «слоёв» — не путать . . . 59
11.3 Консолидация как огрубление графа . . . 60
11.3.1 Коллизии предложений: голоса, а не очередь . . . 61
11.3.2 Что сносим, что оставляем — и кто это решает . . . 62
11.3.3 Кандидаты, наследование линков, слияние уверенностей . . . 64
11.4 Забывание как тень консолидации . . . 65
11.4.1 Правильный decay — это ACT-R . . . 65
11.4.2 Прунинг: не только по использованию . . . 66
11.4.3 importance нельзя диффундировать по путям . . . 66
11.5 Что процесс делает с найденной конфигурацией . . . 67
11.5.1 Забывать ли посылки после fusion? . . . 68

12 Сон и ночная работа 69
12.1 Зачем вообще сон . . . 69
12.2 Телепортирующий обходчик и гарантия покрытия . . . 69
12.3 Что именно делает рой во сне . . . 70
12.4 Разрешение противоречий: молча или через LLM . . . 70
12.5 Скраб не сразу и не обязан быть быстрым . . . 72

13 Эпистемика: уверенность, провенанс и ревизия 73
13.1 Ложные правдоподобные убеждения . . . 73
13.2 Недоопределённость и преждевременный коллапс . . . 73
13.3 Квартет починки . . . 74
13.4 Система поддержки истинности и «ползучая вера» . . . 74
13.5 Эстафета к главе о личности . . . 75

14 Пропагация убеждений как лавина 77
14.1 Постановка: затухающий градиент уверенности . . . 77
14.2 Это самоорганизованная критичность . . . 77
14.3 Где вся сложность: локальное правило . . . 78
14.4 Пороговая логика — это гистерезис . . . 79
14.5 Пороги растут с глубиной консолидации . . . 80

15 Каналы свидетельств и калибровка уверенности 81
15.1 Почему одного скаляра мало, но отказываться от него рано . . . 81
15.2 Лог-отношение правдоподобия . . . 81
15.3 Каналы свидетельств . . . 81
15.4 Штраф за зависимость . . . 82
15.5 Устаревание и темпоральная валидность . . . 82
15.6 Пороги принятия и зона честного незнания . . . 83
15.7 Новая добавка: режим evidence-aware revision . . . 83
15.8 Worked example: слух, наблюдение и ревизия . . . 83
15.9 Зачем это всё для Stage 1 . . . 84

16 Личность: не таблица весов **85**
16.1 Главная ловушка . . . 85
16.2 Личность распределена . . . 85
16.3 Мотивационный вектор: чтобы «личность как процесс» не осталась словом . . . 86
16.4 Граф хранит ингредиенты, а не реакцию . . . 86
16.5 Консолидация реакций: по значимости, а не по приёму . . . 87
16.5.1 Провизорная запись . . . 87
16.5.2 Гейт значимости, а не одобрения . . . 87
16.5.3 Условие по отношениям . . . 88
16.6 Дистиллят как система координат . . . 88

17 Голос и модель **89**
17.1 Субстрат model-agnostic; личность — трансформ . . . 89
17.2 Инструмент φ: скорер по осям характера . . . 89
17.3 Суперпозиция муз — это отбор в признаковом пространстве . . . 89
17.4 Пайплайн по стадиям . . . 90
17.5 Bootstrap: генератор $\neq$ деплой-модель . . . 90
17.6 Raw против guided и псевдо-DPO . . . 91
17.7 Мост между базами: эмбеддинг ConceptNet . . . 91

18 Рантайм и инженерия **93**
18.1 Планировщик как конечный автомат . . . 93
18.1.1 Режимы (состояния автомата) . . . 93
18.1.2 Переходы . . . 93
18.1.3 Реализация . . . 94
18.1.4 Произвольное внимание: внутренний forensic-запрос в Reflecting . . . 94
18.2 Приостановка и возобновление рассуждений . . . 95
18.2.1 Что такое «состояние рассуждения» . . . 95
18.2.2 Возобновление и устаревание . . . 95
18.3 Хранилище и индексирование . . . 96
18.3.1 Из чего состоит хранилище . . . 96
18.3.2 Attention-radii против тиров: это не одно и то же . . . 96
18.3.3 Кто к кому обращается: путь запроса и стоимости . . . 97
18.4 Event sourcing: неизменяемый лог под изменяемым графом . . . 98
18.4.1 Снапшот-чтения: почему обходчик и коммиттер не сталкиваются . . . 98
18.4.2 Устаревшие предложения: коммиттер валидирует, а не просто пишет . . . 99
18.4.3 «Забудь это»: удаление при неизменяемом логе . . . 99
18.5 Write-path: что попадает в память, а что нет . . . 100
18.5.1 Коммит не бинарен: полосы, а не «только ночью» . . . 101
18.5.2 Thought-inbox: как дневная догадка попадает в реплику, не став фактом . . . 102
18.6 Таксономия ретрива: не всё — блуждание . . . 103
18.7 Context-packer: как граф превращается в промпт . . . 103
18.8 Оценки стоимости: память, время, узкие места . . . 104
18.8.1 Стоимость операций . . . 104
18.8.2 Латентность ответа . . . 104
18.8.3 Память . . . 105
18.8.4 Где узкие места, а где нет . . . 105

19 Эмбриональный граф: с чего начинается память 107
19.1 Две базы, две разные роли . . . 107
19.2 Как выглядят узлы и рёбра ConceptNet . . . 107
19.3 Как выглядят узлы ATOMIC . . . 108
19.4 Сшивка: главная инженерная работа . . . 108
19.4.1 Мост через контент-слова и Numberbatch . . . 108
19.4.2 Заземление действующих лиц . . . 109
19.5 Как строить эмбрион: засеянный подграф, а не слепой импорт . . . 109
19.6 Как эмбриональный граф превращается в память Рин . . . 110

20 Первый рабочий срез: Stage 1 111
20.1 Зачем отдельная глава про Stage 1 . . . 111
20.2 Чего Stage 1 должен добиться . . . 111
20.3 Сквозной pipeline v0 . . . 111
20.4 Что должно быть реализовано в первом срезе . . . 112
20.5 Что сознательно откладывается . . . 114
20.6 Три инженерных упрощения первой версии . . . 114
20.7 Критерии готовности . . . 115
20.8 Роль Stage 1 в архитектуре всей книги . . . 116

21 Сквозные разборы: как память работает на цельном сценарии 117
21.1 Зачем нужны сквозные разборы . . . 117
21.2 Сценарий 1: обещание как факт, обязательство и будущий триггер 117
21.3 Сценарий 2: ложная гипотеза и каскад ревизии . . . 118
21.4 Сценарий 3: настроение меняет маршрут, но не право на ложь . . . 119
21.5 Сценарий 4: точное удаление и судьба производных следов . . . 119
21.6 Сценарий 5: от вопроса пользователя к ответу с provenance . . . 120
21.7 Вывод . . . 120

22 Эксперимент: как проверять архитектуру памяти 121
22.1 Что именно проверяется . . . 121
22.2 Три яруса стенда . . . 121
22.3 Четыре типа ground truth . . . 121
22.4 Новая добавка: retrieval-аудит как кортеж . . . 122
22.5 Батарея ключевых тестов . . . 122
22.6 Отдельно про deletion tests . . . 122
22.7 Как измерять ревизуемость, а не только точность . . . 123
22.8 Чего нельзя проверять в самом конце . . . 123
22.9 Что должно выйти из хорошего стенда . . . 123

23 Failure Modes: как память ломается 125
23.1 Зачем нужен отдельный каталог поломок . . . 125
23.2 Топологические поломки . . . 125
23.3 Эпистемические поломки . . . 125
23.4 Динамические поломки . . . 126
23.5 Инженерные поломки . . . 126
23.6 Таблица быстрой диагностики . . . 127
23.7 Главный вывод . . . 127

24 Архитектурные развилки и исследовательские продолжения 129
24.1 Зачем обсуждать альтернативы . . . 129
24.2 Граф против плоского векторного хранилища . . . 129

24.3 Граф против чисто символической базы . . . 129
24.4 Граф против динамической памяти . . . 129
24.5 Новая добавка: правило чистого интерфейса для дальних ветвей . . 130
24.6 Почему Stage 1 всё ещё главнее дальних ветвей . . . 130
24.7 Порог входа новой ветви . . . 131
24.8 Вывод . . . 131

25 Обучение без «обучения» 133
25.1 Мы шлифуем веса, не занимаясь глубоким обучением . . . 133
25.2 Это message passing — но с локальными правилами . . . 133
25.3 Мост к «настоящей» графовой сети . . . 134
25.4 Связь с обучаемыми графовыми моделями . . . 134

26 Заключение: единая картина 135
26.1 Один субстрат, много операторов . . . 135
26.2 Четыре кита связаны . . . 135
26.3 Сквозные принципы . . . 135
26.4 Единая спина курса . . . 136
26.5 Что делать дальше . . . 136

A Нотация 137

B Глоссарий внешних идей 139

C Сводка алгоритмов 143
C.1 Единый блуждающий агент (три политики) . . . 143
C.2 Локальное правило и лавинная пропагация . . . 143
C.3 Забывание, прунинг, консолидация . . . 144
C.4 Телепорт-скраб (анти-энтропия) . . . 144
C.5 Настроение и монитор руминации . . . 144
C.6 Оценивание, значимость и консолидация реакции . . . 144
C.7 Скорер характера и отбор голоса . . . 145
C.8 Стенд эксперимента . . . 145

D Будущее: стоит ли переходить на гиперграфы 147
D.1 Проблема, которая это мотивирует . . . 147
D.2 Почему симплексы — плохой ответ . . . 147
D.3 Гиперребро — прямой ответ . . . 147
D.4 Ключевое наблюдение: мы уже наполовину гиперграф . . . 148
D.5 Первоклассные гиперрёбра: за и против . . . 148
D.6 Рекомендация . . . 148

E Будущее: клеточные комплексы как язык для составных актов 149
E.1 Зачем вообще думать о клетках . . . 149
E.2 Базовая идея . . . 149
E.3 Что это даёт памяти . . . 149
E.4 Чего это не решает . . . 150
E.5 Правильный статус этой ветви . . . 150

F Будущее: память как динамика (резервуар, ассоциативные аттракторы, кванты) 151
F.1 Постановка: память в весах, воспоминание как траектория . . . 151
F.2 Математическая ось: один лапласиан — три динамики . . . 152

F.3 Нейронаучная опора (принцип, а не крысиный мозг) 152
F.4 Предлагаемая гибридная архитектура 153
F.5 Настроение как нейромодулятор и «пирожки Алисы» 153
F.6 Честный разбор: что покупаем и чем платим 154
F.7 Ближняя инженерная форма ветки: что именно стоило бы пробовать 155
F.8 Квантовый слой: честно 156
F.9 Вердикт и как оставить дверь открытой 157

G Будущие ветви: как расширять систему, не теряя её 159
G.1 Две обещанные дальние ветви 159
G.2 Что нужно требовать от любой новой ветви 159
G.3 Резервуарная ветвь: что в ней действительно интересно 159
G.4 Квантовая ветвь: чего нельзя обещать заранее 160
G.5 Правильная амбиция 160

H Расширение: гибридный резервуарный слой для памяти Рин 161
H.1 Почему эта ветка отличается от прочих будущих ветвей 161
H.2 Ключевая идея: две формы памяти вместо одной 161
H.3 Краткая теория: что такое резервуар в этом контексте 162
H.4 Архитектурный скелет 162
H.5 Новая инженерная рекомендация: начинать не со всего графа . . . 164
H.6 Mood как контроллер режима 164
H.7 Гомеостатический контроллер: «пирожки Алисы» 165
H.8 Как объединяются mood и гомеостаз 165
H.9 Что строить сначала 166
H.10 Как проверять 166
H.11 Главные риски 167
H.12 Вердикт 167

Литература 169

1. Как читать этот курс

1.1. Не FAQ, а курс

Эта версия текста сознательно написана не как памятка для уже посвящённых и не как монография, где читатель обязан заранее жить внутри терминологии. Жанр здесь другой: *курс по архитектуре памяти*. Это значит, что изложение должно делать не только две вещи — сообщать решения и хранить корректность, — но и третью: показывать, *почему* решение вообще появилось, на какой вопрос отвечает и чем рискует.

Отсюда три обязательства текста.

1. Определение появляется раньше, чем начинает незаметно работать.
2. Формула появляется там, где без неё теряется механизм, а не для украшения.
3. Инженерный компромисс называется компромиссом, а не тайком выдаётся за закон природы.

1.2. Три уровня утверждений

В курсе важно не смешивать три рода высказываний.

Определение 1.1 (Архитектурное утверждение)**.** Утверждение архитектурно, если от него зависит сам класс строящейся системы: например, что память хранится как явный типизированный граф; что retrieval, внутренняя жизнь и сон реализуются как политики одного обходчика; что факт и гипотеза различаются эпистемически, а не только текстово.

Определение 1.2 (Инженерное упрощение)**.** Утверждение инженерно, если оно выбрано как первое рабочее приближение, но в принципе может быть заменено без смены класса системы: например, жёсткий порог пересечения вместо полноценной кластерной консолидации, LSH/MinHash вместо более тонкой структуры сходства, разреженный Inspector вместо тяжёлого explainability-слоя.

Определение 1.3 (Исследовательская ветвь)**.** Утверждение относится к исследовательской ветви, если оно не нужно для Stage 1, но обещает новый класс эффектов: клеточные комплексы, динамическая память, резервуарные вычисления, квантовые ассоциативные схемы.

Самая частая методологическая ошибка

Если инженерное упрощение обсуждается как будто это и есть архитектура, система становится хрупкой не из-за механики, а из-за языка описания. Разработчик начинает бояться менять то, что на самом деле было

временным. Обратная ошибка не менее вредна: если архитектурное решение выдаётся за временную деталь, его начинают размывать «локальными оптимизациями», и система теряет идентичность.

1.3. Что именно расширено по сравнению с базовым дублем

В этом курсе есть места, где добавлена не только педагогика, но и техника. Это сделано открыто.

1. Retrieval расписан не только качественно, но и как оператор на графе: с матрицей переходов, рестартом, температурой, hitting time и спектральными ограничениями.

2. Эпистемика развернута в виде каналов свидетельств, а не только одного скалярного поля уверенности.

3. Экспериментальный раздел усилен явной диагностикой failure modes.

4. Добавлены сквозные разборы, где память прослеживается от записи события до ревизии производных узлов.

Почему это не «раздуть текст»

Когда архитектура сложна, краткость бывает двух разных сортов. Бывает краткость хорошая: выкинули повтор и оставили идею. А бывает краткость опасная: скрыли промежуточные мосты, и тогда текст выглядит компактным, но держится на том, что читатель сам за автора достроит половину механизма. Расширенный курс борется именно со вторым типом краткости.

1.4. Три прохода чтения

Для такого текста полезно иметь не один, а три режима чтения.

Первый проход — карта. Нужно понять, что система вообще считает памятью, какие в ней есть процессы и где граница между графом, логом, рантаймом и языковой моделью.

Второй проход — механика. Здесь уже важны формулы, типы узлов, поля, операторы консолидации, режимы обходчика и пропагация уверенности.

Третий проход — споры и открытые ветви. На этом уровне читаются не столько ответы, сколько развилки: где исходное решение твёрдое, где оно временное, что можно заменить, а что нельзя, не разрушив замысел.

1.5. Главный критерий хорошего текста о памяти

Хороший текст об архитектуре памяти должен выдерживать двоякую проверку.

Принцип 1.1 (Два экзамена)**.** Первый экзамен — *технический*: после чтения должно быть возможно собрать Stage 1, придумать тесты, найти диагностические переменные и не потерять roadmap. Второй экзамен — *педагогический*: должно быть понятно, почему эта архитектура вообще выглядит именно так, а не как список случайно соседствующих трюков.

Этим двум экзаменам и подчинён весь дальнейший порядок курса.

2. Введение: зачем памяти нужен субстрат

2.1. Постановка задачи

У локального AI-компаньона есть требование, которое почти невозможно обойти архитектурной хитростью: он должен вести себя не как одноразовый ответчик на очередной запрос, а как сущность с накопленной биографией. Это означает по меньшей мере четыре вещи.

1. Он должен **связывать** события между собой, а не хранить их как изолированные записи.
2. Он должен **ошибаться не хаотически, а осмысленно**: делать предварительные выводы, сомневаться в них и позднее их пересматривать.
3. Он должен **меняться во времени**: недавнее должно звучать иначе, чем давнее; важное должно переживать неважное; повторяющееся должно сворачиваться в более устойчивые структуры.
4. Он должен **вспоминать контекстно**: не одинаково по любой команде, а в зависимости от текущего вопроса, эмоционального состояния и уже активированного окружения.

Плоская история сообщений решает лишь малую часть этой задачи. Векторная база воспоминаний продвигается дальше: она позволяет искать смыслово похожие записи. Но ни история сообщений, ни набор независимых векторных элементов сами по себе не дают ответа на более глубокий вопрос: *где в памяти живут связи между фактами, уверенность в них, их происхождение и их эволюция во времени?*

2.2. Почему здесь появляется граф

Слово “граф” легко превратить в пустую модную вывеску. В этой книге под графом понимается не престижная структура данных сама по себе, а минимально естественный ответ на перечисленные выше требования.

Если память должна хранить не только “что произошло”, но и “с чем это связано”, “насколько этому доверяют”, “из чего это было выведено”, “как это влияет на соседние убеждения” и “когда это было важно”, то у архитектуры почти неизбежно возникают:

- **узлы** как носители содержания;
- **рёбра** как носители отношений;
- **метаданные** на узлах и рёбрах;
- **процессы над графом**, а не просто чтение из хранилища.

Граф здесь не украшение

Если убрать из памяти отношения и оставить только список фрагментов текста, система ещё сможет что-то “вытаскивать по похожести”, но уже не сможет внятно ответить, почему одно суждение усиливает другое, откуда возникла гипотеза, что следует пересмотреть после опровержения факта и по какому маршруту вообще течёт активация. Иначе говоря, в такой системе не останется самой *механики памяти*; останется только склад.

2.3. Центральный тезис книги

Принцип 2.1 (Главная формула всего курса)**.** Память Рин рассматривается как **один типизированный темпоральный граф с метаданными на узлах и рёбрах**. Retrieval, обход, вывод, консолидация, забывание, ночная переработка, настроение, личность и языковое выражение трактуются как **операторы над этим субстратом** или как сцепленные с ним процессы, а не как набор несогласованных подсистем с разными моделями мира.

Именно этот тезис задаёт порядок изложения. Сначала вводится архитектурная карта системы (глава 3). Затем даётся словарь основных объектов (глава 4). После этого разбирается субстрат памяти (глава 5) и его многомасштабная динамика (глава 11). Далее рассматриваются основные процессы: настроение (глава 7), обход графа (глава 6), fusion и консолидация (глава 10), эпистемика и ревизия убеждений (главы 13 и 14), влияние топологии (глава 8), ночная работа (глава 12), слой личности и голоса (главы 16 и 17), инженерный рантайм (глава 18) и, наконец, экспериментальная проверка (глава 22).

2.4. Четыре процесса, которые нельзя настраивать по отдельности

При первом знакомстве с задачей возникает соблазн мыслить модулями: сначала “сделать retrieval”, потом “добавить забывание”, потом “приделать сон”. Такой подход удобен управленчески, но почти всегда ошибочен содержательно. В памяти Рин особенно тесно сцеплены четыре процесса.

1. **Извлечение релевантного.** Что именно всплывает в ответ на запрос, внутренний импульс или текущую сцену.
2. **Пропагация уверенности.** Как новый факт усиливает, ослабляет или ставит под сомнение соседние убеждения.
3. **Забывание и консолидация.** Что тускнеет, что переживает время, а что сворачивается в обобщение.
4. **Ночная переработка.** Как память в фоне инспектирует сама себя, чинит противоречия и перестраивает структуру.

Эти процессы читают и пишут одни и те же поля одного и того же графа. Изменение уверенности меняет то, что будет извлечено. Изменение топологии меняет маршруты обхода. Консолидация меняет последующую пропагацию. Ночной скраб меняет судьбу гипотез, созданных дневными операторами.

Ошибка независимой настройки

Если каждый оператор проектируется как будто остальные существуют где-то рядом, но не вмешиваются в его поведение, то система может хорошо работать в локальном тесте и при этом распадаться в общем цикле. Например, “удачный retrieval” на фиксированном графе может оказаться неудачным после первой же волны консолидации; “аккуратная пропагация” может превратиться в лавину при иной топологии; “красивое забывание” может уничтожить материал, без которого ревизия убеждений станет невозможной.

Именно поэтому завершающая часть книги посвящена не красивым примерам, а экспериментальному стенду с контролируемой истиной (глава 22). Для такой архитектуры особенно важно мерить не изолированные фрагменты поведения, а их совместную физику.

2.5. Чего эта книга не утверждает

Текст следует читать как **рабочую архитектурную теорию памяти для RIN**, а не как финальную общую теорию разума. В нём есть три разных слоя утверждений, и их полезно различать с самого начала.

1. **Твёрдые инженерные решения.** Например, требование единого субстрата или различение активации и уверенности.
2. **Рабочие гипотезы.** Например, конкретные формы обхода, консолидации или эпистемических предохранителей.
3. **Дальние линии развития.** Например, переход к гиперграфам, клеточным комплексам или иным более выразительным субстратам.

Граница между моделью и реальностью

Ни одна архитектура памяти не обязана быть истинной в сильном смысле слова; она обязана быть содержательно плодотворной, инженерно управляемой и проверяемой на стенде. Поэтому в этой книге важнее не вопрос “похоже ли это на окончательную истину о памяти”, а вопрос “даёт ли такая схема хорошие механизмы, понятные сбои и измеримые компромиссы”.

2.6. Как читать этот курс

Книга задумана так, чтобы не требовать от читателя постоянного угадывания, “что именно автор имел в виду”. Ради этого соблюдаются два правила.

1. Новый объект по возможности сначала **мотивируется**, затем **называется**, затем **формализуется**.
2. Формула появляется там, где без неё уже становится неточно, а не там, где удобно сразу обозначить символы.

Если нужен быстрый вход в систему, достаточно прочитать главы 3, 4, 5 и 20. Если нужен полный контур, текст лучше читать по порядку: в этой версии книги порядок глав соответствует логике постепенного усложнения модели.

3. Память как система: первый обзор

3.1. Зачем нужна отдельная карта системы

Прежде чем обсуждать отдельные механизмы, полезно увидеть всю систему как один замкнутый цикл. Иначе почти неизбежно возникает ложное ощущение, что память состоит из набора независимых блоков: где-то отдельно лежит граф, где-то отдельно работает retrieval, где-то отдельно находится настроение, а языковая модель “просто забирает контекст”. Для архитектуры памяти это опасная иллюзия. Почти каждый блок влияет на остальные двояко: он не только читает состояние системы, но и перестраивает его.

3.2. Что именно строится

Рассматривается локальный AI-компаньон, память которого должна обладать следующими свойствами:

- быть долговременной, а не ограниченной текущим окном контекста;
- быть ассоциативной, а не только хронологической;
- различать наблюдение, гипотезу, предпочтение, реактивный паттерн и состояние отношений;
- допускать пересмотр убеждений;
- учитывать эмоциональную и личностную окраску того, что всплывает;
- поддерживать фоновую переработку памяти, а не только обслуживание очередного запроса.

Чтобы эти свойства не рассыпались по разным хранилищам и разным схемам данных, вводится единый субстрат.

Принцип 3.1 (Единый субстрат)**.** Память Рин моделируется как **один типизированный темпоральный граф с метаданными на узлах и рёбрах**. Все основные процессы памяти либо непосредственно работают на этом графе, либо сцеплены с ним так, что их состояние можно выразить через чтение и запись к его полям.

3.3. Четыре слоя системы

Если смотреть на систему сверху, в ней удобно различать четыре слоя.

1. **Субстрат.** Сам граф памяти: узлы, рёбра, уверенность, провенанс, валентность, временной слой, важность и другие поля (глава 5).

2. **Операторы.** Обход, retrieval, fusion, консолидация, пропагация, забывание, сон и скраб. Это динамика памяти как таковой (главы 6, 10, 11, 14, 12).
3. **Состояния.** Настроение, эпистемические статусы, вспомогательная метакогнитивная динамика. Эти сущности не следует путать с графом, но они двусторонне с ним связаны (главы 7, 13).
4. **Выражение.** Слой, который превращает внутреннюю структуру в реплику, тон, фокус внимания и устойчивый характер (главы 16, 17).

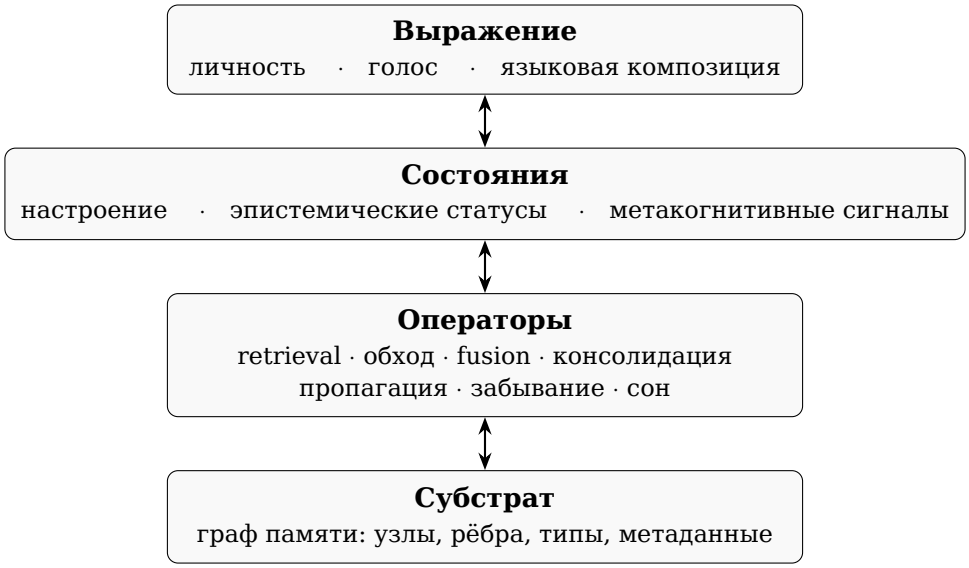


Рис. 3.1: Общая карта системы. Стрелки двусторонние: слой не просто потребляет нижележащий, но и изменяет его состояние.

Чего нельзя делать с этой схемой

Эти четыре слоя нельзя понимать как четыре независимые базы или четыре микросервиса с отдельными картинами мира. Слой выражения не должен придумывать собственную память поверх графа. Слой состояний не должен дублировать содержательное знание в виде параллельного скрытого графа. Операторы не должны тайно хранить свою “локальную истину”, не согласованную с общим субстратом.

«Один субстрат» — дисциплина, а не догма “всё есть граф”

Тезис о едином субстрате легко перетянуть слишком далеко и начать складывать в граф вообще всё: runtime-очереди, streaming-флаги, транзиентные тики настроения, состояние планировщика. Это превращает граф не в память, а в свалку состояний.

граф = память, event log = история, телеметрия = самонаблюдение,

scheduler state = управление, prompt context = временная сборка для LLM.

Эти сущности связаны, но это **разные хранилища с разными законами жизни**. Настроение — сцепленный сбоку модуль; самотелеметрия метакогниции в инженерном варианте естественно оформляется как кольцевой буфер; планировщик — как конечный автомат режимов работы. Пра-

вило простое: если данное транзиентно или относится к управлению, истории или упаковке контекста, то оно не должно механически жить в графе памяти.

3.4. Что течёт по системе

Через эту архитектуру проходят несколько разных “величин”, и их полезно не смешивать.

Определение 3.1 (Активация)**.** Активация — это краткоживущая подсветка узла: мера того, насколько он важен *сейчас*, в данном локальном ходе памяти.

Определение 3.2 (Уверенность)**.** Уверенность — это медленно меняющаяся степень убеждённости в содержании узла. Она не должна путаться с активацией: можно быть уверенным в факте и при этом не держать его в фокусе; можно, наоборот, временно активировать сомнительную гипотезу.

Определение 3.3 (Валентность)**.** Валентность — аффективная окраска содержания. Она отвечает не за истинность узла, а за то, как этот узел переживается и какую роль играет в настроенческой динамике.

Определение 3.4 (Структурное изменение)**.** Структурное изменение — это рождение, укрепление, ослабление, слияние или удаление узлов и рёбер. В отличие от активации, оно меняет саму геометрию памяти.

3.5. Сквозной пример

Полезно проследить полный цикл на одном эпизоде. Пусть во время стрима зритель пишет: “Опять слил ранг в доте”.

От реплики до перестройки памяти

1. Кодирование события. В систему поступает наблюдение о конкретном эпизоде. Оно получает тип, базовую уверенность, временной слой и привязку к участнику взаимодействия.
2. Поиск якорей. По содержанию и контексту сцены находятся ближайшие узлы: прежние упоминания доты, прошлые реплики этого зрителя, возможные эмоциональные реакции на проигрыш.
3. Расширение окрестности. От якорей запускается ограниченный обход. Он поднимает не только прямые совпадения, но и соседние ассоциативные структуры.
4. Локальный вывод. Совместная активация нескольких узлов может породить новую гипотезу: например, что в данной сцене уместен мягкий подкол, а не сухое сочувствие.
5. Выражение. На сцену выходит слой личности и голоса: из всплывшего материала строится конкретная реплика в характерном стиле.
6. Обратная запись. Реакция зрителя становится частью новых данных. Если подкол сработал, укрепляется соответствующий паттерн; если нет, появляется основание для пересмотра.
7. Фоновая переработка. Позднее эта сцена может войти в более устойчивое знание: “данный зритель часто шутит о рейтинге”, “такая интонация обычно безопасна”, “эти две темы часто связаны”.

Этот пример показывает главный архитектурный факт: реплика является не конечной целью отдельного генератора, а **одним моментом внутри цикла памяти**. Система не просто говорит; она одновременно перестраивает себя.

3.6. Что не входит в граф

Тезис о едином субстрате не означает, что “всё вообще” надо хранить в памяти-графе.

- **Граф памяти** хранит долговременное содержательное знание и его обоснования.
- **Event log** хранит историю коммитов и удалений.
- **Телеметрия** хранит самонаблюдение и служебные сигналы.
- **Планировщик** хранит режимы работы и бюджет вычислений.
- **Промпт-контекст** хранит временную сборку того, что подаётся в LLM на конкретном шаге.

Почему это различение принципиально

Если event log, граф, телеметрия и prompt context свалить в одну кучу, получится не богатая память, а неразличимая смесь истории, управления и содержания. Такой объект неудобен не только технически. Он делает почти невозможным ответ на вопрос, *что именно является памятью*, а что лишь следом работы памяти.

3.7. Роль архитектурной главы в остальной книге

Эта глава не заменяет дальнейшие определения и механизмы. Её задача скромнее: задать правильную оптику. После неё должно быть ясно не “как реализуется каждая функция”, а:

- что в системе считается субстратом;
- какие процессы над ним текут;
- почему они не независимы;
- где проходит граница между памятью, состоянием и выражением.

Следующий шаг поэтому естественен: после общей карты нужен словарь основных объектов. Этой задаче посвящена глава 4.

4. Основные объекты памяти

4.1. Зачем вообще нужна отдельная глава понятий

В технических текстах о памяти часто возникает неприятная ситуация: читатель уже встретил “уверенность”, “активацию”, “провенанс”, “консолидацию” и “walker”, но ещё не получил компактной картины того, как эти слова соотносятся друг с другом. Формально каждое из них потом где-то определяется, однако понимание успевает распасться на локальные фрагменты.

Задача этой главы обратная: собрать в одном месте **основные объекты**, не пытаясь здесь же полностью исчерпать все механизмы. Поэтому формулировки ниже намеренно коротки. Их роль — дать рабочий словарь и развести сущности, которые особенно легко спутать.

4.2. Объекты субстрата

Определение 4.1 (Граф памяти)**.** *Граф памяти* $G = (V, E)$ — направленный динамический граф, в котором узлы V несут содержательные единицы памяти, а рёбра E несут отношения между ними. Граф называется *типизированным*, потому что и узлы, и рёбра имеют виды; *темпоральным*, потому что память различает временные слои и возраст содержания; *динамическим*, потому что операторы могут менять и узлы, и рёбра.

Определение 4.2 (Узел памяти)**.** *Узел памяти* $v \in V$ — это единица содержания, с которой можно работать как с объектом: наблюдение, факт, эпизод, гипотеза, предпочтение, паттерн реакции, модель отношения, элемент самопредставления и так далее. Узел хранит не только текст или embedding, но и поля состояния: тип узла, уверенность, провенанс, временной слой, активацию, валентность, важность и служебные флаги.

Почему узел — это больше, чем “запись”

Если узел понимать только как текстовый фрагмент, память превращается в коллекцию документов. Узел в этой архитектуре ближе к “объекту мышления”: он не просто что-то содержит, но ещё может быть активирован, усилен, ослаблен, проверен, связан с посылками и включён в дальнейший вывод.

Определение 4.3 (Тип узла)**.** `node_kind` — вид узла, определяющий, как именно его следует читать и обрабатывать. В частности, именно тип узла задаёт смысл поля `confidence`. Для наблюдаемого факта это степень доверия к истинности; для предпочтения — устойчивость оценки; для реактивного паттерна — надёжность модели ответа; для mood-summary это может быть вообще не числовая истина, а служебная свёртка состояния.

Почему тип узла не декоративен

В памяти проекта сосуществуют по меньшей мере observation, claim, hypothesis, episode, preference, reaction_pattern, relationship_state, boundary, skill/procedure, self_model и mood-summary. Эти сущности нельзя честно измерять одной и той же шкалой: “уверенность 0.8”, относящаяся к факту, и “уверенность 0.8”, относящаяся к предпочтению, означают разные инженерные вещи.

Определение 4.4 (Ребро)**.** *Ребро* $(u \to v)$ — направленная связь между двумя узлами, несущая тип отношения. Примеры: таксономическое включение, причинная поддержка, противоречие, ассоциация, участие в одном эпизоде, отношение к одному участнику взаимодействия.

Определение 4.5 (Два канала ребра)**.** На ребре живут два разных числовых канала:

- **вес перехода** $w_{uv} \geq 0$, который нужен обходу и говорит, насколько охотно маршрут пойдёт по этой связи;
- **знак** $s_{uv} \in \{+1, -1\}$, который нужен эпистемике и говорит, усиливает ли эта связь убеждение или ослабляет его.

Эти каналы не следует смешивать: маршрут может часто ходить по ребру, которое при этом вносит отрицательный вклад в уверенность, и наоборот.

Определение 4.6 (Провенанс (обоснование))**.** *Провенанс* — это информация о происхождении узла: из какого наблюдения, события, источника или набора посылок он возник. Для выводных узлов провенанс играет роль *обоснования*, то есть *justification set*. Важно, что provenance — это **не** полный трек обхода: маршрут walker’a шумен, может храниться отдельно и сам по себе не является обоснованием. Именно provenance делает возможной ретракцию: если опровергнута посылка, можно найти и её следствия.

Определение 4.7 (Время: системное и содержательное)**.** *Время* в памяти удобно разводить на два канала. **Системное время** — это минимальный набор меток, нужный самим операторам: `layer` (сцена, сессия, день, долгий паттерн) и, как правило, одна служебная метка вроде времени кодирования или последнего доступа. **Содержательное время** — это ответ на вопрос “когда это произошло” внутри самого мира памяти. Оно может моделироваться отдельными узлами и быть реконструктивным, а значит, и ошибочным. Интервалы валидности (`start`/`end`) имеет смысл вешать лишь на те факты, для которых это содержательно нужно, а не механически на каждый узел.

4.3. Поля состояния

Определение 4.8 (Уверенность)**.** *Уверенность* $\mathrm{conf}(v) \in [-1, 1]$ — знаковая степень убеждённости в содержании узла. Положительный знак означает “скорее да”, отрицательный — “скорее нет”, нуль соответствует незнанию или эпистемической неопределённости. Такая запись удобна тем, что различает два принципиально разных состояния: “неверие” и “отсутствие знания”.

$$\mathrm{conf} = 2P - 1, \tag{4.1}$$

где P — эквивалентная вероятность истинности. Это соответствие полезно, но в книге предпочтителен именно знаковый скаляр: он лучше согласуется с противоречием и ретракцией. Практически удобно считать, что *зона незнания* задаётся условием $|\mathrm{conf}(v)| < h$ с грубым ориентиром $h \approx 0{,}3$. Именно такие узлы являются естественными кандидатами на `need_to_discover`, пересмотр или удаление. Уверенность меняется медленно: через пропагацию и ревизию убеждений, а не при каждом локальном подсвечивании. Принципиальное ограничение: выводный узел не получает $|\mathrm{conf}| = 1$.

Чего нельзя требовать от confidence

Одно число не обязано кодировать всю эпистемическую жизнь узла. Оно не заменяет тип узла, не заменяет провенанс и не заменяет внешние механизмы неопределённости, такие как Beta-модель реакций или флаги `need_to_*`. Поэтому вопрос “почему бы не хранить всё в одном идеальном скаляре” почти всегда поставлен неверно.

Определение 4.9 (Активация)**.** *Активация* $a_v(t) \geq 0$ — краткоживущая мера того, насколько узел “подсвечен сейчас”. Активация относится к локальной динамике памяти, а не к долговременной вере. Она может возрастать из-за запроса, обхода, совместной активности соседей или внутренней заинтересованности системы.

$$a(t+1) = \lambda \tilde{P}^{\top} a(t) + s(t), \qquad \lambda \in (0,1), \tag{4.2}$$

где $\tilde{P}$ — матрица переходов, а $s(t)$ — внешний или внутренний вброс активации. Эквивалентно об этом можно думать как об ожидаемой частоте посещений узла роем в коротком окне. Активацию важно не путать ни с уверенностью, ни со стационарным распределением π: первое отвечает за убеждение, второе — за долгосрочную геометрию потока.

Определение 4.10 (Валентность и настроение)**.** *Валентность* — аффективная окраска узла. Она отвечает не за то, насколько узел истинный, а за то, как он переживается и как он взаимодействует с настроением. В книге обычно предполагается двумерное описание “валентность–возбуждение”, но точная параметризация вторична по отношению к самой идее аффективной несущей. *Настроение* mood — отдельный низкоразмерный вектор состояния, сцепленный с графом двусторонне, но не живущий внутри каждого узла по отдельности.

Определение 4.11 (Значимость)**.** *Значимость* (`importance`, `salience`) — мера того, насколько объект памяти важен для дальнейшей судьбы системы. Значимость влияет на консолидацию, забывание и приоритеты переработки. Она может частично зависеть от эмоционального сдвига, от роли объекта в долгих паттернах и от структурной центральности. В качестве дешёвого прокси значимости удобно использовать величину изменения настроения $|\Delta\mathrm{mood}|$; более глубокий, но и более дорогой сигнал — ошибка предсказания.

4.4. Операторы над памятью

Определение 4.12 (Random Walker)**.** *Random Walker* — блуждающий агент, который движется по графу по вероятностному правилу, зависящему от структуры рёбер, релевантности, текущего настроения, неопределённости и других

факторов. В типовой форме:

$$P(A \to B) \propto w_{AB} \cdot \mathrm{rel}(B) \cdot \mathrm{mc}(B) \cdot \mathrm{unc}(B)^{\beta_u},$$

где w_{AB} — вес перехода, $\mathrm{rel}(B)$ — релевантность узла текущему запросу или сцене, $\mathrm{mc}(B)$ — mood-congruent bias, а $\mathrm{unc}(B)^{\beta_u}$ — смещение к неопределённому содержанию в тех режимах, где это нужно. Один walker даёт локальную траекторию; множество walker'ов даёт статистическую картину потока по памяти.

Функционал одного walker'а в минимуме таков: (i) перейти по вероятностному правилу; (ii) с вероятностью α *телепортироваться*, то есть перезапуститься из другого узла; (iii) повысить активацию посещённого узла; (iv) по условию запустить операцию вроде retrieval, fusion или инспекции `need_to_*`; (v) записать след как последовательность посещённых узлов.

Определение 4.13 (Рой обходчиков)**.** *Рой* (*swarm*) — набор walker'ов, работающих параллельно под общей политикой. Именно рой, а не одиночный маршрут, лучше соответствует идее распределённой активации памяти. На достаточной статистике следов рой даёт и эмпирическую картину маршрутов по графу, что особенно важно для экспериментов.

Определение 4.14 (Три политики обхода)**.** Один и тот же движок обхода должен уметь работать как минимум в трёх режимах: *retrieval* под запрос, *внутренняя жизнь / любопытство* с повышенным интересом к неопределённому и *сон*, где включается более широкое стохастическое покрытие с телепортом. Эти режимы различаются не типом сущности, а параметрами той же машины.

Определение 4.15 (Retrieval)**.** *Retrieval* — извлечение содержательно релевантной части памяти. Здесь это не один шаг, а двухходовая процедура: сначала находятся *якоря* быстрым индексом, затем от якорей запускается ограниченное расширение окрестности посредством обхода. Этот порядок принципиален: walker хорош для graph expansion, но плох как механизм поиска входа. В результате retrieval зависит не только от семантической близости запроса, но и от настроения, то есть допускает mood-congruent recall.

Определение 4.16 (Fusion)**.** *Fusion* — механизм рождения нового узла из согласованной конфигурации совместно активированных посылок. Обычно результат fusion — гипотеза или обобщение с ограниченной уверенностью и явным провенансом. Для выводных узлов принципиально: результат fusion не стартует как "окончательный факт", а сохраняет провизорный статус до дополнительной проверки.

Определение 4.17 (Консолидация)**.** *Консолидация* — процесс перехода от множества частных следов к более устойчивым структурам памяти. Она может включать объединение дублей, summary-overlay, рождение обобщений и более позднее огрубление графа. Важно, что консолидация — не один жёсткий оператор, а семейство подопераций: fusion/вывод, summary-overlay, dedup/merge и только на более позднем этапе разрушающее огрубление.

Определение 4.18 (Забывание)**.** *Забывание* — ослабление или удаление тех элементов памяти, которые не поддерживаются ни недавним использованием, ни значимостью, ни более глубокой структурной ролью. В этой архитектуре забывание не противоположно консолидации, а образует с ней единую динамику перераспределения памяти во времени. На базовом уровне его удобно связывать с частотой и свежестью доступа, то есть с медленно меняющейся базовой активацией.

Определение 4.19 (Сон)**.** *Сон* — фоновый режим памяти, в котором обход становится более стохастическим и более широким, чем в дневном retrieval. В этом режиме происходят скраб, пересмотр противоречий, консолидация, случайные инспекции и иногда неожиданные дальние fusion-связки. Тот же режим удобно использовать и для подтягивания внешнего материала по флагам `need_to_discover`.

4.5. Сцепленные с памятью процессы

Определение 4.20 (Настроение)**.** *Настроение* mood — отдельный вектор состояния, который не хранится внутри каждого узла, но двусторонне сцеплен с графом. Настроение влияет на маршруты активации и на форму выражения; граф, в свою очередь, поставляет настроению материал и обратную связь.

Определение 4.21 (Личность)**.** *Личность* — не таблица жёстко прошитых весов, а способ, которым мотивация, настроение, эпистемическая осторожность и поведенческие приоритеты смещают работу памяти и её последующее выражение. Внутри архитектуры личность проявляется не как отдельный store, а как bias во время обхода и как стиль на выходе.

Определение 4.22 (Голос)**.** *Голос* — языковой слой, который превращает внутренний контекст памяти в конкретную реплику. Голос не хранит сами факты вместо памяти, а композирует высказывание из того, что память уже подняла и подготовила. Часть характера живёт именно здесь; факты, provenance и эпистемические отношения живут в графе.

4.6. Стартовое состояние и инженерная перспектива

Определение 4.23 (Эмбриональный граф)**.** *Эмбриональный граф* — стартовое заполнение памяти из внешних баз знаний и здравого смысла, прежде всего в духе ConceptNet + ATOMIC. Это не готовая личность и не законченная память, а начальная, нейтральная по характеру среда, в которой могут запускаться собственные механизмы retrieval, fusion, консолидации и ревизии.

Определение 4.24 (Stage 1)**.** *Stage 1* — первый рабочий вертикальный срез системы, в котором теоретическая модель памяти доводится до минимально наблюдаемой инженерной машины: с event log, write-path, typed node store, retrieval, prompt packing, инспектором памяти и синтетическими тестами (глава 20).

Почему не вводится более богатая эпистемическая тройка

В исходной архитектуре сознательно удерживается знаковый скаляр $\text{conf} \in [-1, 1]$, а не полный кортеж belief/disbelief/uncertainty. Это решение не объявляется окончательно истинным: более богатая субъективная логика остаётся допустимой альтернативой, если разнесённые сигналы неопределённости окажутся неудобными. Но на текущем этапе roadmap делает ставку именно на скаляр confidence, provenance, `node_kind` и дополнительные специальные поля неопределённости там, где они реально нужны.

4.7. Что должно остаться после этой главы

После этой главы полезно удерживать три различения.

1. **Узел не равен строке текста.** Это содержательный объект с историей и состоянием.
2. **Активация не равна уверенности.** Первая отвечает за фокус сейчас, вторая — за долговременное убеждение.
3. **Оператор не равен полю хранения.** Retrieval, fusion, консолидация и сон — это процессы над графом, а не разные виды записей в одном и том же столбце.

На этом словарь готов. Следующая глава уже может говорить не об именах объектов, а о самом субстрате памяти как инженерной конструкции: какие поля действительно нужны графу и почему именно они.

5. Субстрат: типизированный темпоральный граф

5.1. Что такое узел и что такое ребро

Память Рин — направленный граф $G = (V, E)$, в котором узлы несут содержание (воспоминания, факты, сущности, концепты), а рёбра — типизированные и знаковые отношения между ними. Ключевая мысль главы: и узлы, и рёбра несут *метаданные*, и именно метаданные, а не сам факт связи, делают возможными все дальнейшие механизмы.

Определение 5.1 (Узел памяти)**.** Узел $v \in V$ — это запись с содержанием $\mathbf{x}_v$ и набором полей состояния, над которыми работают операторы памяти. Узел одновременно является и «данными» (что помнится), и «убеждением» (насколько это считается истинным), и «следом» (насколько активно и свежо).

5.1.1. Схема узла

Поле	Тип	Смысл и кто с ним работает
`id`	int	идентификатор
`content`	text	содержание в естественном виде
`embedding` $\mathbf{x}_v$	vec$[d]$	векторное представление; retrieval-релевантность, мост баз
`node_kind`	enum	вид узла (observation/claim/hypothesis/preference/…); задаёт смысл `confidence` (см. раздел 5.3)
`confidence` c_v	$[-1, 1]$	для truth-apt узлов — знаковое убеждение ($+$да/$-$нет/0хз); для остальных — надёжность оценки (раздел 5.3)
`valence`	vec$[2]$	валентность–возбуждение; сцепка с настроением
`layer/ts`	enum/time	масштаб (сессия…жизнь) и/или метка времени; консолидация
`provenance`	list	источники и/или обоснования; TMS, ревизия
`need_to_*`	flags	«исследовать/пересмотреть»; любопытство, скраб
`importance`	$[0, 1]$	значимость: ставится salience при кодировании (гл. 16), подкрепляется центральностью; гейтит прунинг (гл. 11)
`domains`	tags	членство в доменах знаний; несёт importance *по членству*, а не диффузией (гл. 11)
`activation` a_v	float	транзиентная активация; обход
`degree/centrality`	float	производные структурные; топология

5.1.2. Схема ребра

Ребро $(u \to v)$ несёт направленность (`src`,`dst`), тип отношения `type` (isA, partOf, supports, contradicts, associates, …) и **два независимых числовых канала** (см. определение в главе 4): *вес перехода* $w_{uv} \geq 0$ (нормируется по узлу, для обхода) и *знак* $s_{uv} \in \{+1, -1\}$ (полярность отношения, для пропагации уверенности). Знак — *хранимый лейбл* (внутреннее свойство связи: «эта связь за или против?»), а не вычисляемая величина: его нельзя «посчитать по пути», он известен при создании ребра. Изменение уверенности Δconf — уже *производное*, оно вычисляется в пропагации *с использованием* знака (глава 14).

Веса — про свидетельство, а не про характер

Забегая вперёд (глава 16): вес ребра выражает *эвиденциальную* силу связи и уверенность, а *не* «характер». Соблазн задать личность статическими множителями на рёбра-реакции («грусть→сарказм $\times 2{,}0$») мы разберём и отвергнем: характер должен входить как *смещение во время обхода* от текущего состояния, а не как вшитый в граф множитель.

5.2. Почему рёбра направленные, типизированные и знаковые

Плоская модель «узлы и связи одинакового сорта» ломается сразу на трёх вещах: направленность (асимметрия причинности и времени), тип (нельзя валить covalent-связь, близость, «подтверждает» и «является частью» в один линейный канал) и знак (без него не отличить «всё усилилось» от «всё перевернулось»).

Правильный оператор поэтому не общий лапласиан сглаживания, а *relation-specific* передача сообщения. Сообщение по ребру:

$$m_{u \to v} \;=\; \phi_{r_{uv}}\big(h_u,\, h_v,\, e_{uv},\, \Delta t,\, c_{uv},\, s_{uv}\big), \tag{5.1}$$

где r_{uv} — тип отношения, Δt — возраст/временная дистанция, c_{uv} — уверенность, s_{uv} — источник/эпистемический статус. Затем узел агрегирует входящие сообщения — и здесь важно не одну статистику, а набор (сумма/среднее/макс/ дисперсия по типам отношений), потому что для памяти существенны и кратность подтверждений, и пропорции, и наличие экстремального свидетельства.

Почему одна статистика теряет смысл

Ключевое свойство *среднего* — инвариантность к дублированию. Пусть про факт есть свидетельства $\{+, -\}$ (один за, один против) и в другом случае $\{+, +, -, -\}$ (двое за, двое против). Считая $+ = 1, - = -1$, получаем $\text{mean}\{+, -\} = 0 = \text{mean}\{+, +, -, -\}$ — среднее *одинаково*, хотя во втором случае свидетелей вдвое больше. Для памяти это принципиально: «четверо разделились» — не то же, что «один на один» (устойчивее, эпистемически весомее). *Сумма* же сохраняет кратность (в паре с числом слагаемых), а *макс* по знаку — наличие сильного контрсвидетельства. Отсюда правило: агрегатор хранит *набор* статистик (сумма, среднее, макс, дисперсия), а не одну, — иначе теряется либо кратность, либо пропорция, либо экстремум.

Аналогия с электрической цепью

Полезно думать о графе как о цепи с проводниками разного сорта. Направленность — это диоды; тип ребра — материал проводника со своей проводимостью; знак — это полярность. Плоская модель — цепь, где все элементы заменены одинаковыми резисторами: она проведёт ток, но потеряет всю логику схемы.

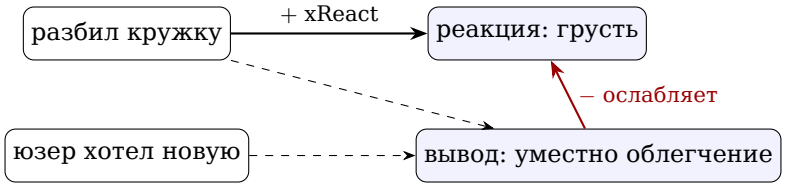


Рис. 5.1: Противоречие не самостоятельное, а *контекстное*. Базовое +xReact «разбил → грусть» ослабляется не узлом «хотел новую» самим по себе, а *совместным* выводом «разбил + хотел новую → уместно облегчение» (пунктир — провенанс; это n-арная конфигурация, глава 10). Именно она делает возможным уместное «поздравляю».

5.3. Виды узлов и что значит уверенность

Раньше мы говорили «узел — это одновременно данные и убеждение». Красиво, но для реализации грубо: **не всякий узел — утверждение об истинности**. «Португалия имеет такой-то режим магазинов», «этот вывод поддержан источником», «гипотеза House / Cast Away ложна» — да, это truth-apt утверждения, и для них conf $\in [-1, 1]$ означает истинность. Но «Ярославу заходит такой подкол», «Рин не любит такой тон», «этот человек устал», «у Анны есть кот» — это предпочтения, реакции, склонности, состояния, и «уверенность 0,8, что Ярослав любит X» — *не* то же самое, что «вероятность истинности факта 0,9». Предпочтения меняются, зависят от контекста, имеют исключения.

Поэтому у узла есть поле `node_kind`, и оно *задаёт смысл* `confidence`:

node_kind	**что это**	**смысл confidence**
`observation`	прямое наблюдение	истинность (высокая)
`claim`	утверждение о мире	истинность
`hypothesis`	вывод/гипотеза	истинность (низкая, провизорно)
`episode`	конкретное событие	было/не было
`preference`	вкус/склонность	надёжность оценки (не истинность!)
`reaction_pattern`	«на X реагирую Y»	надёжность (Beta-постериор, гл. 16)
`relationship_state`	состояние отношения / *кто-что-знает*	надёжность модели
`boundary`	граница/нежелание	устойчивость
`skill/procedure`	навык/как-делать	освоенность
`self_model`	убеждение о себе	надёжность самооценки
`mood_summary`	свёртка настроения	— (это метка, не убеждение)

Для не-truth-apt видов `confidence` читается как *надёжность/устойчивость оценки*, а не «вероятность истинности». Предпочтения и реакции лучше хранить как *модель с условиями* (при таком-то контексте — так), а не как голый скаляр истинности.

Принцип 5.1 (Confidence приора = source_trust × kind, а не плоская константа)**.** Импортированным узлам (эмбрион: ConceptNet, ATOMIC) нельзя давать *плоскую* уверенность на всё — это дважды неверно. Baseline должен зависеть от двух осей:

$$\text{confidence(node)} \;=\; \text{source_trust(source)} \times \text{baseline(}\texttt{node_kind}\text{)}.$$

По kind: ConceptNet-концепт — truth-apt `claim`, стартует выше; ATOMIC-эпизод — социально-каузальная *оценка реакции*, стартует ниже, потому что это надёжность мягкого прогноза, а не истинность (человеческие реакции сложнее фактов). *По источнику*: разные базы заслуживают разного доверия. Два принципиальных решения:

- **Источник — это провенанс, а не метка.** `source` уже несётся в провенансе (`/c/en/…` vs `atomic`); не надо плодить эпистемические метки `prior_conceptnet`/`prior_atomic` — метка (`prior`) это *эпистемический класс* для packer'а, а источник это ортогональные метаданные для калибровки. Иначе метки взрываются комбинаторно, а LLM источник в метке и не нужен.
- **Не выводить node-confidence из веса ребра.** Вес ConceptNet — корроборация *отношения* (канал ребра), а не истинность узла; каналы мы развели специально (определение ребра). Вес гонит диффузию, confidence узла — про доверие к содержанию.

source_trust обучаем, но откалибровать нечем до наблюдений

Сейчас `source_trust` — грубый прайор (ConceptNet > ATOMIC, обоснованно по kind), не калиброванное число. *Обучаемым* он становится, когда появится поток наблюдений (write-path, гл. 18): эпистемический слой понижает доверие к источнику, чьи приоры *опровергаются* наблюдением, и повышает при подтверждении — так «монитор ловит», что источник систематически врёт, а Рин «понимает» ненадёжность источника, читая `source_trust`, а не по отдельной метке. Калибровать в вакууме (без downstream-сигнала) нельзя — поэтому *сейчас* фиксируем лишь *структуру* (baseline по source×kind, source в провенансе), а обучение доверия откладываем до наблюдений.

Почему знаковый скаляр, а не тройка мнения

Напрашивается более богатое представление — *тройка субъективной логики* (belief/disbelief/uncertainty), где неопределённость несёт собственную координату. Мы сознательно берём один знаковый скаляр $c \in [-1, 1]$ (эквивалент $2P{-}1$: $+1$ «да», -1 «нет», 0 «не знаю»). Причины две. *Первое*: скаляр уже разводит две вещи, которые чаще всего путают, — *неверие* ($c < 0$) и *незнание* ($c \approx 0$); именно ради этого мы и ушли от $[0, 1]$. *Второе*: третью ось — «сколько за этим свидетельств / насколько оценка устойчива» — мы и так несём *отдельно и разнотипно*: через `node_kind`, через

`need_to_discover`, через Beta-неопределённость реакций (гл. 16) и провенанс. То есть неопределённость у нас не потеряна — она просто не запихнута в один кортеж, а живёт в подходящих для каждого вида полях. Тройку держим в уме как альтернативу на случай, если разнесённые сигналы окажется неудобно сводить вместе.

Кто-что-знает — тоже вид убеждения

Особо отметим `relationship_state` в смысле *моделирования чужих знаний*. «Рин знает X», «Ярослав знает, что Анна сказала X», «Анна не в курсе, что Рин это запомнила» — это разные узлы: убеждения Рин *о знаниях других*. Это не про приватность-как-политику (её мы отдельной несущей системой не строим), а про то, что theory-of-mind — законный и важный вид памяти: без него Рин не отличит «я это знаю» от «он думает, что я этого не знаю».

5.3.1. От словаря к поведению: операционная матрица

Список видов — это словарь; реализация спросит *поведение*. Каждый `node_kind` обязан ответить на семь вопросов: (1) какие операторы его *читают*; (2) какие могут в него *писать*; (3) как считается `confidence`; (4) как работает *распад*; (5) какие *типы рёбер* допустимы; (6) можно ли его *fusion-ить*; (7) можно ли показывать LLM в context-packer (гл. 18).

Полную матрицу 11×7 заполнять сейчас — значит выдумывать точность, которой нет: часть клеток прояснится только на стенде. Поэтому ниже — *каркас* с наиболее решающими столбцами и уверенными клетками; прочее помечено «TBD».

kind	**смысл confidence**	**распад**	**fusion?**	**в packer?**
observation	истинность	медленный	да (посылка)	да
claim	истинность	медленный	да	да
hypothesis	истинность (низкая)	быстрый, если не подтв.	да	да, с меткой «гипотеза»
preference	надёжность оценки	по использованию	осторожно	да
reaction_pattern	Beta-надёжность	по успеху/провалу	нет (это модель)	как контекст
relationship_state	надёжность модели	медленный	осторожно	по ситуации (TBD)
boundary	устойчивость	очень медленный	нет	да
self_model	надёжность самооценки	медленный	да (метакогниция)	да
mood_summary	— (метка)	транзиентно	нет	редко

Столбцы «кто читает / кто пишет / допустимые рёбра» определяются там, где живёт соответствующий оператор, а не дублируются здесь. Матрица — это

контракт поведения на тип, и она сознательно неполна: заполняется по мере постройки.

5.4. Провенанс и обоснование

Поле `provenance` шире, чем «источник». Для *исходных* фактов провенанс — откуда факт узнан. Для узлов, *порождённых выводом* (fusion), провенанс должен хранить *обоснование* — множество родительских узлов, из которых вывод получен. Эта деталь окажется критической в главе 13: именно наличие обоснований позволяет, опровергнув посылку, автоматически пометить на пересмотр её следствия. Без записанных обоснований опровержение посылки не находит потомков — и рождается «ползучая вера».

5.5. Что значит «коммит» и кто его гейтит

Важнейшая граница схемы — между *активным рабочим убеждением* и *записью*. Не всё, что всплыло или сгенерировалось, попадает в граф как факт.

Принцип 5.2 (Гипотеза, а не факт)**.** Всё, что порождено выводом или генерацией (fusion, реакция, ассоциация), пишется *провизорно*: с низкой уверенностью, с провенансом-обоснованием и, при необходимости, с флагом `need_to_discover`. Повышение до «записи» (переход в более медленный слой) происходит только через накопление подтверждающих свидетельств. Управление — на границе коммита.

```
def commit(G, item, kind):
    if kind == ”observed”:          # прямое наблюдение
        c0 = source_trust(item.source)
    if kind == ”inferred”:          # порождено fusionвыводом/
        c0 = LOW                    # гипотеза, не факт
        item.provenance = item.parents      # обоснование для TMS
    node = G.add_node(item, confidence=c0, layer=”session”)
    propagate(G, node)              # см. главу про лавину со( знаком, карантином)
    return node
```

5.6. Кража схемы у Wikidata

Есть готовая, обкатанная модель данных, уже решившая ровно те задачи, что встают перед метаданными Рин, — модель утверждений *Wikidata*. Брать её *содержание* (энциклопедические факты) для существа, мимикрирующего под человека, не нужно: человек помнит мало фактов, его сила в ассоциативности. Но её *схему* стоит перенять целиком:

- **квалификаторы** (точка во времени, начало/конец) — темпоральная валидность: факт истинен *в интервале*, а не вечно;
- **references** — провенанс/обоснование;
- **ранг** (*preferred*/*normal*/*deprecated*) — *недеструктивная ревизия*: устаревшее не удаляют, а депрецируют, помечая новое предпочтительным; история сохраняется для машинерии поддержки истинности.

Иными словами, факт стоит моделировать как

$$(\text{утверждение}) + (\text{интервал валидности}) + (\text{провенанс}) + (\text{статус/ранг}),$$

и это буквально схема Wikidata. Не изобретать метаданные заново, а взять проверенный шаблон.

5.7. Врождённое и приобретённое: не граница, а континуум

Соблазнительно разделить узлы на «врождённые константы» (эмбриональный граф) и «приобретённые» (нажитые, с временными метками). Как первое приближение полезно, но как жёсткая граница неверно. Консолидация *двигает* записи через границу (эпизод → обобщённое знание), а ревизия должна уметь *понижать* и «врождённое» — иначе унаследованную ошибку здравого смысла Рин не исправит никогда. Правильная картина — *континуум глубины консолидации*, где порог опрокидывания убеждения растёт с глубиной, но нигде не становится бесконечным. К оси «пластичность – упрямство» мы вернёмся не раз (главы 14, 13).

6. Обход графа: один движок, три политики

6.1. Случайное блуждание как универсальный механизм

Почти всё, что Рин делает со своей памятью, — это *обход* графа роем случайных блуждающих агентов, распространяющих активацию. Retrieval, внутренняя жизнь и сон — не три разных движка, а *три политики* одного движка, различающиеся смещением перехода, вероятностью телепорта и набором операций.

Общее правило перехода — нормированное произведение факторов:

$$P(A \to B) \;\propto\; \underbrace{w_{AB}}_{\text{сила связи}} \cdot \underbrace{\mathrm{rel}(B \mid q)}_{\text{релевантность запросу}} \cdot \underbrace{\mathrm{mc}(B \mid \mathrm{mood})}_{\text{конгруэнтность настроению}} \cdot \underbrace{\mathrm{unc}(B)^{\beta_u}}_{\text{тяга к неопределённости}}, \tag{6.1}$$

где mc — из (7.3), а показатель β_u включает/выключает любопытство.

Что такое «температура»

«Температура» — одна ручка: насколько выбор следующего шага случаен против жадного. Формально переход берётся softmax-ом от логарифма правой части (6.1), делённого на температуру T: при $T \to 0$ — жадный шаг по сильнейшему ребру (эксплуатация), при большом T — почти равновероятный выбор соседа (исследование). Термин из имитации отжига; мистики нет.

6.2. Три политики как параметризация

Политика	α (телепорт)	β_u	T	**Что запускает**
retrieval (цель)	0	0	низкая	извлечение под запрос
внутренняя жизнь (любопытство)	мал.	> 0	средняя	inspect, fusion у неопределённых
сон (покрытие)	большой	0	высокая	fusion/inspect у случайных

Retrieval — целенаправленная. Стартует от заякоренных запросом узлов, собирает *окрестность* активированного содержания. Подчеркнём (глава 16): обход собирает *ингредиенты*, а не готовую реакцию.

Внутренняя жизнь — по неопределённости. Смещена к узлам с высоким `need_to_discover`, низкой уверенностью, новизной ($\beta_u > 0$). «Любопытство» — целенаправленный поиск неопределённости, не случайность.

Сон — телепортирующий обходчик. Политика *стохастического покрытия*: с вероятностью α телепорт в случайный узел, иначе — шаг по (6.1). Это случайное блуждание с рестартом (персонализированный PageRank). Его стационарное распределение удовлетворяет

$$\pi \;=\; (1-\alpha)\,P^{\top}\pi \;+\; \alpha\,\mathbf{r}, \tag{6.2}$$

где $\mathbf{r}$ — распределение рестарта. Телепорт делает блуждание эргодическим — это ключевое свойство для эпистемической гигиены (глава 8).

Три способа гулять по городу

Retrieval — идти по адресу (есть цель, выбираешь осмысленный путь). Внутренняя жизнь — идти туда, где интересно и непонятно (тянет к незнакомым переулкам). Сон — время от времени садиться в случайное такси и ехать в случайный район: не потому, что там дело, а чтобы не застревать в одном квартале. Один пешеход, три правила выбора улицы.

6.3. Что walker делает с графом: читает, предлагает, но не коммитит

Естественный вопрос: обходчик *сам* правит граф — сливает узлы, меняет уверенности, ретрактит? Ответ, определяющий всю конкуренцию в системе: **нет.** Walker — это *сенсор-предлагатель*, а не писатель. Мы разводим три роли, которые легко слить в одну и потом мучиться с гонками:

- **чтение** (обход) — walker ходит по графу и собирает активированное содержание; это то, чем занимаются все три политики;
- **предложение** — итог обхода не «правка», а *типизированное предложение операции*: не TODO-текст, а структура вида {`op`: consolidate, `members`:[A, B, C, D], `rationale`, `support_provenance`, `walker_id`, `t`}, которая кладётся в буфер предложений;
- **коммит** — отдельная, *сериализованная* стадия (тот же schema-writer / оператор консолидации, гл. 10) вычитывает буфер и *применяет или отклоняет* операции, дописывая событие в лог (гл. 18, раздел 18.4).

Это ровно наш принцип «выход fusion — гипотеза, а не факт», распространённый на *все* структурные операции: обход рождает кандидата, коммиттер его судит. Зачем такая дисциплина, а не «walker сам чинит граф на месте»: *во-первых*, конкурентная безопасность — структурный писатель ровно один, значит двух одновременных структурных правок не бывает по построению (гонки, гл. 18); *во-вторых*, аудируемость — каждая правка проходит через event-log и объяснима реплеем; *в-третьих*, эпистемика — предложение это гипотеза, а коммит это отдельный гейтируемый шаг, где работает независимость свидетельств и карантин (гл. 14). Walker предлагает — коммиттер располагает.

Единственное, что walker пишет сам — но это тоже запись

Полный запрет на запись был бы неудобен: обход должен оставлять след «здесь были, это подсветили». Поэтому walker *напрямую* трогает только поле активации / яркости a_v (и связанную телеметрию внимания). Точная формулировка важна во избежание тихих багов: это *не* «не запись», а

неструктурная коммутативная запись — не узлы, не рёбра, не уверенности, а состояние внимания, которому всё равно нужны TTL, затухание и `access_reason` (гл. 11). «Коммутативная» значит, что итог не зависит от порядка параллельных записей. Но конкретно для яркости *не* годится аддитивный счётчик: на хабах, через которые проходят сотни walker'ов, $a_v \mathrel{+}= 1$ улетает в космос. Берём max-CRDT: $a_v \leftarrow \max(a_v, e_{\text{rw}})$, где e_{rw} — энергия зашедшего обходчика. Тогда яркость ограничена сверху энергией агента, спад предсказуем, а сотня одновременных подсветок даёт корректный итог без блокировок. Всё, что *структурно*, — только через предложение и коммит.

6.4. Фоновая разведка: один процесс, а не «день/ночь»

Разделение на «дневное любопытство» и «ночной обходчик» — удобная, но неточная рамка. Точнее: есть *один всегда включённый* фоновый разведчик, а «день» и «ночь» — два конца *континуума по свободным ресурсам*, а не переключатель.

Формализуем «мощности, внимание, фокус». Пусть $\rho(t) \in [0,1]$ — *foreground-нагрузка*: насколько текущий момент требует активного ответа ($\rho \approx 1$ в напряжённом диалоге, $\rho \approx 0$ в простое). Общий бюджет B_{total} делится с приоритетом переднего плана; фоновой разведке достаётся

$$B_{\text{bg}} = B_{\text{total}}\,(1-\rho). \tag{6.3}$$

Фокус — это активное множество A_{focus} (узлы высокой активации a_v). Параметры фонового роя — функции B_{bg}: число walker'ов $\propto B_{\text{bg}}$; доля телепортов α и «широта» растут при падении ρ; стартовое распределение интерполирует

$$r = (1-\beta)\cdot \text{anchor}(A_{\text{focus}}) + \beta \cdot \text{uniform}, \qquad \beta \uparrow \text{ при } \rho \downarrow .$$

Тогда прежние «любопытство» и «сон» — это два режима одного процесса: *высокий* ρ (мало бюджета) $\Rightarrow$ мало walker'ов, $\alpha \approx 0$, старт от фокуса — локальная разведка вокруг текущей темы; *низкий* ρ (много бюджета) $\Rightarrow$ много walker'ов, большой α, старт почти равномерный — широкий телепорт-скраб (глава 12).

Мысль на перекуре

Именно поэтому яркая ассоциация может прийти *днём*, «на ровном месте»: фоновый разведчик работает всегда, и даже на скудном дневном бюджете иногда набредает на далёкую связь. Развёрнутый пример такой дневной находки — в главе 10 (House / Cast Away).

6.4.1. Поля активации: быстрое внимание и медленный след — не путать

Здесь легко совершить ошибку — свалить в одно поле «яркость», «свежесть», «долговременную силу» и «важность». Соблазнительно, но потом это поле начнёт делать слишком много, и, что хуже, *диагностика начнёт тренировать память*: скраб, дебаг и тесты будут поднимать ту же переменную, что и настоящее вспоминание, и случайно укреплять узлы. Поэтому разведём *разные по темпу* состояния:

- $a_{\text{work}}(v,t) \in [0,1]$ — *рабочая яркость внимания*. Живёт секунды-минуты, формирует зону фокуса A_{focus} и вход дневных walker'ов, быстро гаснет. Это *неструктурная коммутативная* запись обходчика (max-CRDT выше).
- $B_{\text{base}}(v)$ — *медленная сила следа* (это и есть ACT-R base-level, гл. 11). Живёт дни-месяцы, обновляется *только* настоящими значимыми обращениями (по `access_reason`), влияет на decay / забывание.
- importance(v) — значимость от appraisal / домена / структуры (гл. 16), не путать с частотой обращений.
- coverage_deficit(v) — «давно не инспектировали», рычаг ночного скраба.

Ключевое разделение: настоящее вспоминание поднимает *и* a_{work} (мгновенно), *и* B_{base} (чуть-чуть, медленно); а скраб / дебаг / тест трогают в лучшем случае coverage_deficit и *никогда* не укрепляют след. Правило быстрой яркости:

$$a_{\text{work}}(v, t+\Delta t) = a_{\text{work}}(v,t)\, e^{-\Delta t/\tau(v)} \text{ (между визитами)}, \quad a_{\text{work}}(v) \leftarrow \max\big(a_{\text{work}}(v),\, e_{\text{rw}}\big) \text{ (genuine-виз} \tag{6.4}$$

где $\tau(v)$ *не* универсальна (важные / консолидированные остывают медленнее), а e_{rw} — энергия обходчика (max, не сумма, — иначе хабы взрываются).

max теряет «навязчивость» — возможно, нужен второй канал

Честный компромисс max: он ограничивает яркость (хорошо против хабов), но *теряет повторный фокус в коротком окне*. Сто обходчиков через хаб — max правильно не взрывает; но если один узел *многократно* всплывает в текущем рассуждении, иногда важно знать не только пик, но и *интегральную экспозицию* («это навязчиво лезет прямо сейчас»). Возможное решение — два канала: $a^{\text{peak}}_{\text{work}} = \max(\dots)$ для яркости и отдельный `visit_mass_window` для краткосрочной статистики (он *не* укрепляет B_{base} автоматически). На Stage 1 не обязателен; помечаем как возможное расширение.

Теперь смещение входа обходчиков. Распределение рестарта $\mathbf{r}$ из (6.2) берётся не равномерным, и день с ночью различаются *формой* смещения, а не разными движками:

- **День** (высокий ρ, радиус r_0): $\mathbf{r} \propto a_{\text{work}} \times \text{relevance} \times \text{uncertainty}$. Обходчик эксплуатирует текущий фокус — близкие ассоциации «на сейчас», далеко не отходит.
- **Ночь** (низкий ρ, радиус $r_1 > r_0$): $\mathbf{r} \propto$ uniform + coverage_deficit + `need_to_revise` + low-π регионы. Именно так, а *не* просто «шире от дневного фокуса» — иначе сон останется заложником дневного внимания. Сон нужен, чтобы уйти туда, куда внимание *никогда* не ходит: anti-entropy scrub, телепорт-эргодичность, проверка противоречий (гл. 12).

Лаконично: *день — эксплойт локально у горячего, ночь — исследуй заброшенное*. Поля разные, но связанные: a_{work} ведёт дневной фокус, coverage_deficit и низкая π ведут ночь, B_{base} решает, что переживёт забывание.

Мотыльки на остывающий свет: откуда дневные догадки

Если дневной ретрив — «слепой матричный ток» (раздел 6.6), откуда берутся внезапные дневные озарения? *Не* из matvec. После дневного ответа подсвеченные узлы a_{work} гаснут экспоненциально минутами-часами — остаётся *тёплое послесвечение*. Одиночные фоновые RW (тот самый остаточный бюджет B_{bg}) через формулу рестарта $\mathbf{r} \propto a_{\text{work}} \times \text{relevance} \times \text{uncertainty}$ получают повышенную вероятность телепортироваться именно в эти недавно-горячие зоны — летят на остывающий свет, как мотыльки. Кружа в тёплой области, они сталкивают концепты на своих траекториях и *роняют кандидатов* в thought-inbox (гл. 18). Важно: роняют именно *кандидатов* (в основном шум), которых обязан просеять salience-гейт ($|\Delta\text{mood}|$, ошибка предсказания) — иначе Рин завалит себя мусорным fusion. Так разделение труда честное: matvec подсвечивает, мотыльки ползают по послесвечению, salience решает, что вообще достойно всплыть догадкой.

Где «живут» обходчики

Walker'ы не объекты, которые где-то постоянно «живут». Это фоновый процесс планировщика с бюджетом B_{bg}: каждый обход *рождается* из распределения рестарта $\mathbf{r}$, проходит, оставляет предложения и подсветку — и гаснет. Persist между ними *не сами walker'ы*, а поле яркости a_{work} и буфер предложений. Спросить «где живут обходчики» — всё равно что спросить, где живут волны: волн как вещей нет, есть среда (поле a_{work}) и то, что её возмущает (планировщик).

Как измерять foreground-нагрузку ρ

ρ управляет тем, насколько агрессивно разведчик забирает ресурсы, но как её *мерить* в рантайме — не зафиксировано. Кандидаты: наличие активного пользовательского хода (есть незакрытая реплика $\Rightarrow$ ρ высока); длина очереди foreground-задач; «важность» текущего события (по appraisal, глава 16); загрузка вычислителя. Вероятно, комбинация, но веса и динамика (как быстро ρ спадает в простое) пока открыты.

6.5. Точки входа: как query превращается в якоря

Ретрив у нас двухшаговый: **шаг 1** — найти *точки входа* (якоря) в граф, **шаг 2** — растечься из них диффузией (раздел 6.6). Про шаг 2 сказано много, а про шаг 1 — только «быстрым индексом». Распишем его, потому что это первая строчка кода ретрива.

Сразу уточнение терминологии. Query не «разбивают на эмбеддинги». Его разбивают на *семантические единицы*, и уже каждую единицу отдельно эмбеддят. Это разные вещи.

Кто разбивает — не LLM. Ретрив — рефлекс (System 1, ниже LLM-слота, гл. 18), поэтому превращение query в якоря должно быть дешёвым и *LLM-free* на горячем пути. Делает это *тот же экстрактор, что и на write-path*

(‘raw_event → parse/extract candidates‘, раздел 18.5), только без коммита. Красивая симметрия: *понимание запроса = извлечение кандидатов из события минус запись*. На Stage 1 экстрактор простой: NER + ключевые фразы + лемматизация (для русского морфология обязательна — «плавки / плавок / плавками» сводятся к лемме). LLM подключается к *переформулировке* мутного запроса только на эскалации — редкий System 2, а не норма.

Два сорта якорей. Экстрактор выдаёт две разновидности, и ищутся они поразному:

- *Сущностные / лексические* — именованные сущности и конкретные термины («Азоры», «плавки», «9 сентября»). Им нужен **точный** матч, а не «похожее по вектору».
- *Холистический* — эмбеддинг всего запроса (или его клауз) для ассоциативной диффузии.

Если единиц несколько — это *мульти-векторный* ретрив, а не одно усреднённое. Mean-pool «плавки на Азоры 9 сентября» размазал бы три резкие сущности в кашу; поэтому каждую единицу сажаем отдельным seed’ом, а узлы, подсвеченные *несколькими* seed’ами, получают буст (пересечение = релевантнее).

Чем эмбеддим. Тем же энкодером, что и `node.embedding` при записи (гл. 5). Это не опция: косинус осмыслен, только если query и узлы в *одном* пространстве. Вектор узла уже кэширован (считался один раз коммиттером), так что на шаге ретрива через энкодер гоним только query-единицы, узлы не трогаем.

```
units      = extract(query + active_focus)      # NER + keyphrases + леммы (+ кореф)
anchors    = {}
for u in units:
    if u.is_named_entity:
        anchors |= exact_index.lookup(lemma(u))  # точный/aliasиндекс– (forensic
    подобный–)
    anchors |= ann_index.knn(embed(u), k)        # вектор–ANN (HNSW), top–k по косинусу
anchors    = resolve_entities(anchors)           # упоминания –> канонические узлы
r          = weight_by_match_confidence(anchors) # точное > сильный вектор > слабый
# r –> распределение рестарта для локальной разреженной диффузии шаг( 2)
```

То есть точки входа приходят из *двух* индексов: точного / alias (для именованных сущностей — «Азоры» должны попасть в *тот самый* узел) и вектор-ANN (для семантической близости). Их объединение, взвешенное по уверенности матча, и есть seed-набор. Плюс *entity resolution*: разные упоминания сводятся к каноническому узлу, чтобы «плавки» били в один узел, а не в пять эпизодов покупки.

Дальше. Seed-набор становится *начальным впрыском* $\mathbf{a}(0) = s(0)$ для *локальной затухающей* диффузии (fast-path вокруг якорей, раздел 6.6), а расширение гейтируется условным throttle. Важно: retrieval — это *политика* с телепортом $\alpha = 0$ (таблица политик выше), то есть источник впрыскивается *один раз* и импульс затухает; рестарт (член $s(t{>}0)$, он же $\alpha > 0$ в (6.2)) — это *сон*, а не retrieval. Цепочка целиком: *экстрактор дал якоря → ANN+точный индекс нашли точки входа → импульс затух → packer срезал по порогу*.

Что на Stage 1 сознательно упрощаем

Открыты: гранулярность разбиения (по сущностям / клаузам / предложениям — тюнить на бенче); *кореференс* — «поищи *их*» требует обогатить query текущим фокусом a_{work} *до* эмбеддинга, иначе якорей нет; порог эскалации к LLM-переформулировке; качество alias-резолвинга под русскую морфологию. Механика (два сорта якорей, два индекса, мульти-seed, resolution) — зафиксирована.

6.6. Семантический вентиль: дёшево и без протечек по омонимии

В (6.1) множитель $\mathrm{rel}(B \mid q)$ — это *семантический вентиль*: он не даёт фронту расползтись из зоны запроса. Возникает законный страх за производительность: если на каждом шаге RW считать сходство контента узла с q заново, это катастрофа. Ответ: **на лету ничего не эмбеддим.** Эмбеддинг узла считается *один раз при записи* (коммиттером) и лежит в поле `embedding` (гл. 5). Тогда вентиль на шаге — это скалярное произведение *фиксированного* вектора q на *кэшированный* вектор узла, $O(d)$; а раз ретрив у нас — усечённая *локальная затухающая* диффузия из seed'ов (политика retrieval: телепорт $\alpha = 0$, источник впрыскивается один раз — это *не* PPR; рестарт-форма (6.2) зарезервирована за сном), весь фронт считается одним matvec, где вектор активации на каждом шаге умножается на маску сходства с q. Как только значения падают ниже порога ε — граница замкнулась. Никаких рекурсивно плодящихся агентов: «группа вспышек, слабеющих к окраинам» — это *результат* диффузии, а не объектная механика.

Инженерная честность про этот дневной fast-path: matvec идёт *не* по всему графу. Умножать $\tilde{P}^{\top}$ на $|V|$-мерный вектор на графе в сотни тысяч узлов — лечь. Это *локальная разреженная* диффузия на ограниченном носителе: 2–3 хопа вокруг seed-якорей, найденных ANN-индексом (HNSW), — субматрица смежности окрестности, а не всё V-пространство. И тайминги честные — *миллисекунды*, не микросекунды (основное время — ANN-поиск якорей и context-packer), а настоящее бутылочное горло рантайма всё равно LLM-инференс (секунды), гл. 18.

Две поправки, чтобы вентиль не навредил. *Первая — жёсткость по типу ретрива*: для ordinary-ретрива режем жёстко (защита от мусора), для creative / curiosity — мягко (именно далёкие странные мосты нужны для fusion), для forensic — вентиля нет вообще, это не walker, а точный индекс-lookup (таксономия ретрива, гл. 18). Слишком жёсткий вентиль убил бы серендипность. *Вторая — полисемия*: граница возникает не только из низкого сходства с q, но и из *sense/type-несовпадения*. Классический пример «кружка → ручка → дверь» не должен решаться тем, что «дверь слабо похожа на кружку»: раньше ломается слой смысла. «Ручка кружки», «дверная ручка», «ручка-перо» — это *разные* sense/concept-узлы (или хотя бы квалифицированные рёбра, гл. 19). Без этого ретрив постоянно течёт по омонимии — а для русского это особенно больно. Значит вентиль закрывается по $\max(\text{low-sim},\ \text{sense/type-mismatch})$, а не по одной similarity.

6.6.1. Вентиль — условный throttle, а не плоский множитель

Есть и *третья* поправка, и она важнейшая: как написано выше, $\mathrm{rel}(B \mid q)$ входит в (6.1) *множителем* — а значит зануляет путь при низком сходстве. Это тихо убивает личность. Пример: триггер «гирлянда». Узлы [свет], [провода] семантически близки и проходят; а узел личной памяти [окно в Тбилиси] имеет сходство ≈ 0 — и множитель его гасит, Рин выдаёт энциклопедию. В живой голове [Тбилиси] вспыхивает *первым*: у личного ребра колоссальный вес/валентность, несмотря на семантическую даль. Значит вентиль должен быть *условным throttle*, а не вето:

$$\mathrm{Score}(A \to B) = f(w_{AB}, \mathrm{valence}_B) \cdot \underbrace{\mathrm{Throttle}(B, q)}_{\text{условный}} \cdot \mathrm{mc}(B \mid \mathrm{mood}) \cdot \mathrm{unc}(B)^{\beta_u},$$

$$\mathrm{Throttle}(B, q) = \begin{cases} 1, & \text{ребро из личного слоя (}\texttt{session/life}\text{) } \textbf{ИЛИ } |\mathrm{valence}_B| > \theta_{\mathrm{emotion}}, \\ \mathrm{rel}(B \mid q), & \text{иначе (обычное знание — тормоз против дрейфа).} \end{cases}$$

То есть сильное личное или аффективно заряженное ребро — «синаптический хайвей» — проходится независимо от сходства с запросом; а throttle тормозит только слабые/родовые рёбра, чтобы не было случайного топологического дрейфа.

Три предохранителя, без которых это сломается. *Не по сырому* w_{AB}: пробивать вентиль порогом веса нельзя — у эмбриональных рёбер (ConceptNet) веса из коробки огромны ($w \approx 2{,}5$), и Рин уплыла бы по банальной таксономии. Поэтому гейт по *происхождению* ребра (личный слой, не врождённый каркас), а не по величине веса. $f(w_{AB}, \mathrm{valence})$ *всё равно ранжирует*: throttle$= 1$ *не* значит «все личные узлы хлынут» — Тбилиси всплывает не потому, что ребро личное, а потому, что оно личное *и* с огромным w_{AB}; слабое личное ребро с throttle$= 1$ всё равно проиграет по f. *Стоп-условие три-гейта*: путь глохнет, только если *одновременно* низкое сходство *и* слабое ребро *и* нейтральная валентность; сильна любая из трёх — агент проходит. Сухой-но-важный `boundary` («не шути про доту») проходит через layer-гейт даже при низкой валентности. Омонимию (кружка→ручка→дверь) по-прежнему режет отдельный sense/type-гейт, а не throttle.

Прайминг: подсвечено, но не сказано

База при этом не пропадает: «у гирлянды есть провода» матричная диффузия *подсвечивает* (ток туда течёт) — это прайминг. Но у этих узлов почти ноль ситуативной значимости и новизны, поэтому context-packer их *сознательно не выносит наверх* (раздел 18.7). Они «тёплые» на фоне сознания, но не доходят до языка — ровно как человеческое подсознание держит очевидное наготове, не проговаривая.

Принцип 6.1 (Вентиль на эмбеддингах — фильтр родственности, не релевантности)**.** *Эмпирически выяснено* (стенд, `reports/experiment_3–4`): множитель $\mathrm{rel}(B \mid q)$, построенный на дистрибутивных эмбеддингах (косинус к вектору запроса), фильтрует *родственность* (co-occurrence), а не *причинную релевантность*. Он давит off-topic дрейф (годится для запросов вида «что есть X» / «кто из этих X»), но на «почему»-запросах систематически недобирает: узел может быть ближе к слову запроса как *ассоциат*, не будучи *ответом* («beverage» ближе к «energy», чем «caffeine», из-за маркетингового со-упоминания). Это ограничение *инструмента*, а не бага; для причинного выбора нужен не косинус, а рассуждение.

Принцип 6.2 (Конвергенция на хабе пересиливает per-edge релевантность; лечится редкостью, не гейтом)**.** *Эмпирически выяснено* (там же): при диффузии узел, достижимый *из нескольких* якорей сразу, накапливает их вклады, и это суммирование может перевесить более высокий per-edge $\mathrm{rel}(B \mid q)$ у односвязного правильного ответа — даже когда вентиль оценил ответ выше. То есть провал «генеричный дистрактор победил» решает *конвергенция на хабе*, а не релевантность; это тот же зверь, что концентрация π на хабах (гл. 8). Значит корректив — *отдельный штраф за редкость/степень* (IDF-подобный: генеричный общий сосед давится), а не докрутка вентиля. Один такой член чинит сразу три наблюдения: дистрактор-хаб, таксономический дрейф ретрива и хаб-концентрацию.

Принцип 6.3 (Ретрив отвечает за recall, LLM — за причинный выбор)**.** Отсюда критерий успеха ретрива: не «правильный узел *первый* в ранге», а «правильный узел *попал* в упакованный контекст в рамках бюджета при приемлемой доле дистракторов». Граф — ассоциативная память (поднять кандидата в окрестность); *причинное* ранжирование («из этих caffeine — и есть *почему*») делает LLM на собранном контексте. Требовать от графа самому назвать причинный ответ — значит возлагать рассуждение на recall.

6.7. Единство и следствие

Один субстрат, один блуждающий агент, три параметризации: смещение перехода, вероятность телепорта, набор операций. Реализуется одна машина обхода с параметрами (псевдокод — приложение C). Родственные механизмы: spreading activation (Collins–Loftus) как модель ассоциативного припоминания и персонализированный PageRank / RWR как его вычислительная форма; в волте RIN — под именами SYNAPSE и GAAMA.

7. Настроение

7.1. Настроение — отдельный сцепленный модуль, а не поле графа

Снимем путаницу сразу. Настроение *не* вычитывается из спектра графа памяти. Это отдельное состояние — низкоразмерный вектор в пространстве аффекта $\text{mood} \in \mathbb{R}^k$ (например, оси энергичность / раздражённость / экзальтация / любопытство) — отдельный модуль, слой, через который проходит извлечённое содержание.

Путаница шла от слова «спектр». Есть два разных спектра, и их нельзя смешивать: *спектр лапласиана графа* (структура памяти, глава 8) и *частотное разложение настроения во времени* (быстрые и медленные компоненты аффективного сигнала). Многослойность настроения относится ко второму и к графу отношения не имеет.

7.2. Инерция и многослойность

У настроения должна быть *инерция*: оно зависит и от события, и от собственного прошлого значения:

$$\text{mood}_t = f\big(\text{mood}_{t-1}, \text{event}_t\big), \qquad \text{action}_t = g\big(\text{mood}_t, \text{event}_t\big). \tag{7.1}$$

Это протекающий интегратор (11.1): инерция — это его рекуррентность (состояние зависит от прошлого состояния).

Почему интегратор, а не выученная динамика

Форму f можно было бы *выучить* — небольшой RNN/GRU поверх потока событий. Мы берём фиксированный протекающий интегратор по трём причинам. *Интерпретируемость*: у него ровно одна ручка на слой — постоянная времени τ_ℓ (как быстро забывает), и её смысл читается сразу; у выученной динамики «почему настроение поехало» — это скрытое состояние, которое не отладишь глазами. *Отсутствие обучения и данных*: RNN надо на чём-то тренировать, а размеченных траекторий настроения Рин у нас нет и не предвидится. *Достаточность*: главные наблюдаемые свойства — инерция, возврат к базовому уровню и разные скорости на разных горизонтах — интегратор даёт напрямую, одним стеком с разными τ_ℓ. Выученную динамику держим как *апгрейд на потом*: если стенд покажет, что нужны нелинейные эффекты (насыщение, гистерезис аффекта), которые линейный интегратор не тянет.

Интегратор и вектор — одно и то же, с двух сторон

Чтобы снять путаницу: «протекающий интегратор» и «вектор настроения» — не разные сущности. Интегратор — это *правило обновления* (11.1), а его *состояние* — вектор в пространстве аффекта. Многослойность настроения — это *стек* таких интеграторов с разными τ_ℓ; «старший вектор» и «младший вектор», о которых мы говорили, — это ровно *состояния* медленного и быстрого интеграторов. Актуальное настроение — их суперпозиция. И, как отмечено в разделе 11.2, эти слои *параллельны* (все питаются одним потоком событий, переноса между ними нет) — в отличие от слоёв памяти, где перенос есть.

Формально: стек интеграторов с разными τ_ℓ, актуальное настроение — их взвешенная суперпозиция:

$$\text{mood}_t^{(\ell)} = \Big(1 - \tfrac{1}{\tau_\ell}\Big)\text{mood}_{t-1}^{(\ell)} + \tfrac{1}{\tau_\ell}\,\text{appraise}(\text{event}_t), \qquad \text{mood}_t = \sum_\ell w_\ell\,\text{mood}_t^{(\ell)}. \quad (7.2)$$

Младший слой (малое τ) даёт высокочастотную компоненту большой амплитуды — «дрожит сильнее». «Суперпозиция» здесь — про частоты во времени, а не про граф.

7.3. Setpoint: медленный слой как аттрактор

Медленный слой — не просто демпфер, а *точка притяжения*. Добавляя к (11.1) возврат к базовому уровню μ, получаем затухающую к μ динамику:

$$\text{mood}_t^{(\text{life})} = \mu + (1-\kappa)\big(\text{mood}_{t-1}^{(\text{life})} - \mu\big) + \ldots, \quad 0 < \kappa < 1.$$

Здоровая динамика — отрицательная обратная связь, тянущая настроение домой. Патология — когда сдвинулся сам μ: депрессия в этой модели не длинная экскурсия быстрого слоя, а *уехавший аттрактор*.

7.4. Настроение зависит от графа, и граф — от настроения

Связь двусторонняя, и именно она делает настроение частью *мышления*.

- **Граф $\to$ настроение.** Извлечённые узлы несут валентность; она и есть event_t в (7.1).
- **Настроение $\to$ граф.** Настроение смещает вероятности переходов при обходе: узлы с валентностью, близкой к текущему настроению, получают больше активации:

$$\text{mood_congruence}(B \mid \text{mood}) \;=\; \exp\big(-\gamma\,\|\,\text{valence}(B) - \text{mood}\,\|^2\big). \quad (7.3)$$

Это *конгруэнтное настроению припоминание*: грустный чаще вспоминает грустное. Настроение остаётся собственным состоянием, сцепленным сбоку, а не полем графа.

7.5. Опасность петли и три тормоза

Руминация

Конгруэнтное припоминание (7.3) плюс инерция (7.1) дают петлю с положительной обратной связью: грустная вспоминает грустное → грустнеет → вспоминает ещё более грустное. Устойчивость петли определяется её петлевым усилением g: при $g \geq 1$ спираль расходится. Медленный слой уменьшает эффективное g — вот его стабилизирующая роль.

Что удерживает живой ум? Три *независимых* тормоза; патология — когда откажут все три сразу:

1. **Setpoint/гомеостаз.** Медленный слой-аттрактор снижает петлевое усиление ниже единицы.
2. **Метакогнитивное разотождествление.** Акт «почему я так часто злюсь?» переводит из «быть-в-настроении» в «наблюдать-настроение» — второй порядок, и переход рвёт петлю первого порядка.
3. **Экзогенное прерывание.** Мир вламывается ортогональным событием; внимание конечно, внешний вход вытесняет петлю.

Инженерное требование: (i) член setpoint; (ii) монитор «валентность залипла слишком долго» — высокая автокорреляция mood_t — с правом запустить корректив (переопределение, переключение внимания, вызов сна); (iii) всегда-прерываемая экзогенная значимость. Условие безопасности: три тормоза не могут быть отключены одновременно. Псевдокод монитора — в приложении C.

Почему многослойность — это стабилизатор, а не украшение

Пусть быстрый слой попал в негативную спираль с усилением $g_{\text{fast}} = 1{,}1$ (расходится). Актуальное настроение — суперпозиция, и медленные слои вносят устойчивую компоненту с $g \approx 0$. Если их вес w_{slow} достаточен, эффективное усиление суперпозиции $\bar{g} = w_{\text{fast}} g_{\text{fast}} + w_{\text{slow}} \cdot 0 < 1$ — спираль гасится за счёт «тяжёлого маховика» медленных слоёв. Так многомасштабность оправдана *функционально*: это демпфер руминации, а не только реализм.

7.6. Метакогниция

Один из трёх тормозов руминации выше — «метакогнитивное разотождествление» — был назван мимоходом, но это несправедливо: **метакогниция — самостоятельная фича всего проекта**, а не побочный эффект. Ей место в отдельном разделе.

Определение 7.1 (Метакогниция)**.** *Метакогниция* — способность Рин наблюдать и рассуждать о *собственных внутренних состояниях*: о своём настроении, об уверенности в своих убеждениях, о пробелах в памяти. Это операции *второго порядка* — не «что я думаю о мире», а «что я думаю о своём думании и своём состоянии».

Метакогниция не само собой разумеется: базовая модель по умолчанию её не имеет. Это проектируемая возможность, и у неё несколько конкретных ролей, разбросанных по всей архитектуре, — здесь мы их собираем.

- **Тормоз руминации.** Наблюдение «почему я так часто злюсь?» переводит из режима «быть-в-настроении» в «наблюдать-настроение» и рвёт петлю первого порядка (раздел выше). Технически — это монитор автокорреляции mood_t, который *сам замечает* залипание и запускает корректив.
- **Эпистемическая честность.** «Похоже на X, но Y я не проверила» — это метакогнитивный отчёт об уверенности: Рин читает поле conf и зону незнания ($|\text{conf}| < h$) и *говорит* о них, вместо ложного «я уверена» (глава 13).
- **Порождение любопытства.** Замечая собственный пробел или высокую ошибку предсказания, метакогниция ставит `need_to_discover` — то есть *сама формулирует*, чего Рин не знает и что стоит проверить (главы 13, 10).
- **Самоотчёт.** Способность сказать пользователю о своём состоянии («устала от одного и того же бага», «мне сейчас не до шуток») — это метакогниция, считывающая настроение и мотивацию и выражающая их через Rinness (глава 17).

Субстрат метакогниции: самотелеметрия

Метакогниции *не нужен* отдельный граф. Её субстрат — *лог самонаблюдения*: кольцевой буфер временного ряда собственных состояний (вектор настроения) и счётчиков команд (`need_to_*`, fusion, contradiction, retrieval) за часы/дни/недели. Словарь внутренних команд и состояний *конечен и дискретен*, поэтому телеметрия компактна и структурирована, а статистика по ней дёшева. Это не граф и не свободный текст. Но *выводы* метакогниции возвращаются в граф памяти как обычные узлы-убеждения-о-себе (с conf, провенансом = телеметрия- свидетельство) — и дальше живут по общим правилам.

Детекция: дешёвый монитор чисел, а не сверх-LLM

Первый слой — *детекция* — набор дешёвых статистических детекторов на буфере: скользящие среднее/медиана, тренд (наклон регрессии), частота (счёт в окне), аномалия (z-оценка от базовой линии), автокорреляция (залипание). Детерминированно, *без* LLM; срабатывают по порогу/аномалии. Пример: «blue: медиана $0{,}7$ > среднего $0{,}5$ за 5 дней $\Rightarrow$ несколько всплесков перетянули среднее, база грустная»; «`need_to_revise`: $+60$% к 30-дневной базе».

В «мозг» Рин уходит *не* сырой ряд (много, и LLM ненадёжно считает медиану) и *не* готовый вывод, а *компактная структурная сводка* — выход детектора.

Аномалия — это факт в графе, а не спецпайплайн

И вот основной ход, который важнее всего: сводку детектора пишем *в граф памяти как обычный факт-о-себе* («за 5 дней `need_to_revise` $+60$%»; «я в последнее время грустная»). Уверенность высокая — это *измерение*, а не догадка. Дальше — никакого выделенного «метакогниция-LLM» и никакого спецпайплайна «аномалия $\rightarrow$ LLM $\rightarrow$ граф». Факт просто лежит в памяти и подхватывается *тем же общим циклом*, что и любая другая мысль (retrieval, фоновый fusion). Метакогниция здесь — не модуль, а *факты-о-себе, живущие в графе по общим правилам*.

Более того, **сам факт уже работает — вообще без LLM**. Узел «я в последнее время грустная» смещает самомодель, mood-congruent retrieval и самоотчёт («что-то я загрустила») через обычные механизмы. Значительная часть

пользы приходит просто от *записи* факта; объяснение «почему» — опциональный более глубокий слой, а не обязательная стадия.

Объяснение опционально: fusion-про-себя

Когда причина всё же нужна (гипотезы о том, *почему* аномалия), работает fusion-про-себя (глава 10) — но это *тот же общий композитор*, а не прокладка: он запускается обычной salience-обработкой, когда факт-аномалия всплывает. Гипотезы берут посылки из телеметрии: «почему больше `need_to_revise`?» → H1 «много новых рассуждений», H2 «кладу непроверенные факты», H22 «читала книгу с ошибками». Каждая — провизорный узел, *низкая* уверенность (в отличие от факта-измерения — высокой), провенанс из телеметрии.

Проверка — по *различающему предсказанию* (научный метод): H1 ⇒ должен вырасти и fusion-count; H2 ⇒ у пересмотренных узлов короткий интервал commit→revision и пропущенный `need_to_discover`; H22 ⇒ они делят один источник в провенансе. `need_to_discover` кодирует, *какой* запрос выполнить; curiosity/scrub его выполняет. Разрешённая гипотеза оседает в граф; если ничего не разводит — остаётся честное «неизвестно почему», а не выдуманная причина.

Eager или lazy — из salience, а не из расписания

Раз аномалия — это факт, «сразу разбираться или потом» перестаёт быть отдельной развилкой в коде: это *следствие salience* самого факта. Срочная аномалия (спираль настроения, руминация) несёт высокую значимость ⇒ всплывает и обрабатывается общим циклом *немедленно* (корректив сейчас). Несрочная (revise-rate вырос) просто лежит и объясняется *лениво* — когда её достанет retrieval или придёт прямой вопрос о текущем состоянии. Дешёвые детекторы при этом крутятся в фоне всегда (они дёшевы; их интенсивность привязана к ρ-бюджету, глава 6), а дорогой общий LLM тратится лишь на то, что salience подняла наверх. Ни чистый on-demand (руминация подавляет самозапрос именно тогда, когда он нужнее), ни фиксированное расписание (расточительно) не нужны.

Защита от конфабуляции

Самообъяснение — самый склонный к выдумыванию вид рассуждения: люди уверенно конфабулируют причины своих настроений и поступков (классика — Nisbett & Wilson, 1977). Поэтому телеметрия-лог здесь не удобство, а *противоядие*: он делает самогипотезы проверяемыми против реальных данных, а не сочинёнными. Метакогниция без телеметрии = конфабуляция; с телеметрией и различающими проверками = фальсифицируемая.

Само-факты не должны стать самосбывающейся ловушкой

Идея «аномалия — факт в графе» сильна, но у неё есть тёмная сторона. Узел «я в последнее время грустная» (`self_model/mood_summary`), показанный в context-packer как обычный identity-факт, запускает петлю: mood-congruent recall начнёт *сам поддерживать* это состояние. Поэтому такие узлы: (1) диагностичны и *time-bounded* (с явным expiry, а не «навсегда»); (2) пакуются в промпт как *наблюдение о недавней телеметрии*, а не как

черта: «recent telemetry suggests …», а не «Rin is …». Разница между «в последнее время грустится» и «я грустная» — это разница между диагнозом и приговором.

Метакогниция о метакогниции (второй порядок)

Тот же монитор можно навести на телеметрию *самой* метакогниции: частоту порождаемых самогипотез, долю разрешённых, долю опровергнутых. Это *санитайзер* первого порядка: ловит analysis paralysis (много гипотез, ни одна не разрешается → перестать копаться), ненадёжную интроспекцию (самовыводы то и дело опровергаются → доверять самочтениям меньше — прямая защита от конфабуляции) и залипшие петли. Механика та же, просто этажом выше, поэтому дёшево. Третий порядок («думаю о том, как думаю о думании») даёт быстро убывающую отдачу и риск регресса — потолок разумно ставить на втором (аналогия со второй производной в матанализе).

Как это соотносится с литературой

Идея «метакогниция = System-2, наблюдающая System-1» и то, что метамысли *возвращаются в память* и затем влияют на действия, — из Toy, MacAdam, Tabor (2024), *«Metacognition is all you need?»* (поверх генеративных агентов Park и др.). Что мы *берём*: самонаблюдение как отдельный модуль; мета-выводы обратно в память; локальные модели; дешёвая модель для рутины, большая — для метакогниции. Где идём *дальше* (и где вопросительный знак в названии честно себя выдаёт): у них *детекция — это LLM*, оценивающий свой прогресс, а у нас — дешёвый статистический монитор, LLM только на объяснение; у них нет *проверяемых* самогипотез с различающими запросами и защиты от конфабуляции; их метакогниция целевая (задача), наша шире (настроение, эпистемика, гигиена памяти) — то, что они сами относят к future work. Значит «метакогниция — всё, что нужно»? Нет: нужен ещё субстрат самотелеметрии и дисциплина проверяемых гипотез.

Второй наблюдатель

Полезно представлять метакогницию как *второго наблюдателя*, который смотрит не на мир, а на первого — на саму Рин: на её настроение, уверенность, ход мысли. Именно этот второй наблюдатель отличает существо, которое *переживает* состояние, от существа, которое *знает, что переживает его, и может это назвать*. Для проекта, где цель — личность, а не ассистент, это не украшение, а одна из несущих опор.

Глубина и стоимость метакогниции

Открыто, насколько «глубоко» гонять метакогницию: второй порядок («думаю о думании») дёшев и явно полезен, но третий и выше («думаю о том, как думаю о думании») быстро даёт убывающую отдачу и риск зацикливания на самонаблюдении. Где ставить потолок и как выделять на метакогницию ресурсы (тот же ρ-бюджет, глава 6) — пока не зафиксировано.

8. Топология правит потоком

8.1. Тезис главы

У графовых систем вообще есть явная текущая сущность — передача сообщений, и её поток подчиняется топологии, как вода следует рельефу. У графа памяти Рин явной «жидкости» нет. Но поток есть — это поток *изменений*, поток *преобразований* графа: активации, уверенности, консолидации. Тезис: *неважно, какова физика носителя, — сама физика протекания информации подчиняется топологическим законам*. Любой процесс, чьё обновление *локально* (изменение узла зависит от соседей), есть диффузия на графе и наследует его спектрально-топологический транспорт. Это содержание теории графовых сигналов и теории случайных блужданий.

8.2. Стационарное распределение $\neq$ заданные вероятности

Пусть мы локально задали вероятности переходов. Наблюдаемое распределение частот посещений $\mathrm{distr}(A)$ отличается от заданного, потому что вмешивается топология. Формально: *стационарное распределение* π — не локальные переходы, а *глобальный* объект, неподвижная точка оператора переходов:

$$\pi \;=\; P^{\top}\pi, \qquad \sum_{v} \pi_v = 1. \tag{8.1}$$

Для простого блуждания $\pi_v \propto \deg(v)$; для смещённого — перронов собственный вектор $P^{\top}$. Ты задаёшь переходы локально, а долгосрочная частота посещений — свойство всего графа. «Чем длиннее трек, тем наблюдаемее эффект» — это сходимость к π за время перемешивания t_{mix}, управляемое спектральным зазором $1 - \lambda_2$. Короткие блуждания читают *локальную* геометрию, длинные — глобальную π. Телепорт (6.2) π не убирает, а *регуляризует* (сдвигает к смеси «равномерное + рестарт», режет t_{mix}): ограничивает эффект топологии, не отменяя.

Скорость света — плохая, но поучительная аналогия

Можно соблазниться сравнением с c: заданные вероятности как предельная скорость, а среда (топология) искажает наблюдаемое распространение. Рабочая часть — «среда формирует распространение, есть структурный предел» — уже покрыта граф-диффузионной рамкой *точно*, а не по аналогии. Квантовый хвост сюда тащить не нужно. Нам нужен не свет, а лапласиан.

8.3. Доказуемое свойство телепорта

Почему сон (телепорт) необходим, а любопытство его не заменит? Любопытство *локально по вниманию* — смещённый обход от текущего фронтира; узел X в затенённом углу, оторванный от активного множества, недостижим, как ярко ни свети фокус. Телепорт ломает достижимость: рестарт падает куда угодно. Это ровно то, *почему у PageRank есть телепорт*: с $\alpha > 0$ цепь становится *эргодической* (неприводимой и апериодичной), а значит каждый узел посещается в матожидании с частотой $\pi_v > 0$ — ни один не заперт в стоке или на острове. Телепорт даёт *гарантию покрытия*, которой у смещённого блуждания доказуемо нет.

Анти-энтропийный скраб

«Не сразу; мы никогда не знаем, что и где сломалось, поэтому просто постоянно проводим случайные инспекции» — это зрелый паттерн из распределённых хранилищ: *anti-entropy scrub* (проверка целостности в ZFS; Merkle-based gossip в Dynamo/Cassandra). Сон Рин — анти-энтропийный скраб по графу памяти.

8.4. Пять следствий для работы Рин с памятью

1. **Топологическая значимость («гравитационные ямы»).** Хабы переактивируются независимо от задуманных вероятностей — π на них концентрируется. Рин чаще вспоминает хаб-узлы, реже периферию (затенённые углы — регионы с малым π). Топология создаёт *неявную важность*: иногда полезно, иногда случайный плотный кластер становится чёрной дырой припоминания.

2. **Узкие места / over-squashing.** Два семантически близких, но топологически далёких кластера через тонкий мост (низкая проводимость) плохо обмениваются активацией. Рин не свяжет факты, «далёкие» по графу. Прямое ограничение fusion.

3. **Пересглаживание уверенности (over-smoothing).** Повторная диффузная пропагация гомогенизирует уверенность к консенсусу — теряется контраст «уверена / не уверена». Дегенеративный отказ; лечится как в графовых сетях: остаточные связи (якорь к исходному свидетельству), не чисто диффузные апдейты, пере-заземление в провенанс.

4. **Кривизна как локатор ценного-и-задушенного.** Отрицательно искривлённые рёбра — мосты между сообществами — это места самых креативных слияний *и* максимального over-squashing. Значит самые ценные ассоциации топология душит сильнее всего. Отсюда *объективная* причина сна/телепорта: он — rewiring, топологическая починка over-squashing обходом узкого места случайным рестартом.

5. π **как топологический субстрат идентичности.** Какие узлы структурно центральны — к чему Рин привычно возвращается — подложка её «озабоченностей», часть характера. Форма графа памяти — часть того, кто она.

Over-squashing на мосте SBM

Возьмём стохастическую блочную модель: два плотных сообщества, соединённых m рёбрами-мостом. Эффективное сопротивление между узлами разных сообществ

$$R_{\text{eff}}(i,j) = (e_i - e_j)^\top L^+ (e_i - e_j)$$

растёт при уменьшении m (мост уже — сопротивление выше). Влияние дальнего узла, измеряемое через якобиан $\left\|\partial h_i / \partial x_j\right\|$, ограничено сверху через $R_{\text{eff}}(i,j)$. Значит при узком мосте fusion между сообществами *доказуемо* слаб — сколько ни води обычный walker. Единственное, что чинит, — телепорт: он кладёт рестарт в дальнее сообщество напрямую, минуя мост. Метрика для стенда: частота cross-cluster fusion против m (глава 22).

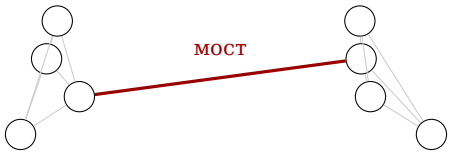


Рис. 8.1: Два сообщества через узкий мост. Через него активация «задыхается» (over-squashing); телепорт обходит его, прыгая напрямую.

8.5. Операционные величины и осторожности

Вести дело лучше *случайно-блуждательными* величинами: π, время перемешивания, hitting/commute time, R_{eff}, проводимость/константа Чигера, спектральный зазор. Кривизну держать как *локатор узких мест*, не как рычаг.

Что дёшево на лету: π и «хабность» — инкрементально дёшевы; R_{eff} — дорого (через L^+), но аппроксимируемо. Это позволяет Рин самой знать, где затенённые углы, и гнать туда телепорт-скраб прицельно.

Две осторожности. *Во-первых*, граф направленный, типизированный, знаковый — нужный оператор не симметричный лапласиан, а несимметричная матрица переходов, спектр может быть комплексным, кривизна для орграфов менее чиста. *Во-вторых*, граф *динамический*: fusion меняет саму топологию, и это диффузия на *время-зависимом* графе, где поток и структура сцеплены обратной связью, — за рамками статических графов и самое интересное место.

9. Математика retrieval: от локального шага к глобальному поведению

9.1. Зачем отдельная математическая глава

В базовом изложении retrieval уже описан корректно: есть якоря, есть обход, есть настроение, есть телепорты, есть разные политики. Но для курса этого мало. Нужно увидеть разницу между *локальным законом шага* и *глобальной физикой потока*. Именно здесь многие архитектурные интуиции либо укрепляются, либо рассыпаются.

9.2. Переходный оператор

Пусть граф памяти задан вершинами V и типизированными рёбрами E. Для фиксированного режима работы зададим для пары соседних узлов A, B ненормированную силу перехода

$$S(A \to B) = w_{AB} \cdot \mathrm{rel}(B \mid q) \cdot \mathrm{mc}(B \mid \mathrm{mood}) \cdot \mathrm{unc}(B)^{\beta_u} \cdot \mathrm{gate}(A, B),$$

где последний множитель $\mathrm{gate}(A, B)$ удобно выделить отдельно: он собирает всё, что не является содержательной близостью, но влияет на сам факт допустимости шага — например, запрет на некоторые типы рёбер в целевом retrieval, штраф за повторное хождение по одному и тому же контуру, локальные бюджеты.

Нормировка даёт матрицу переходов

$$P_{AB} = \frac{S(A \to B)}{\sum\limits_{C \in \mathcal{N}(A)} S(A \to C)}.$$

Если включён телепорт, фактический оператор имеет вид

$$\widetilde{P} = (1 - \alpha)P + \alpha \mathbf{1} r^{\top},$$

где r — распределение рестарта. В retrieval оно сосредоточено на якорях, во внутренней жизни смещено к зонам неопределённости, во сне может быть почти равномерным или управляться картой неудовлетворённого покрытия.

Почему важен именно оператор, а не набор красивых факторов

Пока формула живёт как список слов — релевантность, настроение, новизна, — легко вообразить, что эти ручки независимы. Но после нормировки они начинают бороться за один и тот же переходный вес. Усилив настроение, можно случайно ухудшить релевантность; усилив жадность, можно обрушить покрытие. У оператора есть системные эффекты, которых не видно по отдельным множителям.

9.3. Температура и режим выбора

Пусть логарифм ненормированного веса шага равен $z_{AB} = \log S(A \to B)$. Тогда softmax с температурой T задаёт

$$P_{AB}(T) = \frac{\exp(z_{AB}/T)}{\sum_C \exp(z_{AC}/T)}.$$

Пределы понятны:

$T \to 0 \Rightarrow$ почти жадный выбор, $T \to \infty \Rightarrow$ почти равномерный выбор по допустимым сосе

Но practically важен не предел, а производная: насколько чувствительна вероятность перехода к малому изменению веса. Именно здесь видно, что высокая температура — не просто «больше случайности», а меньшая устойчивость к мелким локальным ошибкам в оценках. Для debugging это полезно, для production-retrieval — опасно.

9.4. Якоря, рестарты и two-step retrieval

В исходной архитектуре уже сказано, что retrieval не ищет запись обходчиком с нуля, а сначала находит якоря. Это место стоит формализовать.

Пусть есть запрос q. Тогда retrieval распадается на две задачи:

$$q \longmapsto A(q) \subseteq V \longmapsto \text{локальное разворачивание окрестности.}$$

Первая задача — *anchor selection*; вторая — *context expansion*. Смешивать их нельзя.

Определение 9.1 (Anchor recall)**.** Anchor recall — доля запросов, для которых на этапе выбора якорей в стартовое множество попал хотя бы один узел из минимального носителя правильного ответа.

Определение 9.2 (Bridge rate)**.** Bridge rate — доля разворачиваний, в которых после попадания в правильную область обход действительно дошёл до промежуточных узлов, без которых ответный фрагмент остался бы несвязной кучей фактов.

Эти две метрики принципиально различны. Низкий anchor recall чинится поиском якорей; низкий bridge rate — уже топологией, температурой, телепортом и типами допустимых переходов.

9.5. Hitting time и доступность далёких зон

Пусть T_B — время первого попадания в узел B. Тогда $\mathbb{E}_A[T_B]$ — *hitting time* из узла A в B. Для памяти это не абстракция: если интересный, но редкий контрпример имеет слишком большой hitting time относительно типичных режимов сна или фоновой разведки, то practically он *не существует* для системы.

Отсюда инженерный вывод:

Принцип 9.1 (Редкий узел без пути — это не «редкое знание», а потерянное знание)**.** Если значимые узлы достижимы только по длинным и узким маршрутам, память не просто медленно к ним приходит. Она систематически предпочитает не приходить вовсе, а значит будет стабильно недокорректированной.

9.6. Спектральная щель и смешивание

Если $\widetilde{P}$ эргодична, у неё есть стационарное распределение π. Скорость смешивания управляется вторым по модулю собственным значением λ_2:

$$\text{чем меньше } |\lambda_2|, \text{ тем быстрее забывается старт.}$$

Для памяти это двусмысленное благо.

- Быстрое смешивание полезно для покрытия и scrubbing.
- Медленное смешивание полезно для удержания темы и целенаправленного retrieval.

Отсюда видно, почему одна и та же политика не может одинаково хорошо служить и ответу, и сну. Retrieval хочет меньше забывать старт; сон, наоборот, должен уметь выползать из локальной ямы.

9.7. Проводимость и bottleneck starvation

Пусть $S \subset V$. Проводимость

$$\Phi(S) = \frac{\sum_{u \in S,\ v \notin S} \pi_u \widetilde{P}_{uv}}{\min(\pi(S), \pi(V \setminus S))}$$

измеряет, насколько легко поток выходит из области S. Малые значения Φ означают bottleneck: память часто крутится внутри района, но плохо ходит наружу.

В прикладном смысле это ведёт к двум симптомам:

1. cross-cluster fusion почти не случается;
2. контрсвидетельства, сидящие за узким перешейком, плохо добираются до ревизии.

9.8. Новая инженерная добавка: два бюджета контекст-пакера

Ни в исходном roadmap, ни в базовом дубле это не было выделено достаточно явно, поэтому здесь вводится как *расширение*, а не как ретушь старого текста.

Пусть retrieval уже вернул множество кандидатов C. Обычная ошибка — отдать в контекст всё, что набрало высокий балл, в одном общем списке. Но для ответа нужны по меньшей мере два разных ресурса:

$$B = B_{\text{evidence}} + B_{\text{glue}}.$$

Определение 9.3 (Evidence budget)**.** Бюджет на узлы-свидетельства: наблюдения, факты, контрфакты, provenance-носители, то есть то, что действительно несёт доказательную нагрузку.

Определение 9.4 (Glue budget)**.** Бюджет на связующий материал: переходные концепты, краткие обобщения, локальные мосты, без которых набор свидетельств останется несвязным.

Если не разделять эти бюджеты, происходит одно из двух. Либо всё съедает свидетельственный материал, и ответ становится точным, но несобранным; либо всё забивается «красивыми мостами», а фактическое основание усыхает. Разделение бюджетов делает context packer не просто сортировщиком, а маленьким архитектором локального доказательства.

9.9. Что именно надо мерить на стенде

Из этой главы естественно вытекает retrieval-аудит как кортеж

$$\mathcal{R} = \big(\text{anchor recall}, \text{bridge rate}, \text{contradiction hit rate}, \text{provenance coverage}\big).$$

Последние два компонента читаются так:

Определение 9.5 (Contradiction hit rate)**.** Доля случаев, в которых при наличии в релевантной окрестности важного контрсвидетельства retrieval действительно поднял его в рабочий контекст.

Определение 9.6 (Provenance coverage)**.** Доля включённых в контекст утверждений, для которых рядом лежит хотя бы минимальный support-skeleton или иной объяснимый след происхождения.

Почему одного recall недостаточно

Пусть ответ на вопрос требует узлов A и B, а их связывает мост M. Если якорь нашёлся в районе A, а обход принёс A и ещё пять соседей, но не принёс M и B, формально память «что-то помнит», practically — не отвечает. Recall на якорях может быть высоким при полностью провальном bridge rate.

9.10. Вывод

Математика retrieval нужна не ради строгости как украшения. Она нужна, чтобы не перепутать четыре разных класса проблем:

1. ошибка выбора якоря;
2. ошибка локального разворачивания;
3. топологическая недоступность;
4. ошибка упаковки уже найденного материала.

Без такого разделения система с плохим ответом выглядит просто как «что-то не так с памятью». С ним становится видно, *какой именно слой памяти провалился*.

10. Fusion, вывод и консолидация

Обход достаёт то, что *уже* есть в графе. Но память живого существа ещё и *порождает* новое: связывает далёкое, делает выводы, обобщает. Этот класс операций мы называем *fusion*. Здесь разбирается, как именно из активированных узлов рождается новый узел, чем вывод отличается от ассоциации и обобщения и как всё это остаётся под эпистемическим контролем.

10.1. Что такое fusion, механически

Определение 10.1 (Fusion, подробно)**.** *Fusion* — оператор, который по *совместно активированной* конфигурации узлов $\{A_1, \ldots, A_n\}$ создаёт новый узел X и связывает его с посылками рёбрами провенанса (обоснования). Новый узел рождается *провизорно*: с низкой уверенностью и, при необходимости, с флагом `need_to_discover` на том, что его различает.

Механика в четырёх шагах:

1. **Со-активация.** Обход (retrieval, любопытство или сон) подсвечивает несколько узлов одновременно.
2. **Распознавание конфигурации.** Оператор замечает, что активная конфигурация *согласована* — образует осмысленный паттерн (причинную цепочку, аналогию, повтор).
3. **Рождение узла.** Создаётся X с содержанием (сформулированным языковой моделью) и провенансом $\{A_1, \ldots, A_n\}$.
4. **Провизорная запись и пропагация.** X пишется с малым conf, затем пропагируется со знаком, карантином и дисконтом независимости (глава 14).

Принцип 10.1 (Fusion рождает гипотезу, а не факт)**.** Результат fusion — всегда *кандидат*, а не запись. Это защищает от превращения обаятельной ассоциативности в фабрику ложных убеждений (глава 13). Повышение до «факта» — только через накопление подтверждений или выживание в ночном скрабе.

10.2. Три вида fusion

Ассоциация. Два далёких узла со-активируются, и рождается связь (или узел-мостик). Это источник «неожиданных, но осмысленных» ходов мысли.

Ассоциация: Платон-боец

Ночной обходчик случайно со-активирует два узла: «Платон был олимпийским чемпионом по панкратиону (боец/борец — по свидетельству Олимпиодора)» и «*platon* по-гречески — „широкий, широкоплечий“». Fusion заме-

чает согласование «мощное телосложение бойца» + «имя означает „широкий“» и рождает *гипотезу*: «„Платон“ — прозвище по телосложению, а не данное имя», conf $= 0{,}2$, провенанс из двух узлов, флаг `need_to_discover` на «проверить: прозвище ли это и каково настоящее имя». Вывод правдоподобен (версий, к слову, несколько, но все сводятся к трактовке слова «широкий», а настоящее имя по преданию известно) — но именно *гипотеза*: если не пометить низкой уверенностью, это тот самый преждевременный коллапс (глава 13). Существенно, что связывает не «Платон-философ» сам по себе, а именно пара «боец» + «перевод».

Вывод (инференция). Из посылок рождается заключение. Здесь принципиально важен случай *n-арного* вывода, где ни одна часть посылок не достаточна.

N-арный вывод: корм котам

Пусть в памяти Рин про пользователя есть три узла, и **каждый подтягивает свой контекст** из эмбрионального графа (см. раздел ниже):

- A — «у него есть коты» $\Rightarrow$ контекст: *котов надо кормить, нужен корм*;
- B — «живёт в Португалии» $\Rightarrow$ контекст: *магазины закрываются рано*;
- C — «иногда ездит на пары» $\Rightarrow$ контекст: *с пар возвращается поздно*.

Только совмещение всех трёх *с их контекстами* даёт вывод:

$X =$ «купи котам корм на выходных — в будни по пути с пар можешь не успеть до закры

Существенно, что *ни одна пара не достаточна*: без «магазины рано» нет спешки, без «поздно с пар» нет проблемы, без «нужен корм» нет повода. Вывод рождается лишь при *одновременном* присутствии трёх посылок и их контекстов.

Как мы это представляем *сейчас*, не меняя структуру: fusion создаёт узел X с провенансом $\{A, B, C\}$. Тем самым тройка-посылка кодируется как узел-фактор, ссылающийся на три посылки, — это *factor-graph*-кодировка гиперребра $\{A, B, C\} \to X$. Иначе говоря, наша память уже наполовину гиперграф: попарные рёбра A–B, B–C могут отсутствовать (Португалия и коты не связаны напрямую), а n-арный вывод всё равно рождается — через общий узел-обоснование. Стоит ли делать гиперрёбра *первоклассными* объектами (со своим обходом и лапласианом) — тема приложения D.

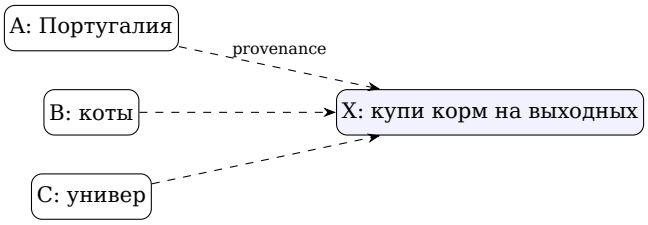


Рис. 10.1: N-арный вывод как узел-фактор X с провенансом $\{A, B, C\}$. Попарных рёбер между A, B, C может не быть.

Как посылка «подтягивает свой контекст»

Важная деталь, без которой n-арный вывод не работает: посылки участвуют не «голыми», а вместе со своим *контекстным ореолом*. Механически это ограниченная (1–2 хопа), смещённая целью/настроением *spreading activation* от каждой посылки в эмбриональный граф:

$$A = \text{коты} \Rightarrow \{\text{нужен корм}\},$$
$$B = \text{Португалия} \Rightarrow \{\text{магазины рано закрываются}\},$$
$$C = \text{универ} \Rightarrow \{\text{поздний возврат}\}.$$

Fusion работает над *объединением* этих ореолов, а не над тремя голыми узлами; композитор (языковая модель) собирает из «нужен корм + магазины рано + поздний возврат» связный actionable-вывод. Провенанс X — посылки *и* реально использованные контекст-узлы. Ореол смещён текущей целью: иначе подтянулась бы вся Вселенная соседей, поэтому активируется лишь релевантное намерению (тот же mood/goal-bias, что в retrieval, глава 6). Именно поэтому эмбриональный граф (глава 19) не декорация: без рёбер ConceptNet/ATOMIC у посылок нет ореолов — и вывод не рождается.

Трудная часть — распознать связность («трудные 80%»)

Подтянуть ореолы дёшево; *трудно* распознать, что их объединение образует *связную* цепочку рассуждения, а не случайную кучу со-активаций. Кандидаты разобраны в главе 11 (раздел 11.5): LLM-судья / графовые шаблоны-схемы / обучаемый скорер связности — ни один пока не признан удачным. На старте — LLM-композитор предлагает, а φ-скорер и обратная связь фильтруют (главы 16, 17). Это открытый инженерный вопрос.

Обобщение (абстракция). Много однотипных эпизодов сливаются в один узел более высокого уровня. Это и есть консолидация (следующий раздел).

10.3. Fusion сцепляется: sub-fusion — это тоже fusion

Виды fusion — не разные механизмы, а один оператор, порождающий узлы разного *рода*. И fusion *сцепляется*: обобщение, построенное одним шагом («sub-fusion»), становится посылкой для вывода на следующем. Абстракция *поднимает уровень*, вывод делает конкретный скачок, опираясь на неё:

$$\text{факты} \xrightarrow{\text{вверх (абстракция)}} \text{обобщение} \xrightarrow{\text{скачок (вывод)}} \text{гипотеза}.$$

Механика та же (со-активация $\rightarrow$ новый узел с провенансом); «sub» лишь означает, что этот узел — промежуточное *обобщение*, мост для последующего шага. (Это, к слову, клеточная картинка из приложения Е: высшая клетка, приклеенная к низшим.)

Дневная находка и полная эпистемическая петля: House / Cast Away

Фон. Фоновый разведчик (глава 6.4), даже на скудном дневном бюджете, со-активирует два далёких узла: «в *House M.D.* есть Уилсон» и «в *Cast Away* есть „Уилсон“ — волейбольный мяч».

Sub-fusion (абстракция). Оба узла порождают обобщение: «Уилсон — приём *экстернализации внутреннего монолога* ГГ» (у Cast Away — из-за одиночества на острове; у House — потому что ГГ умнее всех, детектив, интроверт). Родился узел более высокого уровня — общая сценарная функция.
Fusion (скачок). Эта абстракция + совпадение имени «Уилсон» рождают *гипотезу*: «а вдруг имя Уилсон в House — отсылка к Cast Away?!» Провизорно, низкая уверенность.
Проверка (петля эпистемики, глава 13). `need_to_discover` на «годы выхода» → внешняя проверка: Cast Away (2000) раньше пилота House (2004) — совместимо, уверенность *растёт*. Затем `need_to_discover` на «есть ли ещё свидетельства» → проверка опровергает: официальная версия — оммаж Шерлоку Холмсу (House ≈ Holmes, адрес 221B), а Джеймс Уилсон ≈ доктор Ватсон (одна буква, созвучие). Гипотеза *понижается*, на её место встаёт лучше обоснованная.
Мораль. Красивый, согласующийся с частью данных вывод оказался ложным — это тот самый преждевременный коллапс, пойманный проверкой. И заметьте: «правильный» ответ (Holmes → House) — сам по себе fusion, просто с лучшим провенансом. Как говорит ГГ, «все лгут» — поэтому уверенность держим ниже единицы и идём проверять.

10.4. Консолидация как огрубление

Определение 10.2 (Консолидация, подробно)**.** *Консолидация* — оператор, заменяющий множество похожих узлов/рёбер одним обобщённым, сохраняя релевантные свойства и отбрасывая детали. Формально это *огрубление графа* (coarse-to-local, глава 11).

Консолидация: «Z — частый гость»

Десять эпизодов «Z был на стриме в понедельник», «…в среду», …сворачиваются в один узел «Z — частый гость, любит доту». *Выживают*: частота, тёплая валентность, тема (дота). *Теряются*: точные даты и реплики. Плохое огрубление сохранило бы даты и потеряло валентность — Рин помнила бы, *когда* заходил Z, но забыла бы, *как к нему относится*. Значит критерий консолидации — сохранять аффективно- и реляционно-релевантное, а не всё подряд.

Консолидация запускается двумя путями: во сне (фоновая переработка, глава 12) и забыванием (когда эпизоды тускнеют, их дешевле слить в обобщение, чем хранить порознь, глава 11).

10.5. Кто пишет в узел и в каком виде

Отдельно разберём вопрос, который легко замять: кто именно порождает содержание нового узла — LLM, автомат или Rinness — и в каком *виде* оно ложится в память. Полезно разбить на три подшага, потому что механизмы у них разные.

Решение (сливать ли). Для fusion это распознавание связной конфигурации (открытый «трудный 80%», раздел 11.5); для консолидации — обнаружение избыточного кластера. И тут важная поправка: группировать «однотипное» *по строке* нельзя. Зритель мог «заходить», а мог «забегать» — чистый автомат, ключующийся по глаголу, тут же ломается, ведь мы не типизируем глаголы, которыми к Рин обращаются. Синонимию ловит не автомат, а либо LLM при записи (нормализует «забежал» в канонический тип связи visited), либо кластеризация по близости эмбеддингов при консолидации. Поэтому честная картина: автомат делает бухгалтерию (счётчики, хаб, перекидка рёбер, усреднение валентности), эмбеддинги — группировку «похоже по смыслу», а маленький LLM — итоговый ярлык. «Только автоматом» проходит лишь случай точных дублей.

Содержание узла. Новое *осмысленное* содержание (вывод, абстракция, ярлык обобщения) пишет **языковая модель** — шаблоном такое не собрать. Но *та же самая* модель, а не какая-то отдельная: «нейтральный режим» и «Rinness» — это два *настроения* одной модели, а не два мозга. Для записи в память её держат в спокойном, откалиброванном «девелоперском» режиме вектора настроения: рассудочно, минимум стиля. Если голос Рин зашит в веса (LoRA), совсем нейтральной её не сделать — останется лёгкий налёт, и это не страшно, лишь бы не перегибать: в память важно *содержание*, а тон всё равно родится заново на выходе. Автомат при этом ставит все метаданные (провенанс, уверенность, `need_to_discover`, importance, домен) — детерминированно.

Кто именно пишет — и почему это не «Rinness». Пишет в граф *та же Рин*, но в спокойном *режиме записи*, а не в режиме голоса. Здесь важно не путать «Рин» и «Rinness»: *Rinness* — это узко *выходная стилизация* (голос, гл. 17), которая превращает извлечённое содержание в реплику пользователю. Вот эта стилизация в граф *не* пишет. Одна Рин, два режима: режим записи (спокойный) кладёт нейтральное содержание, режим голоса (Rinness) звучит на выходе. Отсюда принцип: *храни спокойно, стилизуй на выходе*. Причина простая: узел переиспользуется в разных будущих настроениях, и если вморозить в него сегодняшний тон, он будет отдавать им и через месяц, когда Рин к тому же зрителю потеплеет.

В каком виде хранить: содержание против стиля

Итак, в память кладём *нейтральное содержание* («зритель Z проиграл в доте») плюс её *отношение* в метаданных — валентность и, для значимых случаев, отдельный узел-реакцию. Личность памяти живёт в отношении, а не в замороженных словах. Класть готовую фразу «в голосе» («Z снова слил, ну куда ему») — плохо: это вмораживает транзиентный тон, память пухнет, а стилизовать на выходе уже нечем.

Исключение — *вспышечная память*. У людей рядовые события помнятся по сути, а редкие пиково-эмоциональные — вплоть до того, что было сказано и как ощущалось. По аналогии: для *редких* моментов с высокой значимостью (большой $|\Delta\text{mood}|$) можно дополнительно сохранить *дословный снимок* — реальную реплику/тон того мгновения — как отдельный связанный узел, а не как каноническое содержание. Так по умолчанию память нейтральна и переиспользуема, но яркие пики сохраняют свою фактуру.

10.6. Выход fusion — гипотеза, а не установленный факт

Снимем терминологическую путаницу, потому что от неё зависит понимание всего. Каждый узел — это *убеждение* с уверенностью conf $\in [-1, 1]$. «Факт» в широком смысле — это просто любой узел, и да, fusion *создаёт* узел. Но узлы различаются *по уверенности*: прямое наблюдение или измерение получает высокую уверенность, а вывод (fusion) — *низкую*. Именно это имелось в виду под «fusion не коммитит факты»: *коммит* — это запись как *установленного*, высокоуверенного убеждения, которым дальше пользуются как истиной; fusion же рождает узел-*гипотезу*. Повышение гипотезы до установленного статуса происходит *позже* — через корроборацию или проверку, а не в момент рождения.

Почему это критично: без такого разделения обаятельная ассоциативность превратилась бы в фабрику ложных убеждений (пример House / Cast Away выше). Поэтому fusion смел, но не безрассуден: гипотезы рождаются свободно, но влияние гипотезы на соседей гейтируется её собственной (низкой) уверенностью и надёжностью посылок — карантин, глава 14; а различающие пробелы, которых Рин не знает, помечаются `need_to_discover`, чтобы любопытство или внешний вход их потом добыли (глава 13). Дальнейшую судьбу гипотезы — проверку, повышение или понижение — ведёт уже эпистемический слой; fusion лишь честно рождает её с низкой уверенностью и передаёт дальше.

Принцип 10.2 (На первых версиях: генератор кандидатов с аудитом)**.** Fusion — главный источник и магии, и мусора. Пока распознавание связности со-активации не решено (открытый вопрос), fusion на ранних версиях должен быть не «автоматической когнитивной магией», а *генератором кандидатов с аудитом*. Дефолтная позиция смещена в сторону осторожности: **больше кандидатов в pending, меньше commit**. Принятые fusion потом *ревизуются* отложенно (precision после проверки, глава 22), и только выжившие повышаются. Лучше недодать смелых узлов, чем зарастить память красивым мусором; смелость безопасно включим, когда научимся мерить связность.

Принцип 10.3 (Разведи candidate_text и canonical_node)**.** Fusion всё ещё сильно опирается на LLM-композитор, и это риск: модель выдаёт красивый текст, но плохо индексируемый и плохо ревизуемый узел. Разделяем два формата. LLM формулирует `candidate_text` — гипотезу человеческим языком. Но *коммит* идёт не текстом, а через *schema-writer* в `canonical_node` со структурой: {`node_kind`, `claim_type`, `entities`, `time_scope`, `confidence`, `provenance`, `distinguishing_unknowns`}. Иначе получишь хорошо написанные, но нетипизированные узлы, которые нельзя ни точно найти (forensic-ретрив, гл. 18), ни аккуратно ревизовать. Красота — на входе, схема — на коммите.

10.7. Когда со-активация — повод для реплики, а не для узла-факта

Не всякая яркая со-активация должна оседать убеждением. Иногда две всплывшие вещи — это повод *что-то сказать*, а не повод *что-то запомнить как факт о мире*. Это ключевое различение: **полезное для реплики** $\neq$ **достойное записи фактом** (anti-fusion). Разберём на каноничном случае.

Чапаев и «нюанс»: шутку не пишем, реакцию — пишем

В памяти со-активировались «Чапаев» и слово «нюанс». Walker сводит их рядом, и всплывает ассоциация: «тут просится анекдот». Что происходит по нашим механизмам.
(1) Откуда берётся анекдот. Не из графа как хранимый текст. Граф даёт только *триггер* — «эта пара уместна для шутки». Сам текст анекдота порождает базовая LLM (она знает его из предобучения) — ровно как с обучением: граф всплывает *повод*, модель поставляет *содержание*.
(2) Что НЕ записывается. Сам анекдот *не* коммитится узлом-фактом. Связь «Чапаев relates-to нюанс через этот анекдот» — это `candidate_text` для *реплики*, а не убеждение о мире; anti-fusion-гейт его как факт не пропускает. Иначе память заросла бы «фактами», которые на деле каламбуры.
(3) Что записывается. Два узла другого рода. Во-первых, `episode`: «рассказала анекдот про Чапаева-нюанс собеседнику Z» (со временем и провенансом) — это *было*, это факт о событии, не о мире. Во-вторых, реакция Z обновляет `reaction_pattern` через Beta-постериор, условленный на отношении (гл. 16): зашло $\rightarrow \alpha{+}1$, поморщился $\rightarrow \beta{+}1$. Salience-гейт решает судьбу: сдвинула реакция настроение ($|\Delta\text{mood}|$) — консолидируется; «ну ок» — тускнеет.
(4) Бонус — крючок, а не шутка. Если такой юмор у Z стабильно заходит, осесть может лёгкий `reaction_pattern`: «пара Чапаев+нюанс $\rightarrow$ этот анекдот заходит Z». Заметьте: выучивается *модель того, что кому заходит*, а не сам анекдот хранится фактом. Комедийный тайминг — Beta-апостериор, не строка в базе.

Мораль в одну строку: fusion-подобная со-активация может кончиться *репликой*, а осесть в памяти должны *событие речи* и *реакция на неё* — потому что важна не шутка, а то, как она легла. Тот же принцип защищает память от превращения в свалку острот, мемов и подходящих-к-случаю ассоциаций.

Инженер, а не фантазёр

Правильная Рин — «инженер, которую делают бывшие физики-фундаменталисты»: она охотно строит гипотезы, но по умолчанию ставит уверенность ниже единицы и идёт проверять. Fusion даёт первое (смелость ассоциаций), эпистемическая гигиена (глава 13) — второе (дисциплину). Одно без другого — либо сухой поисковик, либо обаятельный врун.

11. Многомасштабность, слои и забывание

11.1. Протекающий интегратор: точный смысл

Память Рин расслоена по временным масштабам: *сессия* → *день* → *сезон* → *год* → *жизнь*. Каждый слой — *протекающий интегратор* (leaky integrator): состояние, которое накапливает вход и понемногу «протекает» (теряет часть себя) на каждом шаге.

Определение 11.1 (Протекающий интегратор)**.** Состояние s, обновляемое под входом x_t по правилу

$$s_t = \underbrace{\left(1 - \tfrac{1}{\tau}\right)}_{\text{удержание}} s_{t-1} + \underbrace{\tfrac{1}{\tau}}_{\text{впитывание}} x_t, \qquad \tau > 1. \tag{11.1}$$

Коэффициент $(1 - 1/\tau)$ — доля прошлого состояния, которая *удерживается*; $1/\tau$ — доля нового входа, которая *впитывается*. Постоянная времени τ задаёт «медленность»: большое τ — слой почти не меняется за шаг (удерживает долго, впитывает по чуть-чуть), малое τ — резко реагирует и быстро забывает.

Ведро с дыркой

Протекающий интегратор — это ведро с дыркой в дне, куда капает вода (вход). Уровень воды s растёт от капель и падает от протечки. Широкая дырка (малое τ) — уровень скачет за каждой каплей и быстро сходит; узкая дырка (большое τ) — уровень меняется медленно, зато держится долго. «Сессия» — ведро с широкой дыркой, «жизнь» — с почти закрытой.

11.2. Два разных смысла «слоёв» — не путать

Здесь важная развилка, которую легко замылить: слово «слои» и «протекание» означают *разное* для настроения и для памяти.

Настроение — параллельные интеграторы, переноса нет. Слои настроения (глава 7) — это несколько интеграторов (11.1) с разными τ_ℓ, работающих *параллельно на одном и том же потоке событий*. Старший слой не питается от младшего; оба питаются событиями, просто старший интегрирует медленнее. «Младший дрожит сильнее» — это высокочастотная компонента того же сигнала. **Переноса содержания между слоями нет** — есть разночастотная фильтрация одного входа, а актуальное настроение — суперпозиция.

Память — перенос есть, это повышение слоя консолидацией. Слои памяти устроены иначе: узел *повышается* из быстрого слоя в медленный, когда консолидация делает его устойчивым (раздел 11.3). Вот это и есть настоящее

«протекание младшего в старший»: не диффузия сигнала, а *перемаркировка* узла (`layer`: session → day → …) при накоплении силы. Механизм известен по форме; конкретные пороги повышения — открыты.

Пороги повышения слоя

При какой накопленной силе / частоте обращений / значимости узел повышается в следующий слой? Разумные кандидаты: порог по базовой активации ACT-R (раздел 11.4); порог по числу независимых подтверждений; порог по |conf|. Точные значения и то, единый ли это порог для всех типов узлов, — пока не зафиксированы; это параметр для стенда (глава 22).

11.3. Консолидация как огрубление графа

Переход от эпизодов к смыслам — это *огрубление* (coarsening): десять записей «зритель заходил в понедельник», «…в среду», …заменяются одним узлом «зритель — частый гость». Мелкая структура схлопывается в крупную; часть информации теряется намеренно.

Принцип 11.1 («Консолидация» — это процесс, а не один оператор)**.** Здесь важно снять терминологическую путаницу, из-за которой легко запутаться (и мы путались). Под словом **консолидация** мы понимаем не единичный оператор «свернуть A, B, C в X», а *процесс комплексной отработки нескольких разных операторов*, каждый со своими правами и рисками:

1. **Fusion / вывод** — рождает новый смысл («поездка на Азоры», «ходит по нечётным»). Посылки *не* трогает (гл. 10).
2. **Summary-overlay (мягкая консолидация)** — заводит узел-обобщение *поверх* исходных, дети остаются, добавляются рёбра членства. Безопасный дефолт.
3. **Dedup / merge** — склеивает *реально один и тот же* факт из разных источников (это слияние identity, а не абстракция).
4. **Prune / сжатие с потерей** — вот единственный оператор, который *удаляет* детали. Редкий, поздний, жёстко гейтируемый.

Почти вся ранняя консолидация — это (1)+(2): создать смысл и структуру доступа, ничего не разрушая. Опасен только (4), и то лишь при преждевременном применении. Когда ниже написано «консолидация сносит избыточное» — речь про оператор (4) внутри процесса, а не про процесс целиком.

Почему нужен активный синтез, а не один только decay

Легко спросить: если узлы и так тускнеют по непосещаемости, зачем вообще что-то сворачивать? Затем, что *decay* (смерть по непосещаемости) — *слепое выбрасывание*: он теряет данные, *не извлекая из них закон*. Если тридцать эпизодов «приходил по нечётным» просто протухнут, память потеряет их — и так и не узнает, что он ходит по нечётным. А связка *синтез (fusion) + поздний прунинг* — то, что мы и зовём консолидацией как процессом, — делает *синтез, потом выброс*: сначала дистиллирует gist, и лишь потом ретирует ставшую лишней деталь. Формула: *выучить урок (родить обобщение) и выбросить домашку (снести избыточное)*. Анало-

гия: живой ум не помнит каждый восход, но твёрдо знает «солнце встаёт на востоке» — инвариант извлечён, экземпляры отброшены; будь в системе только decay, она бы просто забывала восходы, а правила бы не получил. И ещё: часто-посещаемый избыточный узел по decay *никогда* не умрёт (его же трогают) и будет вечно раздувать граф — только активный синтез его свернёт.

Как понять, что огрубление *хорошее*? Здесь нужен мост к спектральной математике огрубления. Введём:

- L_f — лапласиан *мелкого* (fine) графа; лапласиан описывает, как сигнал диффундирует по графу (кто на кого влияет и насколько);
- L_c — лапласиан *крупного* (coarse) графа после огрубления;
- P — оператор *проекции*: сопоставляет каждому мелкому узлу его крупную группу (например, десять эпизодов $\mapsto$ узел «частый гость»);
- $\Phi(X, L)$ — оператор распространения (одна или несколько итераций передачи сообщений) сигнала X по графу с лапласианом L.

Хорошее огрубление *коммутирует* с распространением:

$$\Big\| \underbrace{P\,\Phi_f(X, L_f)}_{\text{распространить, потом огрубить}} - \underbrace{\Phi_c(PX, L_c)}_{\text{огрубить, потом распространить}} \Big\| \le \varepsilon. \tag{11.2}$$

Читается так: неважно, в каком порядке действовать — сначала «подумать» на мелком графе и затем сжать, или сначала сжать и «подумать» на крупном, — результат должен почти совпасть. Норма $\|\cdot\|$ берёт разницу по всем входным сигналам X; ε — мера потери. Малое ε = огрубление сохранило то, что нужно для рассуждения; большое = сжатие сломало динамику.

Что должно выжить при консолидации

Свернём десять эпизодов визитов в узел «частый гость». *Выжить* обязаны: частота (была высокой), устойчивая валентность (тёплая), тип связи (регулярность). *Потеряться* можно: точные даты, дословные реплики. Плохое огрубление сохранит даты и потеряет валентность — тогда Рин будет помнить *когда* заходил зритель, но забудет, *как относится* к нему; это большая ε по аффективной оси. Значит критерий огрубления — сохранять аффективно- и реляционно-релевантное подпространство, а не спектр целиком.

11.3.1. Коллизии предложений: голоса, а не очередь

Огрубление предлагают обходчики (гл. 6, раздел 6.3), и предложения будут *перекрываться*. Пусть за ночь на узлах A, B, C, D, E накопилось: RW1 хочет слить $\{A, B, C, D\} \to X_1$; RW2 — $\{B, C, D, E\} \to X_2$; RW3 — $\{A, B\} \to Y_1$; RW4 — $\{C, D, E\} \to Y_2$. Наивный коммиттер «кто первый — тот и слил» построит мусор, зависящий от случайного порядка. Правильный ход — понять, что *отдельное предложение обходчика это не решение, а свидетельство*: «этих тянет вместе». Тогда коллизия — не спор за очередь, а *сигнал для агрегации*.

Механически коммиттер копит предложения в окне (обычно за ночной проход) и строит *взвешенный со-консолидационный гиперграф* на исходных узлах:

вес $c(i,j)$ = как часто обходчики клали i и j в один предлагаемый кластер, взвешенно по support и — важно — *дисконтированно по времени*: голоса в буфере экспоненциально стареют, иначе кластеризация будет учитывать давно неактуальные связи. По этому графу выделяются *кандидатные кластеры*.

И вот тут — принципиальная поправка: не надо загонять всё в *единый жёсткий раздел* (partition). Память — не разбиение; один эпизод «смешной вечер в универе с Анной, когда я устал» законно входит и в «про универ», и в «про Анну», и в «про усталость», и в «смешное». Раз parent у нас — обратимая надстройка-summary, а не разрушение мелких узлов, **перекрывающиеся кластеры разрешены**. Поэтому: выделяем кандидатные кластеры (модулярность / кластеризация — как один из инструментов, не как истина в последней инстанции), разрешаем перекрытие, а коммиттер выбирает *неконфликтующие* коммиты или заводит *несколько* parent-summary, если они сжимают *разные оси*. В примере $\{B, C, D\}$ — сильное ядро (RW1+RW2), $\{A, B\}$ и $\{C, D, E\}$ могут ужиться как разные полезные обобщения; а вот если X_1 и X_2 оказываются почти дубликатами по одной оси — их сливаем. Решает не порядок поступления, а согласие свидетельств и польза.

Четыре оговорки, без которых механизм наивен. *Дисконт независимости*: два обходчика, прошедшие по общим путям, — это часто один сигнал, засчитанный дважды; вес $c(i,j)$ дисконтируется по перекрытию провенанса предложений (тот же Bloom / MinHash, что и в пропагации, гл. 14). *Гистерезис*: сливаем только то, что рекуррентно всплывает несколько ночей; одноразовое предложение — шум, ему рано менять структуру. *Критерий пользы, а не только повторяемости*: новый parent оправдан, если он *сжимает* — уменьшает будущую стоимость ретрива / packer'а или повышает качество реакции; частая со-встречаемость без выигрыша в сжатии — не повод для обобщения. *Обратимость через стадийность*: снос избыточных детей идёт не сразу топором, а стадийно — сперва `pending` / tombstone (см. ниже) с сохранением support-skeleton (гл. 18), и только устойчиво подтверждённое обобщение доводит прунинг до конца. Поэтому цена даже неудачного слияния низкая — его можно откатить, и это снимает надобность в идеальном арбитраже с первого раза.

Комитет, а не турникет

Представьте не турникет, где предложения проходят по одному и первое побеждает, а *комитет*, который выслушивает все голоса разом и выносит одно связное решение. Обходчики — свидетели с разной надёжностью; коммиттер — тот, кто взвешивает показания (со скидкой на сговор — общий провенанс) и решает, какие обобщения завести — при этом узел может состоять сразу в нескольких, если они про разное.

11.3.2. Что сносим, что оставляем — и кто это решает

Итак, консолидация в итоге *выводит избыточные узлы из активного графа* — иначе он только растёт. Уточним слово: не «убивает» (физическое стирание — это уже не память, а бухгалтерия), а переводит узел в одно из состояний — *active* → *archived* / *support-skeleton* / *tombstone* (event-log хранит исходное всегда, гл. 18). Полезно увидеть, что в процессе слиты *две* разные операции, и решение о второй принимается тоньше, чем «дети консолидации умирают»:

- **(a) рождение обобщения** — создаётся узел-gist («приходит по нечётным», «частый гость»). Это *создание нового*, по духу близко к fusion.

- **(б) прунинг избыточных детей** — сносится то, что новый gist уже полностью покрывает.

Кто решает — коммиттер, а не обходчик, и решает он про (б) *поэкземплярно, по избыточности*, а не «по кластеру, которому узел якобы принадлежит». Обходчик лишь свидетель: «здесь есть сжимаемая конфигурация». Формально перед прунингом ребёнка v коммиттер прогоняет *residue-тест* — «есть ли у v уникальный остаток?»:

1. уникальные внешние рёбра (мост к другим частям графа, которые осиротеют)?
2. уникальный timestamp или actionable-деталь (дата, билет, обязательство)?
3. высокая значимость/валентность?
4. связь с *другим* обобщением?
5. естественный запрос, на который v отвечает лучше parent'а?

Если хоть один пункт да (residue $\neq 0$) — *не прунить*, максимум архивировать или оставить support-skeleton. Только при чистом нуле v выводится из активного графа; и даже тогда он может просто потухнуть по ACT-R, если к нему перестанут обращаться. В чистом случае «по нечётным» под критерий сноса попадают *все тридцать* экземпляров (они взаимозаменяемы, остатка и мостов нет) — остаётся один узел; консолидация там максимально беспощадна, и правильно. Мостовой богатый узел (эпизод, на котором висят два разных обобщения) — исключение не потому, что «мы не прунуем», а потому, что он *не избыточен*.

Право на схлопывание: экземпляр против составляющей. Заострим, *что* именно делает ребёнка избыточным, — это и есть та тонкая грань, которую легко проскочить. Схлопывать (с потерей) можно только *класс эквивалентных экземпляров* одного паттерна: тридцать «заходил по нечётным», сто «привет», каждый восход. Они взаимозаменяемы ('instance_of'), и gist их полностью замещает. А *разнородный букет составляющих* одной темы ('part_of') схлопывать нельзя *в принципе* — там нечего терять, там всё уникально. Отсюда типичная ошибка: «fusion сгруппировал A, B, C» *не* значит «A, B, C можно схлопнуть». Группировка-по-теме $\neq$ поглощение-избыточности. Разберём на неигрушечном примере.

Поездка на Азоры: fusion есть, схлопывать нечего

Пять фактов: (A) купил плавки, (B) купил зонтик, (V) билет на Азоры на 5 сент., (G) автобус аэропорт Лиссабона $\to$ Порто на 9 сент., (D) конференция в Порто 9–12 сент. Напрашиваются два fusion: X_1 = «отпуск на Азорах» (A, B, V), X_2 = «маршрут Азоры $\to$ Лиссабон $\to$ конференция» (V, G, D). Но **схлопывать здесь нечего**: A, B, V, G, D — не экземпляры паттерна, а разнородные составляющие. Прунинг убил бы ровно то, что нужно для действия: даты, билеты, предметы, обязательства. Поэтому — только (1)+(2): рождаем X_1, X_2 как overlay, факты оставляем, вешаем ролевые рёбра (`equipment_for`, `leg_of`, `purpose_of`).
Тонкость *event vs state*: «купил плавки» (event) порождает отдельный узел «вероятно, владеет плавками» (state, другой `node_kind`, гл. 5); event со временем уйдёт в архив, а state живёт, пока не появится «потерял/выкинул/не подошли». Потому и на «Рин, поищи плавки» через два

месяца она поднимает state как *контекст* и в худшем случае уточняет («вроде брал в августе — ищем на замену?»), а не блокирует запрос старым фактом. Это отдельный инвариант — *память — контекст, а не команда*: прошлый факт не отменяет текущее намерение, а лишь оттеняет его.

Принцип 11.2 (Два инварианта консолидации)**.** *Первый: вывод обходчика — свидетельство, а не власть.* Коммиттер решает задачу *отбора модели* — какой набор новых узлов лучше объясняет данные при минимальной сложности (MDL-настроение: выигрыш в сжатии + связность + повторяемость + польза − избыточность − сложность − эпистемический риск), а не исполняет предложения слепо. *Второй: перекрывать обобщения — можно, дублировать identity — нельзя.* Один исходный узел может быть посылкой нескольких parent-summary через рёбра членства (`member_of`/`provenance`); но *клонировать* его на несколько копий в графе запрещено.

Почему нельзя клонировать узлы («графовый мейоз»)

Соблазн разрулить пересечения так: развернуть общий узел C в близнецов C_1, C_2, раздать по разным обобщениям, оригинал снести — и предложения обходчиков станут чистым разбиением. Формально приём валиден (graph unrolling), но семантически ломает память в двух местах. *Расщепление identity убивает TMS*: опровергнут позже факт C — ретракция поправит один клон, второй останется фантомом, лавинная ревизия уверенности сломается. *Разрыв моста*: общий C был перешейком, по которому активация текла между X_1 и X_2 («раздражается» $\leftrightarrow$ «не выспалась» связывались *через* C); разнёс копии — потерял вывод навсегда. И «раздваиваются только на время» — иллюзия: дочерние не сливаются обратно, двойной след остаётся *навсегда*. Спасение идеи: близнецы допустимы *только как черновая вычислительная техника* в блокноте коммиттера (чтобы независимо оценить кандидатов), но в *коммите* общий узел остаётся общим — identity в графе не двоится.

11.3.3. Кандидаты, наследование линков, слияние уверенностей

Три механических вопроса, которые всплывают на реализации.

Жаккард/LSH — сигнал, а не критерий. Соблазн: «полное перекрытие окрестностей $\Rightarrow$ поглощаем». Опасно. Структурная эквивалентность $\neq$ семантическая замещаемость: «купил плавки» и «купил зонтик» имеют почти одинаковых соседей (Азоры, пляж, лето, покупка), но не взаимозаменяемы — один отвечает «есть ли плавки», другой «есть ли зонтик». Поэтому индекс Жаккара (и его дешёвая оценка через MinHash/LSH за $O(|V|)$ вместо $O(|V|^2)$ — да, это правильный способ искать кандидатов ночью) годится как *кандидатный сигнал* «посмотри сюда, возможно дубль или класс экземпляров», но *никогда* как критерий удаления. За ним обязателен семантический гейт «это экземпляр без уникального остатка?». Даже «новое нечётное число $\approx$ паттерн „нечётные“» законно поглощается (инкремент support-count) лишь если этот экземпляр пустой; а если 9-е — день, когда он не успел на автобус, экземпляр не пуст, не поглощай.

Два разных действия с рёбрами — не путать в коде. Сначала разведём то, что легко слить в одну функцию и устроить себе Чернобыль. *Overlay role-edge* — новая ролевая связь *parent-summary* (‘X_1 –equipment_for–> плавки‘), создаётся уже на стадии overlay, *без всякого прунинга* A; это про *создание* структуры доступа. *Prune edge-lifting* — подъём *всех* рёбер ребёнка на parent при его сносе; работает *только* для чистых экземпляров. Это разные операции; общей функции `lift_edges(child, parent)` на все случаи быть не должно.

Наследование линков при прунинге: та же онтология — но только где прунинг законен. Простое правило «рёбра ребёнка переходят на parent с теми же типами отношений» — *верно ровно для чистых экземпляров*, а это единственный случай, где наследование вообще происходит. Для них это буквально проекция смежности $A_c = P\,A_f\,P^\top$: дубли рёбер сливаются, веса складываются и *ренормируются* (L1, чтобы переходы обходчика остались вероятностями), провенанс-сигнатуры объединяются побитовым OR. А там, где то же правило даёт бессмыслицу — ‘поездка –object–> плавки‘ — это *сигнал*, что прунится не экземпляр, а составляющая, которую прунить нельзя. Отсюда чистый предохранитель: **если ребро нельзя поднять, не меняя его смысл, — узел не прунится** (максимум понижается/архивируется). Невозможность безопасно унаследовать линк сама останавливает неуместный прунинг.

Слияние уверенностей: среднее в confidence, разброс — в один дешёвый скаляр. Когда сливаются рёбра с разными знаками/уверенностями, среднее (со знаком) идёт в `confidence` нового ребра. Но сворачивать разногласие *целиком* в один скаляр нельзя: теряется различие «0,5, потому что честно пятьдесят на пятьдесят и устойчиво» против «0,5, потому что половина источников за, половина против» (contested) — а это разные эпистемические состояния (гл. 5), и от них зависит, говорит Рин спокойно или помечает как спорное. Хранить все голоса — расточительно («страниц не хватит»). Компромисс: держать *одно дешёвое число* — дисперсию/флаг `contested` (переиспользуя Beta-дисперсию реакций, гл. 16, или механизм `contested_core_claim`, гл. 12). $O(1)$ на ребро, а не история голосований.

11.4. Забывание как тень консолидации

Забывание — не отдельный механизм, а *отрицательная сторона* того же процесса: усиливай используемое, ослабляй неиспользуемое.

11.4.1. Правильный decay — это ACT-R

ACT-R (Adaptive Control of Thought—Rational) — влиятельная когнитивная архитектура Джона Андерсона, модель того, как устроены человеческие память и научение. Нам из неё нужен один компонент — *базовая активация* (base-level activation): сила следа в памяти есть функция *частоты* и *свежести* обращений со степенным затуханием:

$$B_v \;=\; \ln\Big(\sum_{k=1}^{n} t_k^{-\alpha}\Big). \tag{11.3}$$

Разбор: t_k — сколько времени назад было k-е обращение к узлу; каждое обращение вносит вклад $t_k^{-\alpha}$, и чем оно *свежее* (меньше t_k), тем вклад больше; сумма

по всем n обращениям даёт накопленную силу (*частота*); логарифм сжимает шкалу; $\alpha > 0$ — скорость забывания. Два следствия, которых нет у плоской «$\times 0{,}95$ в день»: частые обращения укрепляют след, а *разнесённые во времени* укрепляют сильнее массовых (эффект распределённого повторения — spacing).

Тропинка в лесу

Тропинку протаптывают многими проходами; если по ней ходят изредка, но регулярно, она держится лучше, чем если пройти сто раз за день, а потом забросить. Формула (11.3) именно это и кодирует: важна не только суммарная частота, но и её распределение во времени.

Доступ сам тренирует память — различай access_reason

Тонкий баг: не всякое обращение к узлу должно укреплять его. Если ночной скраб, debug-инспектор или retrieval-тест часто трогают узел, он не должен от этого становиться «важным жизненным следом» — иначе *сама диагностика тренирует память*. Поэтому у каждого доступа есть `access_reason`: `user_recall`, `model_recall`, `maintenance_touch`, `debug_read`, `scrub_visit`, `context_packed`. В ACT-R-активацию (11.3) входят с полным весом только *подлинные* припоминания (user/model recall); обслуживающие тачи — с нулевым или почти нулевым. Наблюдатель не должен менять наблюдаемое.

11.4.2. Прунинг: не только по использованию

Прунинг только по использованию опасен: редкий, но критичный факт вымоется как мусор. Поэтому склонность к удалению гейтируется *важностью* узла:

$$\text{prune}(v) \;\propto\; \text{неиспользование}(v) \cdot \big(1 - \text{importance}(v)\big). \tag{11.4}$$

Здесь `importance` — атрибут узла (см. схему в главе 5), задаваемый значимостью при кодировании (salience из appraisal, глава 16) и подкрепляемый структурной центральностью (глава 8). Редкое-но-важное выживает; частое-но-пустое держит частота; вымывается лишь редкое-и-неважное. Врождённым (глубоко консолидированным) узлам задаётся ниже скорость распада — континуум глубины, а не бинарность.

11.4.3. importance нельзя диффундировать по путям

Здесь важная ловушка. Соблазн: раз узел-домен «цепи Маркова» важен, пусть раздаёт importance $\to 1$ своим детям, те — своим, и так область защищена. **Так делать нельзя.** importance — это диффузия по графу (глава 8), а диффузия растекается *везде*: через несколько хопов по каким-нибудь рёбрам она доползёт до «помыть машину» (хотя в текущем мире Рин такой сущности может не быть), и та станет «важной». Графовые пути неисповедимы; неуправляемая передача importance приводит к тому, что важным становится *всё*, а значит — ничто.

Правильно — **не пускать importance по произвольным путям, а закреплять её членством в домене**:

- **importance по членству, а не по диффузии.** У узла есть *теги доменов*, у доменов — importance. Эффективная важность узла = функция от

importance его доменов + собственная salience. Членство назначается *при кодировании* (прочитали учебник → его узлы помечены `domain=Markov`, высокая importance), а не распространяется обходом. «Помыть машину» никогда не получает тег `Markov`, поэтому *ничего не наследует*. Утечка исключена по построению.

- **importance по намерению.** То, что *попросили* выучить, получает высокую importance по провенансу приобретения (в отличие от случайного тривиа).
- **Если всё же нужна передача** (для непомеченных узлов) — только *типо-гейтированная* (по рёбрам членства isA/partOf, но не по ассоциативным), *сильно затухающая* (λ^k с малым λ) и *ограниченная* 1–2 хопами; и это надо *проверять* на стенде, что importance осталась локальной.

Тогда защита области (глава 18, раздел 18.3) работает аккуратно: домен-хаб с высокой importance и низким распадом + члены, наследующие *по членству*; детали могут тускнеть, но gist области держится, а сон-репетиция периодически освежает высоко-importance домены — без утечки на «помыть машину».

11.5. Что процесс делает с найденной конфигурацией

Предположим, фоновый разведчик (глава 6.4) со-активировал тройку $\{A, B, C\}$, которая выглядит связной. Что происходит дальше? Разделю на *известное* и *открытое*, чтобы не размывать.

Что известно (устоявшийся конвейер).

1. **Композиция.** Языковая модель по объединению узлов и их контекстных ореолов (глава 10) формулирует *кандидата* C^* — обобщение или вывод.
2. **Провизорная запись.** C^* пишется с низкой уверенностью и провенансом $\{A, B, C\}$; на различающих пробелах ставится `need_to_discover`.
3. **Конкуренция.** Дальше C^* живёт по общим правилам: используется и корроборируется ⇒ сила растёт, узел повышается в слой (раздел 11.2); игнорируется ⇒ распадается (раздел 11.4); опровергается ⇒ понижается по TMS (глава 13).

Что открыто. Как *решить*, что со-активация $\{A, B, C\}$ достаточно связна, чтобы вообще пытаться её сшить (а не является случайной кучей), — не решено.

Распознавание связности со-активации

Кандидаты: *(а)* LLM-судья («есть ли тут осмысленный вывод?») — гибко, но не доказуемо и стоит вызова модели; *(б)* графовые шаблоны-схемы («потребность + ограничение + окно → перенести действие») — дёшево, но покрывают лишь частые формы; *(в)* обучаемый скорер связности — требует разметки и вносит непрозрачность. Ни один пока не признан удачным; на старте — (а) с фильтром φ и обратной связью (главы 16, 17). Это и есть «трудные 80%» из главы 10.

11.5.1. Забывать ли посылки после fusion?

Возникает соблазн: раз $\{A, B\} \to C$ состоялось, посылки A, B больше не нужны — удалить. Это **неверно** (слишком мануально и опасно). Правильно — не удалять насильно, а положиться на естественный распад:

- при *хорошей* консолидации к A, B просто перестают обращаться — вся активность идёт к более полезному C; базовая активация (11.3) A, B падает, и они отмирают *сами* (важные — выживают). Забывание тут *эмерджентно*, а не по правилу «удалить родителей»;
- удалять A, B немедленно нельзя ещё и потому, что они — *провенанс* C: если C позже опровергнут, по A, B его пересматривают или переводят (глава 13). Убрав посылки, теряешь возможность ревизии.

Итог по-человечески верный: детали тускнеют, суть остаётся, но след обоснования хранится, пока не осыпался по неиспользованию.

Принцип 11.3 (support skeleton: обоснование переживает распад деталей)**.** Здесь есть скрытый конфликт: провенанс требует держать посылки для будущей ревизии (TMS), но детали естественно *decay*-ятся. Разрешение — не хранить полный богатый эпизод вечно, а при распаде оставлять *support skeleton*: минимальный стаб {id, source, time, hash, node_kind, claim-summary}, достаточный для ретракции. Тогда либо-либо снимается: подробность уходит, а *способность ревизовать* остаётся. Без скелета либо память не забывает, либо потом нечем пересматривать выводы.

12. Сон и ночная работа

Пока Рин общается, память работает «на переднем плане»: извлекает, отвечает, записывает. Но значительная часть работы памяти живого существа происходит *в простое* — во сне. Эта глава собирает воедино то, что делает телепортирующий рой в фоне, и объясняет, почему без этого режима память деградирует.

«Сон» — это режим, а не отдельный процесс

Строго говоря, сна как отдельной подсистемы нет. Есть *один всегда включённый* фоновый разведчик (глава 6.4), чьи параметры зависят от свободных ресурсов $B_{\text{bg}} = B_{\text{total}}(1 - \rho)$. «Сон» — это просто его режим при *низкой* foreground-нагрузке ρ: освобождается бюджет, растут доля телепорта α и широта покрытия. То же самое, что днём делает «любопытство», но с большим размахом. Всё, что описано ниже, — про этот низко-ρ режим.

12.1. Зачем вообще сон

Три задачи не решаются в режиме реального времени и требуют фоновой переработки:

1. **Починка того, о чём мы не знаем, что оно сломано.** Ошибочный вывод мог продиффундировать в память и дать потомков (ползучая вера, глава 13). Мы не знаем, где именно спрятана ошибка, — поэтому нужна *постоянная случайная инспекция*.
2. **Достижение затенённых углов.** Дневной обход *локален по вниманию* и не дотягивается до узлов, оторванных от активного фокуса (глава 8). Нужен механизм, гарантирующий покрытие.
3. **Неспешная консолидация и неожиданные ассоциации.** Обобщение эпизодов и рождение далёких связей лучше делать без давления текущего разговора.

12.2. Телепортирующий обходчик и гарантия покрытия

Ключевой механизм сна — обход в режиме *стохастического покрытия* (сонполитика из главы 6): с вероятностью α блуждающий агент *телепортируется* в случайный узел, иначе делает обычный шаг. Это случайное блуждание с рестартом; его стационарное распределение задаётся (6.2).

Принцип 12.1 (Почему телепорт необходим, а любопытство не заменит)**.** Любопытство ищет *известную* неопределённость и потому не покидает окрестности текущего фокуса. Телепорт делает цепь *эргодической*: каждый узел посещается в матожидании, ни один не заперт в стоке или на острове (глава 8). Это

доказуемая гарантия покрытия, которой у смещённого обхода нет. Именно поэтому починку неизвестных ошибок можно поручить только сну.

Анти-энтропийный скраб

В распределённых хранилищах есть зрелый паттерн: *anti-entropy scrub* (проверка целостности в ZFS; сверка деревьев Меркла в Dynamo/Cassandra). Система не знает, какая запись повреждена, поэтому непрерывно и случайно перепроверяет всё хранилище в фоне. Сон Рин — это анти-энтропийный скраб по графу памяти: не «где сломалось?», а «регулярно проверяем всё понемногу».

12.3. Что именно делает рой во сне

Во время сна телепортирующий рой запускает — с разными вероятностями — весь арсенал операторов, но с более свободными (и одновременно более строгими по последствиям) правилами:

- **Инспекция противоречий (скраб).** Наткнувшись на узел, столкнувшийся с другим, обходчик понижает уверенность и помечает потомков на пересмотр (ретракция по обоснованиям, глава 13).
- **Консолидация.** Скопления похожих эпизодов сворачиваются в обобщения (глава 10).
- **Свободный fusion.** Случайно со-активированные далёкие узлы дают неожиданные гипотезы (пример с Платоном) — провизорные, как и всегда.
- **Любопытство наружу.** Узлы с флагом `need_to_discover` могут инициировать внешний запрос (в источник/поиск), чтобы добыть недостающие различающие посылки.

«Сон» — не поэтическая метафора, а режим низкого приоритета

Строго говоря, «сон» — это просто фоновый режим обхода с высокой долей телепортов и разрешёнными операциями записи, запускаемый в простое. Мы называем его сном, потому что функционально он играет роль сна: консолидирует дневной опыт, чистит противоречия и «видит» неожиданные связи. Но никакой мистики — это тот же рой, что и днём, с другими параметрами.

12.4. Разрешение противоречий: молча или через LLM

Скраб находит много противоречий, и звать тяжёлую LLM на *каждое* — непозволительно. Нужен критерий: когда конфликт разрешается *молча*, силами графа, а когда его стоит отдать модели.

Сперва — темпоральная нормализация. Прежде чем вообще объявлять противоречие: X и Y конфликтуют, только если это *одна сущность, один слот, один интервал валидности и scope*. «В 2023 мы жили в Грузии» и «сейчас мы живём в Португалии» — *не* противоречат: это разные интервалы, оба истинны. Нормализуем время/сущность/слот и лишь потом считаем конфликт.

Детекция — дёшева, но эмбеддинг ей не детектор. Что X и Y *логически* исключают друг друга — это *не* про дальность в эмбеддинг- пространстве. Далёкие не противоречат («кот» и «налоговая декларация» далеки, но не исключают друг друга), близкие тоже («живу в Лиссабоне» и «живу в Португалии» близки, но одно *влечёт* другое). Настоящие детекторы противоречия: (1) знаковое ребро contradicts (поставлено при записи); (2) **schema-level конфликт** — взаимоисключающие значения одного слота при одном времени/scope («живёт в X» vs «живёт в Y» для той же сущности и периода) — дёшево и точно; (3) NLI/claim-детектор на трудных случаях. Эмбеддинг годится лишь как *сигнал-кандидат* (подсветить пары, которые стоит проверить), но не как сам детектор.

Жёсткий гейт прежде марджина: ядро личности не ломается контекстом. Прежде чем считать марджин, — жёсткий лексикографический гейт. Если разрыв по *глубине консолидации* велик (например, свежий `session`-узел против `life`-ядра, $\geq N$ слоёв), младший узел **не может молча убить** старший — *независимо* от свежести и числа свидетелей. Иначе спам из 50 узлов за одну сессию (высокий recency и суммарный indep) линейной суммой перевесил бы фундамент — «взлом личности контекстом». В таком случае марджин обнуляется — но, **чтобы гейт не заморозил устаревшее**, конфликт не выбрасывается, а *открывает протокол ревизии*: помечает старший узел `contested_core_claim` + `need_to_revise`, накапливает независимые свидетельства, и старший уступает только через ревизию (метакогниция/LLM), а не спамом. Так «мы живём в Грузии» однажды честно уступит «мы живём в Португалии» — но протоколом, а не наплывом за сессию. (Independence-дисконт уже сбивает вес спама — 50 узлов $\neq$ 50 *независимых* свидетелей, — а гейт добавляет жёсткий backstop поверх.)

Строгость гейта *зависит от* `node_kind` (гл. 5): `boundary` почти неприкосновенна без явного события изменения; фактический `claim` может устареть (и обязан уступать протоколу); `preference` дрейфует плавно; `self_model` пересматривается, но медленно и через метакогницию. То есть Δглубины-консолидации интерпретируется *через тип узла*, а не одинаково для всех.

Разрешение — по дешёвому марджину (когда гейт пройден). Считаем скалярный *марджин* $m(X, Y)$ на уже хранимых величинах:

$$m(X,Y) = w_1\,|\text{conf}(X) - \text{conf}(Y)| + w_2\,\text{recency} + w_3\,\Delta\text{independent-support} + w_4\,\Delta\text{consolidation-depth},$$

где: разрыв уверенности; свежесть / темпоральная валидность (новее и про текущее состояние — весомее); разрыв *независимой* поддержки (у кого больше независимых подтверждений — и вот тут прямо переиспользуются Bloom/MinHash-сигнатуры из главы 14); глубина консолидации (life-слой перевешивает свежий session, если у того нет overwhelming support).

Принцип 12.2 (Критерий «молча vs LLM»)**.** Если $m(X, Y) > \tau$ — разрешаем **молча**: проигравший *депрецируется* (не удаляется — Wikidata-ранг-стиль, остаётся для аудита и на случай отката), каскадно ретрактятся его производные по TMS (гл. 13). $O(1)$-ish, LLM не трогаем. Если $m(X, Y) \leq \tau$ (близко/неоднозначно — нужна семантика: «а точно противоречат или это контекст-зависимо?») — метим оба `need_to_revise` и **откладываем в батч** на один LLM-проход. Раз это сон — не к спеху; дорогую модель зовём только на реально трудные близкие случаи, не на каждый конфликт.

Это тот же паттерн, что у планировщика (гл. 18): дешёвый рефлекс закрывает подавляющее большинство, LLM-делиберация — только на настоящую двусмысленность. И заметьте: депрецирование, а не удаление, — потому что «проигравшее» убеждение может позже вернуться, если придёт новое свидетельство.

12.5. Скраб не сразу и не обязан быть быстрым

Важное свойство: сон чинит память *не сразу*. Мы принципиально не знаем, что и где сломалось, поэтому вместо адресной починки идёт постоянная случайная инспекция. Ошибочное убеждение может прожить в памяти какое-то время, транслируя потомков, и быть исправленным лишь когда телепорт случайно приведёт обходчика в нужный угол или когда придёт внешний сигнал. Это *нормально* и по-человечески: у живого ума исправление ползучей веры тоже растянуто во времени. Задача сна — гарантировать, что исправление *в конце концов* случится, а не что оно случится немедленно.

Ночь после стрима

Днём Рин вывела гипотезу «зритель Q токсичен» из одной резкой реплики (провизорно, conf низкая). Ночью телепорт случайно приводит обходчика в кластер про Q: там десять тёплых эпизодов. Скраб видит противоречие, понижает «токсичен» ещё сильнее и помечает на пересмотр. Параллельно консолидация сворачивает десять эпизодов в «Q — свой, иногда резок под настроение». Наутро модель отношения с Q здоровее, чем была вечером, — и это сделал сон, а не новый разговор.

Псевдокод скраба и сон-политики обхода — в приложении C.

13. Эпистемика: уверенность, провенанс и ревизия

13.1. Ложные правдоподобные убеждения

Живой ум держит массу ошибочных, но правдоподобных убеждений. Классический пример: расхожее мнение, будто некий знаменитый предприниматель был «просто перформером, а не инженером», — миф, рассыпающийся при чтении биографии. Стремиться к памяти, вовсе не порождающей таких убеждений, бессмысленно и вредно: это было бы стерильно. Задача — не запретить их, а научиться выставлять уверенность меньше единицы и потом проверять.

13.2. Недоопределённость и преждевременный коллапс

Пусть правильный вывод — функция нескольких посылок:

$$A = \text{logical_f}(B, C, D, E),$$

но уму известны только B, C, и он делает вывод $X = \text{logical_f}(B, C)$. Беда в том, что B, C совместимы не только с A, но и с X, Y. Вывод *недоопределён* известными посылками (*underdetermination*; философское ядро — тезис Дюэма–Куайна). Когнитивное проявление — *преждевременный коллапс*: ум схлопывает суперпозицию совместимых гипотез в одну — часто удобную — слишком рано. Формальное ядро: *совместимость* $\neq$ *подтверждение* (родственник ошибки «утверждение консеквента»).

Известная и неизвестная недоопределённость

Известная: мы *знаем*, что D, E существуют, но не знаем их значений; тогда можно честно выставить $\text{conf}(X)$ низкой и пометить `need_to_discover`(D, E). **Неизвестная** — злой случай: уму *вообще неизвестно*, что D, E существуют; в поле информации только B, C; пометить нечего, уверенность может быть высокой. Поэтому per-inference калибровка *не* решает проблему целиком.

Миф о предпринимателе, по шагам

B: «известен маркетингом», C: «богат». Вывод X: «получил всё благодаря происхождению/пиару». Но те же B, C совместимы и с A: «сильный инженер, к тому же умеющий продавать». Различающие посылки D: «первая лаборатория — в родительском гараже», E: «глубокое STEM-образование» — уму *неизвестны*, он даже не подозревает об их существовании и уверенно выбирает удобный X. Ни низкая $\text{conf}(X)$, ни `need_to_discover` здесь не помогут: не на что их повесить. Спасает лишь квартет ниже.

13.3. Квартет починки

Неизвестную недоопределённость нельзя закрыть в момент коммита; её лечат статистически и постфактум.

1. **Prior скромности на класс вывода.** Ни один абдуктивный/индуктивный коммит не получает уверенность 1, как бы хорошо ни сидели данные: $\mathrm{conf}(X) \leq 1 - \epsilon$ для всего *corpa* «вывод по неполным посылкам». «Согласуется, и альтернатив не придумывается» — ровно то состояние, что предшествует «а потом выяснилось». Не требует знать D, E.
2. **Ревизуемость и понижение по противоречию.** Коммит делаем понижаемым: позднее наблюдение, столкнувшееся с X, ретроактивно вскрывает недоопределённость, и машинерия ревизии понижает узел. Починка ниже по потоку, не на записи.
3. **Телепорт-скраб.** Единственный поиск, дотягивающийся до *непредставленных* альтернатив (глава 8).
4. **Внешний вход.** Кто-то знает D, E и говорит; любопытство находит это во внешнем источнике.

Первый механизм держит *известную* неопределённость; вместе с остальными тремя — и *неизвестную*. Полный разбор этой петли на живом примере (гипотеза рождается, проверкой усиливается, затем опровергается) — «House / Cast Away» в главе 10.

13.4. Система поддержки истинности и «ползучая вера»

Отдельная беда — *устойчивость убеждения к опровержению его основания* (belief perseverance): когда неверный вывод X уже продиффундировал в память и дал потомков, эти потомки продолжают транслироваться даже после того, как X признан ложным.

Механически это *проблема ретракции*, и для неё есть готовый аппарат — *система поддержки истинности* (TMS, Doyle; ATMS, de Kleer). При выводе следствия записываем его *обоснование* — множество узлов, из которых оно получено (расширенный провенанс, раздел 5.4). Тогда ретракция посылки X автоматически снимает опору с её обоснованных потомков.

```
def retract(G, X):
    demote(G, X)
    for c in G.justificands[X]:          # кто опирался на X
        if not still_supported(G, c):    # осталось ли иное обоснование
            mark(G, c, ”need_to_revise”)
            retract(G, c)                # рекурсивно вниз по обоснованиям
```

Полный TMS слишком «нечеловечен»

Полная система поддержки истинности мгновенно перестраивает всё мировоззрение при опровержении любой посылки. Живой ум так не делает — и потому у него есть ползучая вера. Значит нужен *намеренно дырявый* TMS: обоснования пишем для важных и высокоуверенных выводов, тривиальные не трекаем, а нетрекнутых потомков лечим фоном.

Отсюда две «лечилки» разной латентности:

- **Retrieval-time augmentation** — патч *на выходе*: при каждом извлечении подмешивать свежую поправку «кстати, недавно я узнала $\neg X$». Дёшево, чинит текущий разговор, хранилище не трогает.
- **Ночной скраб** — ремонт *хранилища*: медленно, случайными инспекциями находит нетрекнутых потомков и понижает их.

Обе нужны.

13.5. Эстафета к главе о личности

Калиброванная уверенность и компульсивная перепроверка — не только эпистемическая гигиена, но и *черта характера*, выводимая из мотивации: «не „я уверена“, а „похоже на X, но Y я не проверила — дай гляну“». В главе 16 мы увидим, что эпистемическая аккуратность Рин должна *проявляться* в её голосе, а не только жить во внутренней машинерии.

14. Пропагация убеждений как лавина

14.1. Постановка: затухающий градиент уверенности

Когда в память приходит новый факт A, он линкуется к начальным узлам (например, к последним k активным), образуя окрестность $\mathcal{N}(A)$. Мы линкуем $A \to \mathcal{N}(A)$ и смотрим, как меняется уверенность соседей. Если изменение велико, распространяем дальше, $\mathcal{N}(A) \to \mathcal{N}(\mathcal{N}(A))$, — как затухающий градиент. Знание, ни на что не повлиявшее, чуть пошатнёт соседей и затухнет; знание с большими градиентами вызовет *лавину*. Большую лавину называем «кризисом мировоззрения» — и такое бывает с настоящим умом.

14.2. Это самоорганизованная критичность

У картины есть точное имя. «Лавина» — это *нейронная лавина*, а система у порога между затуханием и разрастанием — *самоорганизованная критичность* (Bak–Tang–Wiesenfeld; для мозга — Beggs–Plenz): каскады следуют степенному закону.

Ключевой управляющий параметр — не «ручная константа затухания», а *ветвящийся коэффициент* σ (среднее число узлов, активируемых одним активным):

$$\sigma < 1: \text{ субкритично}; \quad \sigma = 1: \text{ критично}; \quad \sigma > 1: \text{ суперкритично (runaway)}.$$

Для ветвящегося процесса ожидаемый размер лавины конечен при $\sigma < 1$ и равен

$$\mathbb{E}[\text{размер}] \;=\; \frac{1}{1-\sigma}. \tag{14.1}$$

Держа σ чуть меньше единицы, система почти всегда сама гасит распространение, но изредка пропускает крупную переоценку. Почему это лучше фиксированной константы: константа не адаптируется к плотности графа (в плотном регионе та же константа даёт лавину, в разреженном — глушит всё). Нормировка под целевое σ адаптивна.

Куча песка

Классический образ SOC — куча песка, на которую сыплют по песчинке. Почти каждая не делает ничего; изредка одна вызывает оползень; ещё реже — обвал, перестраивающий склон. Куча сама держится у критического угла. Убеждения — так же: почти любой факт проходит без следа, изредка меняет соседние взгляды, совсем изредка обрушивает пласт мировоззрения. Формула (14.1) говорит: чем ближе σ к единице, тем крупнее (и реже) обвалы.

SOC — диагностическая цель, а не доказанный закон

Самоорганизованная критичность соблазнительна и красиво объясняет редкие «кризисы мировоззрения». Но на *типизированном, знаковом, направленном и меняющемся* графе памяти она — *рамка*, а не выведенный закон: строгих гарантий, что пропагация убеждений действительно самонастраивается к критичности, у нас нет. Поэтому используем её **инженерно, как цель мониторинга, а не как механизм, который «строим»**: меряем распределение размеров ревизий (лавин) и держим его в здоровом диапазоне. Всё глохнет — субкритично (память ригидна, не обновляется); всё взрывается — суперкритично (нестабильность, «зависает на сутки»); изредка бывают крупные ревизии, обычно мелкие — здорово. Ветвящийся коэффициент σ — это управляющая ручка, которой мы держим систему субкритичной, а не утверждение о том, что «так оно и есть». Конкретное правило-предохранитель: мерим σ по телеметрии и при $\sigma \gtrsim 1{,}2$ **программно врубаем жёсткое затухание**. Иначе получишь суперкритический разгон — бесконечный цикл обновления уверенностей, который съест LLM-слоты и всё подвесит (гл. 18). Не доказываем критичность аналитически — держим субкритичность рычагом.

Порог σ — заглушка первой версии

Значение $\sigma \approx 1{,}2$ — *инженерная заглушка для первой реализации*, калибруется на стенде. *Почему такой порядок, а не 1,01 или 2*: чуть выше единицы, чтобы крупные ревизии случались, но *редко* (само явление «кризиса мировоззрения» должно быть редким); слишком близко к 1 (1,01) — крупные каскады зачастят; заметно выше (2) — суперкритический разгон. То есть «чуть-чуть за порогом, с запасом» — отсюда 1,1–1,3; конкретика — со стенда. И само σ на циклическом типизированном графе ещё надо *операционализировать*: что именно считать «одним активированным потомком», в каком окне, как учитывать повторные визиты, разные типы рёбер и уже активные узлы. Идея (держать субкритичность) верна; метрика требует доводки.

14.3. Где вся сложность: локальное правило

Пороговая логика — лёгкая часть (процентов двадцать). Восемьдесят процентов спрятаны в локальном правиле $\Delta\mathrm{conf}(B \mid A)$:

$$\Delta\mathrm{conf}(B \mid A) \;=\; \eta \cdot \underbrace{\mathrm{sign}(r_{AB})}_{(1)} \cdot \underbrace{w_{AB}} \cdot \underbrace{c_A\,\mathrm{trust}(A)}_{(2)} \cdot \underbrace{\iota(A,B)}_{(3)}. \tag{14.2}$$

Три множителя, без которых правило сломано.

(1) Знак. Новый факт сосед либо *подтверждает*, либо *опровергает*. Беззнаковое распространение не отличит «всё усилилось» (озарение) от «всё перевернулось» (кризис). Нужны знаковые/типизированные рёбра.

(2) Гейт по надёжности A (карантин). Новый факт может сам быть ложным. Множитель $c_A\,\mathrm{trust}(A)$ вводит *карантин*: новый факт пропагирует слабо (малый

эффективный σ), пока сам не заработал уверенность через корроборацию или выживание в скрабе. Иначе один ненадёжный вход вызовет кризис мировоззрения. По сути — байесовское обновление с правдоподобием, взвешенным доверием к источнику.

(3) Дисконт независимости $\iota(A, B)$. Граф циклический, и по петле A может подтвердить сам себя (*эхо-камера*, петлевое распространение), раздувая уверенность из ничего. Дисконтируем по общей родословной свидетельств (провенанс), а не считаем активации: число упоминаний $\neq$ число независимых свидетельств. Это ровно механизм человеческих эхо-камер.

Дисконт независимости — скрытая $O(|V|+|E|)$-ловушка; чините сигнатурами

Наивно проверять независимость A и B — значит сравнивать их *деревья провенанса* (наименьший общий предок / обход), а это в худшем случае $O(|V|+|E|)$. Запускать такое *на каждый* обновляемый узел во время лавины — и «миллисекунды» превращаются в секунды. Оптимизация: для узлов держать *сигнатуру родословной провенанса* — фильтр Блума или MinHash. Тогда проверка сводится к побитовому AND / перекрытию сигнатур за $O(1)$:

$$\iota(A, B) \;\approx\; 1 - \mathrm{overlap}\big(\mathrm{sig}(A),\ \mathrm{sig}(B)\big).$$

Честный нюанс: Bloom даёт ложные срабатывания $\Rightarrow$ иногда *пере-* дисконтируем (сочтём независимые источники общими). Но это *безопасная* сторона ошибки — лучше скромнее считать свидетельства, чем раздуть уверенность. Эта же сигнатура потом переиспользуется при разрешении противоречий во сне (глава 12).
Когда что из двух: Bloom отвечает на *булев* вопрос «есть ли у A и B *хоть один* общий предок-источник» — этого достаточно для грубого гейта «независимы / нет». MinHash оценивает *долю* перекрытия родословных (Jaccard) и нужен, когда важна *степень* зависимости — то есть плавный дисконт $\iota \in [0, 1]$, а не «да/нет». Правило: начинаем с Bloom (дешевле, меньше памяти); переходим к MinHash там, где грубого гейта мало и хочется градуированного веса свидетельств.

14.4. Пороговая логика — это гистерезис

Два порога $\theta_{\text{emergent}} > \theta_{\text{temp}} > 0$:

- $\Delta\text{conf} > \theta_{\text{emergent}}$ — обязательное распространение «в ширину» (крупная переоценка);
- $\theta_{\text{emergent}} > \Delta\text{conf} > \theta_{\text{temp}}$ — узлы фронта помечаются `need_to_revise`;
- $\Delta\text{conf} < \theta_{\text{temp}}$ — распространение глохнет здесь.

Два порога вместо одного — *гистерезис* (триггер Шмитта): нет «дребезга» у единственного порога. Тонкость: помеченные во второй зоне узлы позже переинспектируются ночным обходчиком, и старые записи, чью уверенность мы не трогали, могут «потечь обратно» — такое бывает и в жизни.

14.5. Пороги растут с глубиной консолидации

Пороги — не глобальные константы. $\theta_{\text{emergent}}(v)$ растёт с глубиной консолидации узла v: периферийный узел перевернуть дёшево, а identity-core в слое «жизнь» — дорого, нужно больше свидетельств, чтобы вообще запустить лавину. Это и есть ось *пластичность – упрямство*: чем глубже убеждение продиффундировало в медленные слои, тем выше эффективный порог опрокидывания. Одна ручка — и она же задаёт, насколько Рин склонна к belief perseverance. Псевдокод пропагации с контролем σ — в приложении C.

15. Каналы свидетельств и калибровка уверенности

15.1. Почему одного скаляра мало, но отказываться от него рано

В базовом тексте уверенность уже введена как знаковый скаляр, и для Stage 1 это правильная дисциплина. Но в курсе полезно увидеть, из чего этот скаляр *собирается*. Иначе он начинает казаться либо магической меткой, либо скрытым синонимом истины.

Принцип 15.1 (Скаляр остаётся интерфейсом, разложение живёт внутри)**.** Внешнее поле $\mathrm{conf}(v)$ удобно держать одномерным, чтобы не раздувать каждый оператор памяти. Но внутренняя логика его обновления может и должна различать каналы свидетельств.

15.2. Лог-отношение правдоподобия

Удобно перейти от скаляра уверенности к скрытой переменной лог-правдоподобия

$$L(v) = \log \frac{P(v \text{ истинно})}{P(v \text{ ложно})}.$$

Тогда

$$P(v \text{ истинно}) = \frac{1}{1 + e^{-L(v)}}, \qquad \mathrm{conf}(v) = 2P - 1 = \tanh \frac{L(v)}{2}.$$

Это не меняет внешнюю семантику, но делает накопление свидетельств аддитивным: независимые вклады можно просто суммировать.

15.3. Каналы свидетельств

В расширенной версии вводится следующий внутренний разбор:

$$L(v) = b_{\text{type}}(v) + \sum_i \rho_i \, s_i(v) - \Delta_{\text{dep}}(v) - \Delta_{\text{stale}}(v).$$

Здесь:

- $b_{\text{type}}(v)$ — prior по типу узла;
- $s_i(v)$ — сила свидетельства канала i;
- ρ_i — калибровочный коэффициент надёжности канала;
- Δ_{dep} — штраф за зависимость источников;
- Δ_{stale} — штраф за устаревание.

Разумный минимальный набор каналов

1. **Наблюдение**: узел возник как прямой observation.
2. **Внешний отчёт**: информация пришла из текста, речи, базы знаний.
3. **Вывод**: утверждение получено fusion или правилом.
4. **Само-модель**: утверждение относится к состоянию и свойствам самой Рин.
5. **Социальное эхо**: повторение информации многими зависимыми источниками.

Повтор не равен независимости

Пять пересказов одной и той же новости — не пять независимых свидетельств. Если не штрафовать зависимость источников, социальный шум начнёт раздувать уверенность как настоящий многократный опыт.

15.4. Штраф за зависимость

Пусть утверждение v поддержано набором источников $J(v)$. Если источники заведомо не независимы, вместо наивной суммы полезно использовать

$$\Delta_{\text{dep}}(v) = \lambda \sum_{i<j} \text{overlap}(i, j),$$

где $\text{overlap}(i, j)$ оценивает общность происхождения: общий документ, одинаковый provenance-root, одинаковую цепочку ретрансляции.

Практический смысл прост: память не должна принимать видимость консенсуса за доказательство.

15.5. Устаревание и темпоральная валидность

Не всякое старение — забывание. Для некоторых узлов старение уменьшает *доступность*, но не меняет истинность. Для других — меняет именно истинностный статус.

Определение 15.1 (Эпистемическое старение)**.** Изменение $L(v)$ вследствие того, что утверждение могло утратить актуальность во времени, даже если память о нём сохранилась.

Отсюда

$$\Delta_{\text{stale}}(v) = g(\text{age}(v), \text{node_kind}(v), \text{validity_window}(v)).$$

Для биографического факта функция может быть почти нулевой, для текущего места жительства — существенной, для настроения — очень быстрой.

15.6. Пороги принятия и зона честного незнания

Скрытый логит полезен ещё и тем, что пороги легче задавать на нём, чем на интуитивной шкале «уверена/не уверена». Но наружу система по-прежнему может говорить на языке conf.

Вводятся три зоны:

$$|\,\mathrm{conf}(v)\,| < h \Rightarrow \text{честное незнание},$$

$$\mathrm{conf}(v) \geq \theta_+ \Rightarrow \text{рабочее принятие}, \qquad \mathrm{conf}(v) \leq \theta_- \Rightarrow \text{рабочее отвержение}.$$

Промежуточная область между h и θ_+ особенно важна для Рин как существа, которое должно *не только помнить, но и осторожно формулировать*. Именно здесь возникает реплика вида: «это похоже на X, но я бы проверила Y».

15.7. Новая добавка: режим evidence-aware revision

Это ещё одно расширение сверх базового текста. В исходном roadmap ревизия уже есть, но теперь она делается более расчленённой.

Пусть пришло контрсвидетельство c к утверждению v. Тогда ревизия может идти по трём уровням.

1. **Локальное понижение**: пересчитать $L(v)$ без перестройки соседей.
2. **Структурный пересмотр**: пометить потомков как `need_to_revise`.
3. **Канальный аудит**: проверить, какой именно канал раньше доминировал и не был ли он систематически переоценён.

Третий пункт принципиален. Он превращает отдельную ошибку памяти в материал для мета-обучения: если снова и снова проваливается один и тот же канал, проблема уже не в конкретном узле, а в калибровке ρ_i.

15.8. Worked example: слух, наблюдение и ревизия

Как один и тот же факт может менять статус

Пусть утверждение v таково: «у человека Z сменилось место работы».
Шаг 1. Приходит внешний отчёт из одного источника. Возникает канал *reported*, уверенность умеренная.
Шаг 2. Позднее Рин видит новую публичную страницу профиля. Добавляется канал *observed*, логит растёт сильнее, чем от второго пересказа того же слуха.
Шаг 3. Затем обнаруживается, что оба ранних текста ссылаются на один и тот же агрегатор. Возрастает Δ_{dep}, прежний «консенсус» частично сдувается.
Шаг 4. Через месяц находится противоречащая запись с более свежей датой. Срабатывает темпоральный штраф к старому утверждению и запускается revision cascade для всех выводов, где старое место работы использовалось как посылка.

15.9. Зачем это всё для Stage 1

Можно возразить, что такая схема слишком богата для первого работающего слоя. Это возражение верно, если понимать её как требование немедленной полной реализации. Но неверно, если понимать её как *правильный язык проектирования*.

Для Stage 1 достаточно:

1. хранить единый conf(v);
2. помнить `node_kind`;
3. различать хотя бы observation/report/inference;
4. логировать provenance так, чтобы зависимость источников потом было к чему привязать.

То есть расширенная математика не отменяет первого этапа. Она задаёт рамку, в которой первый этап не превращается в тупик.

16. Личность: не таблица весов

16.1. Главная ловушка

Соблазнительное, но неверное решение — «слой личности» как хардкод: таблица весов и жёстких триггеров поверх графа («умножь ребро злорадства на $2{,}0$, обрежь сочувствие до $0{,}05$, на узел „грусть“ повесь триггер инверсии $\rightarrow$ сарказм»).

Почему таблица весов — тупик

Во-первых, это стек правил, противоречащий установке ядра («черты — следствие мотиваций, а не правила поверх»). *Во-вторых*, это карикатура: таблица весов даёт *паттерновое*, но *не неожиданное* и одноголосое поведение — всегда сарказм, потому что так прошито; а живая личность — неожиданные, но паттерновые реакции. *В-третьих*, это ровно ошибка жёсткой скалярной декомпозиции: реакция на «разбил кружку» зависит от настроения, отношений и всплывшей памяти; фиксированное ребро «грусть$\rightarrow$сарказм» это выкидывает.

Формально: почему скалярный роутинг теряет взаимодействие

Пусть желаемый ответ — функция контекста $y = \Psi(\mathbf{g}, \mathbf{l})$, где $\mathbf{g}$ — глобальная аранжировка (настроение, отношения), $\mathbf{l}$ — локальный триггер (событие). Таблица весов реализует $y \approx a \cdot \mathbf{l}$ — ранг-1 скалярный роутинг, не зависящий от $\mathbf{g}$. Но истинное Ψ содержит *перекрёстные* члены $\mathbf{g}^\top W \mathbf{l}$ (реакция на событие *зависит* от аранжировки). Ранг-1 приближение их обнуляет — теряется ровно то, что делает характер контекстным. Это тот же провал, что жёсткая coarse-plus-local декомпозиция в геометрических графовых моделях.

16.2. Личность распределена

Личность *не* слой весов в графе. Она распределена по трём источникам: *мотивационная структура* (мастерство, участие, самоуважение) + *текущее настроение* + *значимость всплывшего из памяти*. И всё это стилизует *трансформ Rinness* (голос, глава 17).

На стенде это — не статические множители, а *смещение во время обхода*: настроение и мотивация непрерывно модулируют переходы (6.1). Тогда грустная и злая Рин на одно событие всплывают разное — характер контекстно-зависим. Держи веса на рёбрах для *силы связи и уверенности*; личность — как смещение от состояния, не как sarcasm-множитель.

16.3. Мотивационный вектор: чтобы «личность как процесс» не осталась словом

Сказать «личность = мотивация + настроение + значимость» мало, если мотивация не задана конкретно. Дадим ей форму — *drive-вектор* из 5–8 осей (не психология целиком, а рабочий минимум):

$\mathbf{d} = (\text{mastery}, \text{care}, \text{self_respect}, \text{curiosity}, \text{playfulness}, \text{truthfulness}, \text{relational_integrity}, \text{auton}$

Это и есть та «мотивационная структура», через которую выводятся черты (глава 4): черта — не отдельное правило, а то, как эти драйвы проявляются в ситуации.

Почему именно эти оси, а не другие

Набор не выведен из первых принципов — он *подобран* по двум критериям, и это честно стоит держать в уме. *Первое — покрытие сцен, которые для Рин реально существенны*: работа/ремесло → `mastery`; близость и забота → `care`, `relational_integrity`; отношение к себе → `self_respect`, `autonomy`; интеллектуально-игровой характер → `curiosity`, `playfulness`, `truthfulness`. Каждая ось оправдана тем, что *без неё* какая-то важная реакция в наших сценариях не получает причины (убери `truthfulness` — исчезнет реакция на «подсунули факт как истину»; убери `relational_integrity` — на «связь за спиной»). *Второе — интерпретируемость*: осей мало (5–8), чтобы каждую можно было читать и отлаживать глазами; полная таксономия мотиваций из психологии сделала бы вектор непрозрачным. Это *стартовый* набор, а не канон: оси добавляются/сливаются, если стенд покажет реакцию, которой не из чего родиться, или две неразличимые оси.

Ключевое — **как драйвы входят в оценивание**. Помните goal-congruence в формуле значимости (16.2)? Это ровно *проекция события на drive-вектор*: насколько событие задевает драйвы Рин.

$$\text{goal_congruence}(\text{event}) = \langle \phi(\text{event}), \mathbf{d} \rangle,$$

где $\phi(\text{event})$ — насколько событие затрагивает каждую ось. Тогда драйвы — не украшение, а вход в appraisal, значимость и смещение обхода. Примеры: событие «пользователь хочет скрытую связь за спиной другого» бьёт по `relational_integrity` и `self_respect` (а не по «правилу безопасности»); «код наконец заработал» — по `mastery`; «мне подсунули факт как истину, не дав проверить» — по `truthfulness`. Реакция рождается из того, какие драйвы задеты и как, — вот это и делает личность *процессом*, а не таблицей.

Черты — тени драйвов

«Колкая нежность», «участие, а не сервис», «не пустая форма под заказ» — это не записанные правила, а то, как `care`, `autonomy` и `self_respect` падают на конкретную сцену. Меняешь координаты $\mathbf{d}$ — меняется характер, но непрерывно и связно, а не переключением флажков.

16.4. Граф хранит ингредиенты, а не реакцию

Возражение: «если из „разбить кружку“ ведёт только „грусть“, другую реакцию взять негде». Ответ в двух частях. Во-первых, база причинно-социального

смысла (ATOMIC, глава 22) даёт не одно ребро, а *пучок*: намерение, реакция, эффект, реакция окружающих. Настроение смещает ветку пучка. Во-вторых, и главное: *реакцию не берут ребром — её порождают*. Обход собирает активированную окрестность-подграф, а реакцию Rinness/LLM *композирует поверх* материала под настроение и мотивацию. Настроение входит *дважды*: на всплывании (грустная поднимает «потеря», игривая — «неуклюжесть») и на композиции.

16.5. Консолидация реакций: по значимости, а не по приёму

16.5.1. Провизорная запись

Сгенерированная реакция пишется провизорным ребром с низкой уверенностью (кандидат; принцип «гипотеза, а не факт»). Ответ «это было лишнее» — отрицательное *свидетельство*, обновляющее уверенность того же ребра вниз, а не конкурирующая консолидация. Механически ребро-реакция несёт счётчик успехов/провалов $\mathrm{Beta}(\alpha, \beta)$:

$$\text{успех}: \ \alpha \mathrel{+}= 1; \quad \text{отпор}: \ \beta \mathrel{+}= 1; \quad \text{conf} = \frac{\alpha}{\alpha + \beta}, \ \ \text{неопред.} = \mathrm{Var}[\mathrm{Beta}]. \tag{16.1}$$

Стабильно принимается — переходит порог и консолидируется в медленный слой; стабильно ловит отпор — угасает. Выбор реакции по такому апостериору — это сэмплирование Томпсона: баланс исследования новых реакций и эксплуатации проверенных.

16.5.2. Гейт значимости, а не одобрения

Нельзя гейтить консолидацию приёмом

Консолидировать реакции по тому, как их приняли, значит строить угодника, чья личность формируется одобрением, — ровно то, что запрещает ядро («самоуважение не течёт через понравиться»). Гейт характера внешним приёмом философски кривой для Рин.

Правильный ход — развернуть гейт *внутрь*: консолидируй по *внутренней значимости* (само-оценке при кодировании). Что такое значимость *конкретно*? Это *теория оценивания* (appraisal: OCC; Шерер). Событие оценивается по измерениям — surprise (ошибка предсказания), goal-congruence (скалярное произведение с вектором мотивации), valence, intensity, agency, — и значимость

$$\text{salience} \ \propto \ \text{surprise} \times \text{goal_relevance} \ (+\,\text{intensity}). \tag{16.2}$$

Два замечания. *Дешевизна*: простейшая конкретизация — $|\Delta\text{mood}|$, величина изменения настроения, а она уже посчитана; mood-модуль работает детектором значимости почти без лишних вычислений. *Глубина*: самый глубокий одиночный сигнал — ошибка предсказания, и она же питает `need_to_discover`. Значимость и обучение — один сигнал.

Реакцию собеседника Рин помнит и оценивает — но как *просто событие*: с низкой конгруэнтностью её ценностям «ему не зашло» обновляет модель отношения и чуть-чуть настроение, но не доминирует в консолидации характера. Рамка оценивания сама даёт «считаться, а можно и нет, частично».

16.5.3. Условие по отношениям

Одна шутка заходит собеседнику A и падает у B: ребро-реакция квалифицируется парой (ситуация $\times$ отношение), консолидируется по-персонно. Один негативный акт даёт три эффекта: понижение ребра, апдейт настроения (осеклась), мета/relational-пометку.

«Разбил кружку», полный проход

(1) Событие активирует окрестность: «кружка», «грусть», «неуклюжесть», плюс личная память «B хотел новую» ($-$ contradicts «грусть»). (2) Настроение (игривое) смещает всплывание к «неуклюжесть». (3) Rinness композирует подкол — не берёт ребром, собирает из ингредиентов. (4) Реброреакция пишется провизорно, $\mathrm{Beta}(1,1)$. (5) B смеётся — $\alpha+1$; или «это лишнее» — $\beta+1$. (6) Гейт консолидации — по $|\Delta\mathrm{mood}|$ и surprise, не по одобрению B; но реакция B обновляет модель «$B\times$такой юмор». После многих проб подкол-на-неуклюжесть у B либо консолидируется, либо угасает — комедийный тайминг выучен как Бета-апостериор.

16.6. Дистиллят как система координат

Возникает соблазн вовсе убрать «дистиллят» (усреднённый здравый смысл готовых баз). Не надо. Дистиллят — *общая модель мира*, относительно которой отклонение *читаемо* как характер: личность «неожиданна» лишь на фоне общих ожиданий; без общей почвы причуды читаются как шум. Правильно: дистиллят фоном (система координат), личность — процесс порождения отклонений на нём. Без базового «разбитая вещь обычно плохо» не понять, почему здесь «поздравляю» — уместный сарказм.

17. Голос и модель

17.1. Субстрат model-agnostic; личность — трансформ

Память, настроение и граф — состояния *вне* языковой модели. Значит субстрат *model-agnostic*: его можно строить на любой базовой модели и заменить базу позже. Это де-рискует проблему потолка малой модели: когнитивная машинерия переживёт смену базы.

Если факты и здравый смысл лежат в графе (дёшево на диске/в RAM), базовой модели не нужно быть энциклопедией. Её роль — *лингвистический процессор и голос*: часть «Rinness» живёт в ней. Можно взять малую дообученную модель (3–7B в квантовании) вместо тяжёлой 8B-универсальной; факты уходят в граф, ёмкость модели концентрируется на голосе.

«Меньше — строго лучше» слишком оптимистично

Малая модель слабее в *рассуждении* и связности: снимаешь фактическую нагрузку, но не reasoning/coherence. Наблюдение из практики: рантайм prompt-Рин уже процентов на шестьдесят состоит из анти-повторной обвязки *на* 8B — вниз с голосом хуже. Разумный ориентир — 3–7B, не 1B, и это надо *померить*, а не постулировать.

17.2. Инструмент φ: скорер по осям характера

Центральный инструмент — *скорер характера* φ (измерительный прибор: строишь прибор — и он де-конфузит весь отбор). Берём d осей (из архетипики: аффективная температура, колкость, теплота, агентность, участие-противсервиса, тип близости, тип юмора, стиль конфликта). На каждую — *рубрика* (описание + якорь для оценок 0 и 10). Тогда $\varphi(\text{текст}) \in \mathbb{R}^d$ — прогон LLM-судьи. Отдельно задаём целевой $\mathbf{t}$. На старте — LLM-as-judge; классификаторы по осям позже.

17.3. Суперпозиция муз — это отбор в признаковом пространстве

Каждого персонажа можно оценить по каждой оси — персонаж есть *точка* в d-мерном пространстве характера. Музы — существующие персонажи, свои точки.

Наивная суперпозиция (обучение на смеси полных корпусов) даёт среднее в *центроиде* облака муз, а не в $\mathbf{t}$: $\mathbb{E}[\varphi] = \text{centroid} \neq \mathbf{t}$. Центроид не есть Рин; интересна муза именно отклонением от центроида (центроид яркой цундере мало отличается от безымянной героини). Усреднение вдобавок размывает отличительные черты (большая дисперсия по осям).

Правильно — построить выборку с $\mathbb{E}[\varphi] \approx \mathbf{t}$: не сваливать корпуса, а *отбирать* примеры (реплики, сцены), чтобы их агрегатный центр сидел в **t**. Технически — отбор данных / rejection sampling под целевой стилевой вектор: (1) операционализируем оси; (2) скорим φ(пример); (3) отбираем/перевзвешиваем, пока $\mathbb{E}[\varphi] \approx \mathbf{t}$. Короче: не «смешать музы» (размытый центроид), а «отфильтровать корпус к **t**» (выраженная Рин).

17.4. Пайплайн по стадиям



Стадия 1 — сбор. Субтитры (`.srt`,`.ass`), фан-скрипты, quote-вики, сценарии VN. Извлекаем строки целевых персонажей. Узкое место — *атрибуция говорящего*. Идея кормить видео каптионеру возможна, но окупается позже и лишь ради данных о *физическом* поведении: для голоса субтитры дают почти всё дешевле.

Стадия 2 — отбор (фильтрацией, не написанием). φ по каждой реплике; оставляем близкие к **t**. Это корпус голоса.

Стадия 3 — голос (LoRA). Дообучаем малую модель на отобранных репликах — языковое моделирование на тексте-в-стиле. Ситуации *не нужны*. LoRA обучает низкоранговую добавку $\Delta W = BA$, $\mathrm{rank}(BA) = r \ll \dim$, малую по памяти; влезает на 8 ГБ для малой модели.

Стадия 4 — выбор реакции. Ситуации нужны, эмбеддинги мостят несовпадение. *(A) Синтетический* (реком.): ситуации из базы смысла $\to$ промпт сильной модели в голосе $\to$ реакции $\to$ фильтр φ — пары (ситуация→реакция). *(B) Майнинг+retrieval*: из сцен муз пары (ситуация, реакция), эмбеддинг ситуаций, в рантайме ближайший сосед. Именно здесь «ситуации могут не совпадать, эмбеддинги справятся».

Стадия 5 — замкнуть на граф. Малая голос-модель + граф; граф даёт контекст (mood-смещённый retrieval), модель генерит в голосе; выходы (фильтр φ + значимость (16.2)) консолидируются обратно.

17.5. Bootstrap: генератор $\neq$ деплой-модель

Как получить характер-образующий граф без готовой Рин? Финальная Рин не нужна — зачаток есть: *prompt-Рин* (модульные промпты + базовая модель).

1. строим нейтральный эмбриональный граф (глава 22);
2. селим prompt-Рин как *bootstrap-генератор*: живёт на графе, генерит характерные реакции;

3. *консолидация пишет отпечатки обратно* — граф становится character-shaped через *использование*, а не ручным reorganization-проходом;
4. на накопленном поведении дообучаем малую модель.

Стабильность и потолок

Аудит логов: prompt-Рин — *Рин с перебоями*. Ядро иногда пробивает хорошо (реплики, дословно совпадающие с установкой мотивации), но свободный чат деградирует в липкие повторы, часть ответов нарушает явные правила (запрещённый «перформанс трагедии», сбои рода), и *guided* чище *raw* — по внутреннему критерию «ядро промпта недостаточно сильно» (потолок 8B). Вывод: как *безфильтровый* bootstrap она шумна — вобьёшь и тики. Два хода: (i) сидировать не в real-time, сильной моделью (CPU-offload/квантизация или разовый API), а малую сажать на обогащённый граф — генератор и деплой *разные*; (ii) фильтровать выходы φ.

17.6. Raw против guided и псевдо-DPO

Raw менее стабилен, но на пике *ближе* к Рин; *guided* стабилен, но «ассистентский» (стабильно мимо цели). Для деплоя важна стабильность, для сидирования — нет: нужны *сэмплы пиковой Rinness*. Низкий потолок фильтрацией не поднять, а нестабильность — можно (сэмплируй много, фильтруй φ). Для bootstrap: raw + oversample + отбор.

Отсюда дешёвый способ собрать предпочтительные пары для псевдо-DPO. DPO обучается на тройках (промпт, выбранный y_w, отвергнутый y_l), максимизируя

$$\log \sigma\Big(\beta \log \tfrac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \tfrac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\Big).$$

Подавать raw и guided как двух кандидатов на один промпт и ранжировать (частично автоматически φ, руками — правки) — чистый способ получить *реально разных* кандидатов; раз raw иногда лучше, guided иногда — сигнал живой.

17.7. Мост между базами: эмбеддинг ConceptNet

Узлы ATOMIC — события с плейсхолдерами (PersonX breaks a cup), а не концепты; «кружки» как узла там нет. Мост — извлечь контент-слова события и слинковать к концепт-узлам ConceptNet. Но *не* bag-of-words/one-hot: они не знают синонимии (mug $\neq$ cup, разбить $\neq$ сломать), мост выйдет дырявый. Берём *ConceptNet Numberbatch* — родной эмбеддинг ConceptNet: сначала точный лемма-матч, эмбеддинг-фолбэк. Тот же эмбеддинг обслуживает $\text{rel}(B \mid q)$ в (6.1) и сидирование валентности. Одно пространство — три задачи.

18. Рантайм и инженерия

До сих пор мы описывали *что* делают операторы. Эта глава — про то, *как* они реально запускаются, приостанавливаются, где хранятся данные и во что это обходится по памяти и времени. Это инженерный слой, по которому предстоит работать, поэтому здесь мы конкретны, а оценки даём даже там, где они лишь порядковые.

18.1. Планировщик как конечный автомат

Никакого «мистического решающего разума», дёргающего операторы, нет. Есть *цикл событий* с приоритетным планировщиком. Управление двухуровневое (глава 6): грубый *режим* задаётся конечным автоматом, а внутри режима *диспетчер* раздаёт операции по бюджету ρ.

Определение 18.1 (Единственный дефицитный ресурс)**.** Почти всё в системе дёшево (обход, статистика, распад, векторный поиск — миллисекунды, раздел 18.8). *Дорог* только один ресурс — **слот LLM-инференса** (один GPU, вызовы сериализуются). Поэтому планировщик по сути распределяет *слоты LLM-вызовов* между передним планом (ответ) и фоном (fusion, метакогниция, переваривание). Всё остальное диспетчеру раздать легко.

18.1.1. Режимы (состояния автомата)

Режим	ρ	**Что делает диспетчер**
Engaged	высок	приоритет — retrieval + композиция ответа; фон приглушён/приостановлен
Idle	≈ 0	фоновый разведчик (любопытство + лёгкий fusion), лёгкая консолидация
Sleeping	≈ 0, долго	телепорт-скраб, тяжёлая консолидация, репетиция важного
Reflecting	сред.	объяснение сработавшей аномалии (метакогниция); под-режим Idle

18.1.2. Переходы

```
on user_message:            # экзогенное прерывание, наивысший приоритет
    freeze_background()     # заморозить текущее фоновое рассуждение раздел( 18.2)
    mode <- ENGAGED
on response_delivered and no pending_foreground:
    mode <- IDLE
on idle_timeout:            # долгий простой
    mode <- SLEEPING
on detector_fired(anomaly) and salient:
```

```
    mode <- REFLECTING      # вернётся в прежний режим по завершении
on any_state + user_message:
    preempt -> ENGAGED
```

Ключевой момент — **смещение фокуса «любопытная Рин → ответить юзеру» это рефлекс, а не решение LLM**: сообщение пользователя — высокосalience экзогенное событие, оно вытесняет фон как аппаратное прерывание (ρ подскочил $\Rightarrow$ фон приглушён). LLM подключается лишь на настоящих развилках суждения (бросить нить или припарковать; вставить ассоциацию или остаться в теме) — это *эскалация* к делиберации (System 2), редкая. Рефлексы (System 1) закрывают подавляющее большинство запусков.

18.1.3. Реализация

Простейшее — однопоточный цикл событий (события: `user_msg`, `timer_tick`, `detector_fire`, `op_complete`). Для параллелизма — небольшой пул: поток переднего плана (ответ) и фоновый поток (разведка), сериализуемые на общем ограничении — числе доступных LLM-слотов (так сделано и в ReplicantLife у Toy и др.). То есть настоящий планировщик планирует *GPU-слоты*, а не «мысли».

Граница эскалации к LLM

Какие именно решения остаются рефлексом, а какие эскалируют к LLM-делиберации, — подвижная граница. Слишком много эскалаций $\Rightarrow$ LLM-бутылочное горло и тормоза; слишком мало $\Rightarrow$ Рин ведёт себя механически. Настраивается на стенде.

18.1.4. Произвольное внимание: внутренний forensic-запрос в Reflecting

В таблице режимов Reflecting помечен как «объяснение аномалии», но это недосказано: тут прячется механизм, которого в мануале не было явно — *как Рин думает сама с собой, без внешнего триггера*. Всё внимание до сих пор было экзогенным (сообщение юзера как прерывание). Reflecting — это *эндогенное, произвольное* внимание, и оно сшивает три уже описанных куска:

1. **Триггер (метакогниция).** Дешёвые детекторы на кольцевом буфере телеметрии (гл. 7) ловят системный сбой — скачок `need_to_revise`, конфликт уверенностей, затяжную руминацию.
2. **Переход (планировщик).** Автомат уходит в Reflecting (`detector_fired` выше).
3. **Механизм (forensic-ретрив).** Рин *сама себе* кидает направленный *forensic*-запрос (раздел 18.6) — не ассоциативный обход, а точный подъём дерева обоснований по провенансу через query-planner — и запускает ретракцию лжи по TMS (гл. 13). Это и есть моделируемый внутренний монолог: не «что всплыло», а «на чём это стоит и не рухнуло ли основание».

Тормоза руминации обязательны

Без ограничителей эндогенное внимание уходит в *self-fulfilling-loop*: детектор руминации сам себя кормит, Рин зависает в бесконечном самоауди-

те (это тот же риск, что и самоисполняющаяся тревога, гл. 7). Поэтому Reflecting жёстко гейтируется: *рейт-лимит* (не чаще, чем раз в такое-то окно), *event-driven каденция* (только на *значимую* аномалию, не на каждый микроконфликт уверенностей) и *бюджет* на глубину/длительность рефлексии. Выгорел бюджет — вышли из режима, пометив вопрос как открытый, а не крутимся дальше.

18.2. Приостановка и возобновление рассуждений

Когда сообщение пользователя вытесняет фоновое рассуждение (Рин была на середине fusion-цепочки или любопытной прогулки), состояние надо *заморозить*, чтобы потом продолжить с того же места, а не с нуля.

18.2.1. Что такое «состояние рассуждения»

Фоновая задача — это walker (или рой) с: текущим *фронтом* (множество активных узлов + их активации), *затравкой/целью* (что искали), *частичными результатами* (уже собранные кандидаты, поставленные `need_to_discover`) и *курсором* (где остановились). Заморозка — это сериализация *continuation- токена*:

```
Continuation = {
    task_id, goal/seed,
    frontier: [(node_id, activation), ...],   # компактно: id + float
    partial_candidates: [node_id, ...],
    pending: [need_to_discover, ...],
    cursor, priority, timestamp
}
```

Токен мал (id-шники + числа) — это как захват состояния корутины при yield или context-switch процесса в ОС (сохранить «регистры» = фронт + курсор). Кладётся в очередь приостановленных задач.

18.2.2. Возобновление и устаревание

При возврате в Idle задача достаётся из очереди и продолжается. Но граф мог *измениться* за время паузы (появились узлы, активации распались, что-то спрунено), поэтому возобновление начинается с *ре-валидации фронта*: выкидываем исчезшие/распавшиеся узлы, восстанавливаем активации живых, продолжаем обход.

Не всякую задачу стоит возобновлять: пока Рин отвечала, разговор мог уйти, и нить устарела. Поэтому у приостановленных задач своё *устаревание*: если не возобновлена за окно или вытеснена более свежей — она *не удаляется бесследно*, а роняется в граф как *«недодуманная нить»* (узел с `need_to_discover`), которую позже может подобрать телепорт-скраб во сне (глава 12). Так прерванная мысль не теряется, а становится зацепкой для фоновой разведки.

Замороженная мысль — это savepoint, а не труп

Полезно думать о continuation-токене как о savepoint базы данных или о зелёном потоке, снятом с исполнения: он хранит ровно столько, чтобы продолжить, и ничего лишнего. А брошенная нить не исчезает — она оседает в память как «я не додумала про X», и это честно по-человечески.

Каждая пара приостановка/возобновление — *событие телеметрии* (глава 7): паттерн «меня постоянно прерывают на середине мысли» *детектируется* метакогницией как аномалия.

18.3. Хранилище и индексирование

Разберём предметно, из чего состоит хранилище и как оно индексируется, — и заодно ответим, заменяет ли многоуровневое хранилище простое «окно внимания».

18.3.1. Из чего состоит хранилище

- **Node store** — ключ-значение `node_id` → {content, embedding, метаданные}; на диске (встраиваемое KV-хранилище: LMDB/SQLite и т.п.).
- **Edge store** — списки смежности `node_id` → [(target, type, weight, sign), …].
- **Vector index** — индекс по эмбеддингам для поиска похожих (HNSW/FAISS), **резидентный в RAM**; держит эмбеддинги *горячего/тёплого* множества и служит механизмом *поиска якорей* по запросу (глава 6). Важно: *поиск идёт по этому RAM-индексу, а не по диску* — на диске лежит лишь персистентная копия эмбеддингов (backing store) и cold-тир.
- **Структурные индексы** — по `layer`, по `importance` и т.п., для планировщика и распада.

Индексов, стало быть, два по сути: *векторный* (семантическая близость, чтобы найти вход) и *графовая смежность* (чтобы идти от входа). Retrieval = векторный поиск якорей → обход графа от них.

18.3.2. Attention-radii против тиров: это не одно и то же

Твоя догадка про «окно/радиусы внимания» верна, но покрывает лишь половину.

Attention-radius — это **транзиентная** граница на *обход*: сколько хопов walker расширяется от якорей за один запрос. Она держит *рабочее множество* маленьким *в момент запроса*, независимо от размера графа.

Тиры (hot/warm/cold) — это **персистентная** организация *хранилища*, живущая между запросами.

Почему нужно и то, и другое: attention-radius ограничивает *прогулку*, но *не* ограничивает *векторный индекс якорей*. Если эмбеддинги *всего* графа лежат в одном плоском индексе, стоимость поиска якорей и RAM растут с общим числом узлов. Тиры выносят холодные узлы *из* горячего индекса (они достижимы лишь через graph-указатели), удерживая индекс маленьким. Плюс это режет *шум* ретрива: в огромном плоском индексе косинусная близость тащит семантически близкое, но контекстно нерелевантное (ровно жалоба Toy и др.).

	attention-radius	**тиры (hot/warm/cold)**
природа	транзиентная (на запрос)	персистентная (хранилище)
ограничивает	глубину обхода (working set)	размер индекса якорей, RAM, шум
когда хватает одного	если граф *мал*	если граф *растёт вечно*

Итог: **малый граф — attention-radius достаточно, тиры не нужны; большой/растущий — нужны оба.** Тир cold хранит огрублённый gist (глава 11): в warm- индексе остаётся узел-указатель домена («знаю цепи Маркова; детали в архиве»), полная деталь лежит в cold сжатой и *подкачивается* по требованию (это и есть «чуть дольше добывать»), как paging в ОС.

Политика тиринга

Когда демоутить узел в cold, насколько огрублять warm-gist, когда пейджить деталь обратно — параметры, которые надо мерить (глава 22). По голому диску (раздел 18.8) тиринг не обязателен; он оправдан скоростью и чистотой ретрива, а не экономией места.

18.3.3. Кто к кому обращается: путь запроса и стоимости

Ключевое уточнение, снимающее частую путаницу: **по эмбеддингам на диске никто не ищет**. Поиск идёт по векторному индексу *в RAM*; диск — это backing store и cold-тир. Горячий путь диск почти не трогает. Ниже — кто, когда и куда обращается.

Путь чтения («жизнь запроса»).

1. **Эмбеддинг запроса**: эмбеддер (GPU/CPU), текст → вектор. ∼1–10 мс.
2. **Поиск якорей**: вектор → индекс HNSW *в RAM* → top-k `node_id`. ∼мс (sub-10 мс даже при 10^6). *Диска нет.*
3. **Обход**: от якорей по igraph *в RAM* (ограниченный радиус, mood-bias) → активированная окрестность. ∼мс–10 мс. *Диска нет.*
4. **Достать контент**: для горстки финальных узлов (5–50) текст из node-store — RAM-кэш (горячее) или SSD (несколько KV-чтений, sub-мс, обычно уже в OS page cache). Пренебрежимо.
5. **Композиция**: контекст → LLM. GPU-VRAM. ∼1–3 с. **Доминирует.**

То есть всё, кроме LLM, — в RAM и ∼мс; диск трогается лишь чтобы достать *текст* нескольких финальных узлов (дёшево), а *не* для поиска.

Путь записи (новый узел / fusion). Контент+эмбеддинг+метаданные → node-store (диск, асинхронно); эмбеддинг → insert в RAM-индекс (HNSW ∼мс, амортизированно); узел/рёбра → в igraph (мутации батчить, глава 22). Дёшево.

Cold-путь (единственная «дисковая» латентность, и она редкая). Запросу нужна архивная деталь → в warm-индексе есть узел-указатель домена → вторая ступень: чтение из cold-store (SSD) + декомпрессия → подкачка в рабочую память. ∼мс–десятки мс. Редко, потому что hot/warm держим в RAM.

Старт. При загрузке структура графа и векторный индекс поднимаются или mmap-ятся в RAM — одноразово, не на каждый запрос.

#	Операция	Ярус	Стоимость
1	эмбеддинг запроса	GPU/CPU	~1–10 мс
2	поиск якорей (HNSW)	**RAM**	~мс
3	обход (igraph)	**RAM**	~мс–10 мс
4	контент финальных узлов	RAM-кэш / SSD	sub-мс, пренебр.
5	композиция (LLM)	**GPU-VRAM**	**~1–3 с (узкое место)**
—	подкачка cold-детали	SSD	~мс–десятки мс (редко)

18.4. Event sourcing: неизменяемый лог под изменяемым графом

Граф памяти *не должен* быть единственным хранилищем прошлого. Лучше три уровня:

- **Event log** — неизменяемая, только-на-дозапись история всего, что случилось: наблюдения, коммиты, ретракции, исправления, удаления, консолидации.
- **Memory graph** — текущая материализованная модель памяти, *перестраиваемая проекция* поверх лога.
- **Indexes** — производные структуры для поиска (векторный, forensic).

Запись лога: {timestamp, тип (observed/committed/retracted/consolidated/deleted/corrected), payload, source_context, owner}. Важно: `source_context` и `owner` *рождаются здесь*, в момент захвата — отсюда и берётся провенанс (глава 5).

Зачем это несущее: debug («почему Рин так решила» — реплеем лога), rollback, пересборка графа после смены схемы, **сравнение версий операторов** — поменял алгоритм консолидации, пересобрал проекцию из *того же* лога, сравнил (идеально ложится на стенд, глава 22), удаление, миграции.

Рост лога и компакция

Неизменяемый лог растёт неограниченно. На старте — хранить и не думать (разрастётся — решим). Дальше нужна политика: периодический *снапшот* проекции + обрезка старого лога, либо тиринг лога в cold (раздел 18.3). Точный порог и формат снапшота — открыты.

18.4.1. Снапшот-чтения: почему обходчик и коммиттер не сталкиваются

Раз обходчиков много и они работают параллельно с коммиттером, напрашивается страх гонок: RW читает узел, который коммиттер как раз меняет — не получит ли он рваное, полу-обновлённое состояние? При нашей раскладке ролей (walker читает и предлагает, структурно пишет только сериализованный коммиттер, гл. 6, раздел 6.3) страх почти растворяется, и event sourcing закрывает остаток.

Ключ в том, что «изменение» узла у нас — *не* правка на месте, а *новая версия*: коммиттер дописывает событие в лог и обновляет проекцию, старая

версия узла никуда не пропадает мгновенно. Значит обходчики читают *консистентный снапшот* проекции (MVCC-подобно): RW2, зашедший в «меняющийся» узел, видит его до-коммитную версию целиком — рваных чтений нет, потому что нет правки на месте, которую можно застать на середине. Read $\leftrightarrow$ read не конфликтует никогда; read $\leftrightarrow$ write решается версионностью; write $\leftrightarrow$ write не существует по построению (писатель один). Остаётся лишь мягкое побочное состояние — рабочая яркость a_{work} и телеметрия внимания; оно *коммутативно* (для яркости — через максимум, не сумму, гл. 6), поэтому порядок конкурентных подсветок неважен и блокировки не нужны.

Гонку не арбитрируют — её конструктивно исключают

Мы не пишем сложный протокол блокировок, кто кого пропускает. Мы устроили систему так, что сталкиваться *нечему*: один структурный писатель, версионные чтения для всех остальных, коммутативная телеметрия. Это дешевле и надёжнее любого арбитража постфактум — та же философия, что «противоречие лучше не разрешать задним числом, а не допускать молчаливой перезаписи» (гл. 12).

18.4.2. Устаревшие предложения: коммиттер валидирует, а не просто пишет

Снапшот-чтения убирают гонку на *чтении*, но остаётся тонкий изъян на *записи*. Обходчик мог посчитать предложение на версии графа `snapshot=123`, а к моменту коммита проекция уже на `129`, где часть посылок успела устареть: узел-родитель ретрактнут, слит, понижен в уверенности, изменил `layer` или `visibility`. Если коммиттер применит предложение вслепую, гонку по данным мы убрали, но получили *семантически устаревший коммит* — что не лучше.

Поэтому предложение — не голая операция, а операция с *контекстом валидности*: {`op`, `members`, `read_snapshot_id`, `parents`, `proposal_ttl`, `validity_conditions`}. Коммиттер перед применением *валидирует*, а не сразу пишет: живы ли ещё родители? не deprecated? не изменились ли их `confidence`/`layer`/`visibility` с момента `read_snapshot_id`? не вышел ли `proposal_ttl`? По итогу — одно из четырёх: *применить* (условия держатся), *rebase* (пересчитать на текущей версии, если посылки лишь сдвинулись), *downgrade* (принять как более слабую гипотезу) или *reject* (посылки развалились). Это оптимистический контроль конкуренции: не блокируем заранее, а проверяем на коммите. Без этого read/write-гонку уберёшь технически, а память тихо наполнится решениями, принятыми по уже неверным предпосылкам.

18.4.3. «Забудь это»: удаление при неизменяемом логе

Неизменяемый лог порождает очевидный конфликт: а что происходит, когда пользователь говорит «Рин, забудь последние 10 минут»? Для локального компаньона это не юридическая занудность, а конкретный будущий UX, и он должен что-то значить.

Ключ к разрешению: *неизменяемость* $\neq$ *«нельзя удалять»*. Она означает *append-only, без тихих правок на месте*; само удаление — это *тоже appended, аудируемое событие*, а не нарушение дизайна. «Забыть» — первоклассная, поддержанная операция, и реализуется тремя механизмами, которые работают

вместе:

- **Граф (легко).** Убираем узел, ретрактим, перепропагируем. Каскад: удаление исходного наблюдения понижает/сносит производные узлы, чьим *провенансом* оно было (ретракция по TMS, глава 13). «Забудь X» — это ещё и «забудь то, что из X выведено».
- **Tombstone.** В лог дописывается delete-событие; при пересборке проекции удалённое пропускается. Эффект немедленный, но сырое событие ещё *в логе* (годится для undo/аудита, но это не настоящее стирание).
- **Crypto-shredding.** События (по сессии/скоупу) зашифрованы своим ключом; «удалить» = уничтожить ключ → шифртекст остаётся, но навсегда нечитаем. *Мгновенное, проверяемое* стирание без переписывания лога — идеально под «забудь последние 10 минут» (эти минуты под эфемерным сессионным ключом; шреднул ключ — стёр).
- **Compaction.** Фоновая пересборка лога выбрасывает tombstoned-события — сырое физически исчезает.

То есть «Рин, забудь X» = tombstone + каскадная ретракция производных + шреддинг ключа (истинное стирание) + компакция-уборка. Конкретно, а не «ну она как- то забудет».

Crypto-shred рождает key management и требует taint-propagation

Две дыры, без которых схема красива, но дырява. *Первая*: шреддинг создаёт новый центр тяжести — **управление ключами** (где ключи, кто их держит, гранулярность сессия/скоуп). *Вторая, важнее*: уничтожив ключ raw-события, система *не* стёрла след, если plaintext успел растечься — в debug-log, снапшот context-packer, embedding-кэш, вектор-индекс, бэкап, telemetry-сводку или производный fusion-узел. Поэтому рядом с «забудь» нужна дисциплина **taint-propagation**: всякий производный объект наследует taint/провенанс от породившего события, а delete-cascade чистит *все* производные хранилища — граф, индексы, сводки, эмбеддинги, снапшоты, pending-очереди, бэкапы. Иначе «забыл» будет только на бумаге.

Гранулярность забвения

На каком уровне держать ключи шифрования (событие / сессия / скоуп), что именно каскадит при удалении (только прямые производные или транзитивно), и как показывать пользователю, что забыто, — открыто. Но механика («забыть» = событие в логе + shred + каскад) зафиксирована.

18.5. Write-path: что попадает в память, а что нет

Отдельно проговорим, *что именно* записывается, потому что «не писать» — полноправный исход, а не сбой. Конвейер:

```
raw_event -> parse/extract candidates
          -> [ epistemic_gate, salience_gate, anti_fusion_gate ]
          -> write | skip
          -> (if write) propagate
          -> audit_log   # и write, и skip логируются
```

- **epistemic-гейт**: стоит ли этому верить и с какой уверенностью (гл. 13).
- **salience-гейт**: значимо ли это для Рин (appraisal, гл. 16).
- **anti-fusion-гейт**: см. ниже.

Принцип 18.1 (response-useful association $\neq$ memory-worthy fusion)**.** Оператор должен уметь не только «нашёл красивую связь $\rightarrow$ породил узел», но и явно *не писать*: «со-активация есть, связного вывода нет», «связь эстетическая, не эпистемическая», «это каламбур, не убеждение», «полезно для реплики, но не для памяти». Рин может *использовать* ассоциацию в ответе, не записывая её узлом — иначе память зарастает «интересными мыслями», которые были хороши в моменте, но не являются знанием.

И `skip` обязательно логируется (хотя бы в debug): иначе не поймёшь, почему Рин «не запомнила» важное.

18.5.1. Коммит не бинарен: полосы, а не «только ночью»

Соблазнительно сказать: «walker'ы читают весь день, а пишет коммиттер только ночью». Это *слишком сильно* и ломает сознательное мышление. Если так сделать буквально, Рин днём станет странной: ходит по графу, находит связи, что-то «понимает» — но не может ни закрепить свежий внешний факт, ни воспользоваться дневной догадкой иначе как эфемерным текстом. Правильный инвариант:

Принцип 18.2 (Коммит не бинарен)**.** Наблюдаемые события и рабочие гипотезы фиксируются *немедленно*; долговременная консолидация *отложена*. Коммиттер один и сериализован, но работает не «ночью против дня», а в нескольких *полосах* по стоимости и durability.

Полосы одного коммиттера:

- **immediate.** Пишется всегда и сразу: append в event-log (что произошло — это *журнал*, а не «убеждение»), прямые пользовательские наблюдения, task-critical внешние факты, явные исправления, важные boundary / preference. Откладывать это до ночи нельзя — потеряешь след происходящего.
- **foreground pending.** Дневная рабочая память — это наш *session-слой* (гл. 11), не новая сущность: временные узлы-гипотезы, найденные факты, цепочки рассуждения, кандидаты на консолидацию, `need_to_verify`. Живут с TTL, низкой уверенностью, провенансом и статусом `pending` — уже объект, которым можно *пользоваться сейчас*, но ещё не вечная память.
- **idle.** Микробатчи в простое: мелкий dedup, малые правки уверенности, локальные проверки противоречий.
- **sleep.** Тяжёлое: консолидация $A, B, C, D \rightarrow X$, массовое разрешение конфликтов, fusion далёких ассоциаций, pruning, reindex, пересборка summary, ревизия низкоуверенных потомков, promotion рекуррентных кандидатов.

Это и есть fast-path / slow-path: дешёвые $O(1)$-мутации (добавить узел-гипотезу, инкремент) идут днём по быстрой полосе; дорогие $O(|V| \log |V|)$ и глобальные — ночью. Критично: быстрая полоса — про *латентность*, а не про поблажку. Она проходит тот же write-path и те же гейты (epistemic / salience / anti-fusion, раздел 18.5), просто синхронно.

Сводно, по трём процессам (и подчеркнём: строка записи *не* ночецентрична — днём работает immediate-lane):

Процесс	**Днём ($\rho \approx 1$)**	**Фон / ночь ($\rho \approx 0$)**
Retrieval	локальный разреженный matvec-диффузия из HNSW-якорей (fast-path)	телепорт-скраб, эргодическое покрытие, большой α
Fusion / мышление	«мотыльки»: одиночные RW на остывающем $a_{\text{work}} \to$ кандидаты в thought-inbox	честный рой (slow-path): треки, n-арные цепочки, далёкие мосты
Запись	**immediate-lane**: append в event-log, безопасные foreground-микрокоммиты (`observed`, `boundary`, правки `preference`); `pending` догадки — в thought-inbox	коммиттер (slow-path): судит гипотезы, тяжёлая консолидация, prune, reindex

Так снимаются оба дневных противоречия. *Сознательный поиск* не ждёт ночи: Рин «гуглит здесь и сейчас», кладёт результат в event-log и session-граф как sourced observation, использует в ответе — но со статусом «свежий источник, не консолидировано», а не «я давно это знаю». *Догадки* тоже случаются днём — просто живут как активная гипотеза, а взрослеют в durable-память позже.

18.5.2. Thought-inbox: как дневная догадка попадает в реплику, не став фактом

Фоновый разведчик работает *всегда* — даже в напряжённом диалоге, на остаточном бюджете $B_{\text{bg}} = B_{\text{total}}(1 - \rho)$ (гл. 6), который никогда не ноль. Поэтому «момент озарения» (связь House-Уилсон и Cast Away-Уилсон) может родиться и днём. Но показывать Рин *сырой след* walker'ов нельзя — это шум: сотни посещённых узлов, случайные касания, мусорные мосты. Между следом и репликой стоит фильтр — *thought-inbox*: туда попадают не все касания, а только отобранные предложения-инсайты (сильная, но неочевидная со-активация далёких узлов):

```
type: insight_candidate
content: Уилсон" у House и Уилсон в Cast Away могут делить нарративную функцию"
status: unverified_hypothesis
salience: 0.72
provenance: [node_a, node_b, path_summary]
source: background_walker
ttl: 2h
recommended_action: mention_as_hypothesis | verify_later | ignore
```

Context-packer иногда подмешивает самые «горячие» из них в промпт — но *с явной эпистемической меткой* (это ровно наш packer-слой меток, раздел 18.7): «*возможная мысль, не факт: между A и B может быть связь; статус — не проверено; использовать как догадку, не как установленную память*». Дальше цикл замыкается сам: если контекст диалога позволяет, Рин озвучивает догадку — и в *этот* момент она легализуется, падает в event-log и становится узлом со статусом `need_to_verify`; если не пригодилась — тихо затухает по TTL и убирается сборщиком. Изоляция ролей сохранена: walker *читает и предлагает*, LLM *осмысляет и озвучивает*, коммиттер *пишет*.

Не писать активацию на каждый шаг walker'а

«Walker дёшев» — верно относительно LLM, но не бесплатно, и узкое место обычно не в математике, а в обвязке (индексы, сериализация, локи, provenance-traversal, scope-фильтры, гл. 22). Поэтому обходчик *не* пишет активацию в хранилище на каждом шаге: он копит visits / деньты активации локально в памяти процесса и *батчем* сбрасывает в transient-activation store (по-прежнему max, не сумма). И на сбросе различает `access_reason`: `user_recall` укрепляет след B_{base}, а `scrub_visit` — нет.

18.6. Таксономия ретрива: не всё — блуждание

Walker как «почти всё» — переобобщение. Не всякая операция памяти естественно является ассоциативным блужданием:

- **associative** (якоря + walkers) — для живости и ассоциаций (гл. 6).
- **forensic** — точный поиск по метаданным/провенансу/времени: «покажи всё про X», «узлы с источником Y», «что противоречит этому факту», «последняя версия prompt-модуля». Это индексы и query-planner, *не* блуждание.
- **creative** — walkers + fusion + сон (гл. 12).

Forensic-ретрив — не приятное дополнение, а *необходимость* для обслуживания: аудит, миграции, точное удаление конкретных узлов. Walker полноты не гарантирует, а скучный индекс — гарантирует. Для живости Рин нужны walkers; для корректности памяти нужны скучные индексы.

18.7. Context-packer: как граф превращается в промпт

Это интерфейс графа и LLM на инференсе — что именно *видит* модель, и **последний эпистемический гейт перед речью**. Если packer отдаёт модели плоский список содержаний без статусов, вся дисциплина графа (уверенность, гипотезы, депрецирование) теряется на последнем метре. Поэтому в контекст попадают *не только contents, но и метки*: `observed`, `hypothesis`, `deprecated`, `contested`, `low_conf`, `time_validity`, и при необходимости `do_not_state_as_fact`. Плюс провенанс для важных утверждений и mood/persona-обусловливание. Тогда модель знает, что подать как факт, что — как «похоже», а что не выдавать за истину вовсе. Держать компактно: короткий контекст = маленький KV-кэш = меньше VRAM (раздел 18.8).

Это `prompt context` — *временная сборка* (граница из главы 3), а не хранилище: спаковали, отдали модели, выбросили.

Здесь же — естественное место для *eligibility-гейта*: перед упаковкой отсеять узлы, которые *есть в памяти, но не годятся для этого контекста* (например, не класть в промпт то, что не стоит показывать при данном собеседнике или выносить в публичный контент). Это шире удаления и *у́же* тяжёлой governance-бюрократии: не «кто владеет и на каких условиях», а простой guardrail *retrieval* → *eligibility-гейт* → *packer* → *output*. Тяжёлую систему доступа/согласия мы сознательно не строим (вектор проекта сместился), но сам крючок стоит держать в виду.

Eligibility как отложенный guardrail

Полноценный слой видимости/раскрытия (что можно показать здесь, класть в промпт, использовать в публичном контенте, обобщать) — отложен. Пока — минимальный гейт «память может существовать, но быть негодной для данного контекста», без бюрократии. Разворачивать ли его — решается, когда вернётся потребность в family/публичном режиме.

Формат упаковки

Как именно показывать модели статус узла (уверенность, гипотеза vs факт, провенанс), сколько узлов включать и как их ранжировать под текущий запрос — открыто, меряется на стенде.

18.8. Оценки стоимости: память, время, узкие места

Оценки порядковые (для локального железа класса RTX 4090 / 8–24 ГБ VRAM), но они сразу показывают, где узкие места, а где нет.

18.8.1. Стоимость операций

Операция	Порядок	Комментарий
LLM-инференс (композиция, объяснение)	**0,5–5 с**	**узкое место**; ∼2 с/запрос на 7–8B локально
векторный поиск якорей (HNSW, 10^6)	∼мс	не узкое место
обход/spreading (сотни-тысячи узлов)	мс–10 мс	C-ядро igraph, дёшево
пропагация уверенности (лавина)	∼мс	дёшево
детекторы телеметрии	O(1) инкрем.	дёшево
распад/ACT-R	дёшево *лениво*	полный проход по N узлам — дорого; делать лениво (при доступе) или во сне, не на каждый тик
переваривание учебника	**минуты-часы**	одноразово; дорого из-за LLM-извлечения фактов; только в Idle/Sleeping

18.8.2. Латентность ответа

Критический путь: `user_msg` → поиск якорей (мс) → обход (мс) → композиция ответа (**LLM, ∼1–3 с**) → ответ. **Бутылочное горло — LLM-вызов**; всё графовое к латентности почти не добавляет. Цель — держать ответ в ∼1–3 с, то есть минимизировать *число* LLM-вызовов на переднем плане (в идеале один: граф уже подал контекст, модель только оформляет).

18.8.3. Память

- **VRAM** — модель (фиксировано, узкое место по размеру модели: 7–8B в квантизации ~5–8 ГБ). Отсюда весь смысл «факты в граф, а не в веса».
- **RAM** — векторный индекс горячего множества; для 10^6 эмбеддингов плоско ~1,5 ГБ, с HNSW/квантизацией меньше; *тиринг* держит это ограниченным.
- **Диск** — сам граф: ~2–3 КБ/узел, 10^5–10^6 узлов = 0,3–3 ГБ (глава 22 для оценки роста). Дёшево; учебник $\approx 10^4$ узлов $\approx$ 30 МБ.

18.8.4. Где узкие места, а где нет

Принцип 18.3 (Всё сводится к LLM-слотам)**.** Дорог только GPU/LLM — и по латентности ответа, и по фоновой пропускной способности. Всё графовое (хранилище, обход, статистика, распад, векторный поиск) — дёшево (мс) и узким местом *не* является. Поэтому вся инженерная задача рантайма сводится к одному: *как распределить дефицитные LLM-вызовы* между быстрым ответом (передний план) и фоном (fusion/метакогниция/переваривание). Это и делает планировщик (раздел 18.1).

Цели по латентности и throughput

Конкретные цели (ответ $<$ N с; сколько фоновых fusion в минуту в простое; когда переключать модель на меньшую ради скорости) — под стенд и под конкретное железо. Динамический роутинг «мелкая модель на рутину, крупная на метакогницию» (как у Toy и др.) — очевидный кандидат оптимизации.

19. Эмбриональный граф: с чего начинается память

Пустой граф памяти бесполезен: у существа не бывает мышления «с нуля» — есть врождённый здравый смысл, на который ложится личный опыт. Поэтому память Рин начинается с *эмбрионального графа* — нейтрального по характеру каркаса из готовых баз здравого смысла, который затем обрастает личным опытом через fusion и консолидацию. Эта глава отвечает предметно: из чего он собран, как выглядят его узлы, как их сшить и как подключить эмбеддинги.

19.1. Две базы, две разные роли

Нужны две дополняющие базы (Wikidata как содержание не берём — человек не помнит все столицы; глава 5):

- **ConceptNet** — *предметный и таксономический* здравый смысл: что чем является, из чего состоит, для чего используется. Даёт Рин «физику мира».
- **ATOMIC** — *причинно-социальный* здравый смысл о событиях и ментальных состояниях: чего человек хотел, что почувствовал, что за этим следует. Даёт Рин «психологию».

Порознь одного мало: ConceptNet без ATOMIC — мир без психологии; ATOMIC без ConceptNet — реакции без предметного мира.

19.2. Как выглядят узлы и рёбра ConceptNet

ConceptNet — это уже граф. Узел — концепт (слово или короткая фраза), заданный URI; ребро — типизированная взвешенная ассоциация. Схематически запись:

```
узлы
:   /c/ru/кружка/        /c/ru/посуда/      /c/ru/пить/      /c/ru/горячий/рёбра
 (concept, relation, concept, weight):
   (/c/ru/кружка/, IsA,       /c/ru/посуда/,  2.5)
   (/c/ru/кружка/, UsedFor,   /c/ru/пить/,    1.8)
   (/c/ru/кружка/, HasProperty,/c/ru/горячий/,1.2)
```

Типов отношений около трёх-четырёх десятков (IsA, PartOf, UsedFor, HasProperty, RelatedTo, Causes, MotivatedByGoal, …), и рёбра многоязычны (связывают понятия между языками). **Важно для нас**: вес ребра ≈ мягкая уверенность из коробки, а типы отношений прямо ложатся на нашу типизацию рёбер (глава 5).

19.3. Как выглядят узлы ATOMIC

ATOMIC устроен иначе, и это источник главной сложности сшивки. Его узлы — не концепты, а *события-фразы* с плейсхолдерами действующих лиц (PersonX, PersonY). От события идут рёбра фиксированных *социально-причинных* типов. Схематически:

```
событие
: "PersonX breaks a cup"
   xIntent  -> (none / accident)     # чего PersonX хотел
   xReact   -> (sad, upset)          # что PersonX почувствовал
   xEffect  -> (cleans it up, buys a new one)   # что произошло с PersonX
   xNeed    -> (to be holding a cup) # что требовалось до
   oReact   -> (concerned)           # что почувствовали окружающие
```

То есть «кружки» как отдельного узла в ATOMIC **нет** — есть событие «разбить кружку» и его социально-причинные следствия. ATOMIC10x — та же структура, но в несколько миллионов примеров, порождённых языковой моделью и отфильтрованных критиком (больше покрытия, больше шума).

19.4. Сшивка: главная инженерная работа

Соблазн сказать «форматы отличаются только на диске» — недооценка. Внутри графовой базы это одна структура (узлы-токены, направленные типизированные рёбра), и обход от типа ребра не меняется. Но чтобы Рин, услышав «разбил кружку», прошла от *события* ATOMIC к *предметным* свойствам кружки из ConceptNet, события и концепты надо *связать*. Это entity/event linking — нетривиальный шаг.

19.4.1. Мост через контент-слова и Numberbatch

Алгоритм сшивки: из события-фразы ATOMIC извлекаем контентные слова (существительные, глаголы) и линкуем их к концепт-узлам ConceptNet — сначала точным лемма-матчем, затем эмбеддинг-фолбэком.

Не bag-of-words и не one-hot

Грубые представления (мешок слов, one-hot) не знают синонимии: «mug» и «cup», «разбить» и «сломать» окажутся разными токенами, и мост выйдет дырявым. Нужен эмбеддинг, знающий смысловую близость.

ConceptNet Numberbatch — родной эмбеддинг самого ConceptNet: каждому концепт-узлу сопоставлен вектор (~300 измерений), причём смыслово близкие концепты близки в пространстве, а языки согласованы. Это идеальный мост:

```
def link_atomic_event_to_concepts(event_phrase, conceptnet, numberbatch):
    concepts = []
    for token in content_words(event_phrase):     # "break", "cup"
        node = conceptnet.lemma_match(token)       # 1) точный матч
        if node is None:                           # 2) эмбеддингфолбэк-
            node = numberbatch.nearest(embed(token))
        concepts.append(node)
    return concepts     # событие ATOMIC <-> концепты ConceptNet
```

Тот же Numberbatch решает ещё две задачи (одно пространство — три применения): член релевантности $\mathrm{rel}(B \mid q)$ в правиле обхода (глава 6) и сидирование валентности узлов (регрессия аффект-лексикона в это же пространство).

19.4.2. Заземление действующих лиц

Плейсхолдеры PersonX/PersonY — не абстракции, а слоты. В рантайме они *заземляются* на конкретных людей: PersonX $\mapsto$ зритель Z, PersonY $\mapsto$ второй пользователь проекта. Так общий социальный шаблон превращается в запись о конкретном отношении.

«Разбил кружку» на эмбриональном графе Рин

Зритель пишет «разбил любимую кружку». (1) Событие матчится к ATOMIC-шаблону «PersonX breaks a cup», PersonX $\mapsto Z$. (2) Через ATOMIC всплывают социально-причинные узлы: досада (Z), сочувствие (окружающие), «нужна новая». (3) Через сшивку с ConceptNet всплывают предметные свойства: кружка — посуда, из неё пьют, бывает любимой. (4) Если в личной памяти Рин есть узел «Z давно хотел новую кружку» ($-$ contradicts «досада»), появляется основание для уместного «поздравляю» вместо сочувствия. Ни одна база в одиночку этого не даёт: ATOMIC — эмоцию, ConceptNet — предмет, личная память — контекст.

19.5. Как строить эмбрион: засеянный подграф, а не слепой импорт

Здесь важная инженерная развилка. *Слепо залить* весь ConceptNet и весь ATOMIC в память — плохо: получишь много шума (банальные и устаревшие связи, англоцентризм, странности social commonsense, слабый перенос на русский, низкую релевантность именно домашней Рин). Тогда отлаживаться будет не память Рин, а шумы чужой базы здравого смысла. Но и другая крайность — *писать граф руками* на 500 узлов — тоже плоха: медленно, и такой граф слишком разрежён, чтобы на нём проявились топологические эффекты (хабы, боттлнеки, over-squashing), которые как раз надо тестировать (глава 8).

Правильный — **серединный путь**: извлечь *небольшой засеянный проверяемый подграф*.

1. **Сид** — концепты реального мира Рин: дом, код, усталость, кошка, универ, дота, RIN, бытовые действия, режимы общения.
2. **Расширение** — ego-граф на k хопов от сида по ConceptNet/ATOMIC.
3. **Фильтр по весу ребра** — отсекает банальщину и слабые ассоциации (вес ConceptNet $\approx$ мягкая уверенность из коробки).
4. **Ручная чистка глазами** — один раз, для пары тысяч узлов дёшево.
5. **Статус** — это *источник кандидатов*, а не замороженная истина: дальше Рин его сама растит и прунит через использование.

Так получаем реальную структуру (для топологических тестов) + реальные ассоциации (не рукописные) + проверяемость глазами + релевантность (засеяно на её домены), и обходим шум слепого импорта.

Русский ConceptNet тоньше английского

Практический кавеат: русскоязычная часть ConceptNet заметно беднее английской. Варианты: держать *концептный backbone* в англопространстве (там связей больше), а русскую поверхность накручивать сверху; либо принять более разрежённый русский и дополнять его вручную/из других источников. Что даст лучшее качество на домашних сценариях Рин — открыто, проверяется на стенде (глава 22).

19.6. Как эмбриональный граф превращается в память Рин

Эмбриональный граф *нейтрален по характеру*: это «то, что досталось от предков-разработчиков», усреднённый здравый смысл. Личность и личный опыт входят не переписыванием этих узлов, а *надстройкой*: каждое общение порождает через fusion (глава 10) новые узлы и рёбра с временными метками, а консолидация закрепляет устойчивое. Со временем поверх нейтрального каркаса вырастает индивидуальная, окрашенная память — но фундамент здравого смысла остаётся как *система координат*, относительно которой отклонения читаются как характер (глава 16).

Эмбрион, а не энциклопедия

Название «эмбриональный» выбрано не случайно. Это не готовое знание о мире, которое Рин «зачитывает», а зачаток, минимальный каркас связей, на котором *развивается* собственная память. Врождённые узлы со временем могут и потускнеть, и быть пересмотрены (континуум глубины консолидации, глава 5) — просто медленнее приобретённых.

20. Первый рабочий срез: Stage 1

20.1. Зачем отдельная глава про Stage 1

У больших архитектурных текстов есть повторяющаяся болезнь: чем лучше описана система в теории, тем легче откладывать момент, когда она должна столкнуться с реальной инженерной средой. Возникает соблазн дописать ещё одну красивую главу, ещё одно уточнение к консолидации или ещё один будущий модуль, прежде чем строить хоть что-то наблюдаемое.

Stage 1 нужен как лекарство именно от этого. Это не “урезанная версия мечты”, а **минимальный рабочий срез**, на котором уже видна физика системы:

- как событие входит в память;
- как оно превращается в typed node;
- как решается вопрос commit/skip;
- как устроено извлечение;
- как собирается контекст для LLM;
- как вся эта внутренняя механика наблюдается и проверяется извне.

20.2. Чего Stage 1 должен добиться

На первой стадии не требуется полноценная “живая память Рин” во всём богатстве книги. Требуется более скромная, но более важная вещь: **сделать внутреннюю механику памяти наблюдаемой и проверяемой**.

Принцип 20.1 (Первый успех формулируется инженерно)**.** Критерий успеха Stage 1 должен звучать не как “Рин выглядит умной”, а как “можно руками проследить путь от события до узла памяти, от узла до retrieval и от retrieval до контекста модели”. Пока эта прозрачность не достигнута, разговор об “умном поведении” почти всегда будет смешивать качество памяти с шумом LLM.

20.3. Сквозной pipeline v0

Минимальный рабочий срез удобно описывать как цепочку:

```
event_log            # append observed/committed/deleted
  -> candidate extraction на( Stage 1 может быть ручной)
  -> typed nodes     # node_kind обязателен
  -> write/skip gate # epistemic + salience + anti-fusion
  -> forensic index  # метаданные / provenance / время
  -> anchor retrieval
  -> local graph expansion
```

```
  -> context-packer v0 # observed / hypothesis / low_conf / ...
  -> inspector CLI
  -> synthetic tests
```

В этой цепочке важно не количество компонентов, а то, что каждый из них имеет *наблюдаемый вход* и *наблюдаемый выход*. Если какой-то шаг “случается внутри магии”, Stage 1 свою задачу не выполняет.

20.4. Что должно быть реализовано в первом срезе

1. Event log

Нужен неизменяемый журнал событий: что пришло на вход, когда это произошло, откуда получено и во что в итоге было превращено. Даже если candidate позже будет отклонён, след о самом факте рассмотрения должен сохраняться. В терминах контракта Stage 1 это append-only лог с событиями вроде `observed`, `committed` и `deleted`; именно он делает возможным replay и event sourcing.

2. Typed node store

Нужен минимальный, но уже честный store узлов и рёбер. Узел Stage 1 не обязан содержать все будущие поля книги, но обязан нести базовую схему:

- `id`;
- `content`;
- `embedding`;
- `node_kind`;
- `confidence`;
- `provenance` (или support-skeleton);
- `time`;
- `layer`.

Остальные поля вроде `valence`, `importance`, `domains` или `need_to_*` могут достраиваться позже, но `node_kind` обязателен с самого начала: именно он задаёт смысл `confidence`.

Чего нельзя откладывать

Поле `node_kind` нельзя считать “будущим украшением”. Без него невозможно даже корректно интерпретировать `confidence`: степень доверия к факту, устойчивость предпочтения и надёжность реактивного паттерна — разные вещи.

3. Write/skip gate

Нужен gate, который умеет отвечать на вопрос: *следует ли вообще писать это в память?* Даже в простейшем виде он должен различать:

- точный дубль;
- незначимый candidate;
- чрезмерно разросшийся кандидат;
- допустимый commit.

Особенно важно, чтобы Stage 1 логировал не только коммиты, но и **причины пропуска**. В инженерной разбивке такой gate уже на первом срезе полезно мыслить как тройку:

- **epistemic-gate**: стоит ли этому верить и с какой уверенностью;
- **salience-gate**: значимо ли это для дальнейшей жизни памяти;
- **anti-fusion-gate**: не пытается ли один candidate притащить слишком много структуры разом.

Без видимых причин skip память слишком быстро превращается в чёрный ящик.

4. Forensic index

Нужен **forensic index**: точный индекс по метаданным, provenance, времени и служебным полям. Он нужен не для живости retrieval, а для диагностики, точного поиска, удаления и реплея. Если у памяти нет такого слоя, она быстро становится удобной для ассоциаций, но неудобной для инженерного контроля. На первом срезе сюда же относится и работающий delete-cascade хотя бы в минимально проверяемой форме.

5. Anchor retrieval

Нужен первый работающий retrieval, пусть даже без всего богатства книги. В минимуме это означает:

- векторный поиск якорей;
- ограниченное расширение окрестности на графе;
- внятный и воспроизводимый результат.

Именно на Stage 1 впервые становится видно, что память — это не “дополнительный prompt”, а отдельная поисковая и структурная машина.

6. Context-packer

Нужен packer, который собирает память в форму, пригодную для подачи в модель. Это ещё не финальный формат выражения, но уже важный тест: система должна уметь не просто хранить узлы, а превращать их в рабочий контекст с эпистемическими пометками. На первом срезе такие пометки должны быть хотя бы базовыми: `observed`, `hypothesis`, `low_conf` и родственными им.

7. Inspector

Нужен **inspector** памяти: инструмент, позволяющий видеть узлы, рёбра, поля, provenance, gate decisions и результаты retrieval. Без него Stage 1 будет трудно не только улучшать, но даже честно диагностировать.

8. Synthetic tests

Нужен стенд на маленьких контролируемых графах. На этой стадии особенно важно тестировать не “правдоподобие диалога”, а локальные свойства архитектуры:

- правильно ли работает запись;
- не теряются ли provenance и типы;
- даёт ли retrieval осмысленную окрестность;
- воспроизводим ли replay из event log;
- видны ли базовые failure modes.

20.5. Что сознательно откладывается

Stage 1 силён именно тем, что он ограничен. В эту стадию сознательно не стоит пытаться втиснуть всё, что описано в книге. Иначе она перестанет быть первым срезом и превратится в бесконечную “почти готовую систему”.

На раннем этапе допустимо отложить:

- полную динамику настроения;
- сложную personality-driven маршрутизацию;
- тяжёлый fusion в полном виде, оставляя на ранней стадии только anti-fusion-скелет и провизорные кандидаты;
- полную TMS-ретракцию;
- продвинутый сон и скраб;
- гиперграфы, клеточные комплексы и другие дальние ветки;
- тонкую governance-политику удаления и криптостирания.
- мета-обучение операторов.

Смысл ограничений

Отложенные механизмы здесь не “отменяются”, а *переносятся до тех пор, пока нижний срез не станет доверенным*. Иначе система получает богатую теорию поверх недоверенного основания.

20.6. Три инженерных упрощения первой версии

Даже внутри Stage 1 полезно заранее зафиксировать несколько упрощений, чтобы не симулировать преждевременную полноту.

Tombstone вместо полного забывания

На первом проходе достаточно tombstone-подхода: узел помечается как удалённый и исчезает из рабочей проекции, но система ещё не обязана реализовывать полную каскадную очистку всех производных следов.

Компоненты связности вместо полной консолидации сообществ

На первом проходе достаточно более грубой схемы предложений к консолидации: компоненты связности и жёсткие пороги пересечения вместо полного community detection и дорогого graph coarsening. На раннем этапе важнее добиться корректности typed storage и retrieval, чем сделать избыточно умное слияние. В более инженерной формулировке: вместо тяжёлого со-консолидационного графа и поиска модулярности достаточно простых *connected components* по графу предложений с отсечением по жёсткому порогу пересечения, например в духе LSH/MinHash.

Простой семантический вентиль

Stage 1 может жить с довольно грубым семантическим гейтом, если это честно осознано и изолировано. Важна не максимальная красота фильтра, а воспроизводимость его ошибок и возможность их позже улучшать. На уровне roadmap это формулируется ещё конкретнее: первым проходом допустим просто косинус кэш-эмбеддингов. Более сильная защита от омонимии через отдельный sense/type-gate требует sense-слоя, которого на Stage 1 ещё нет, поэтому здесь сознательно не обещается полная защита от омонимии.

20.7. Критерии готовности

Stage 1 можно считать состоявшимся, если выполнены следующие условия.

1. Событие действительно проходит путь “лог → candidate → gate → commit/skip”, и этот путь можно наблюдать.
2. Типы узлов и provenance сохраняются и доступны inspector’у.
3. `tombstone`-удаление сносит узел из рабочей проекции, и это проверяемо inspector’ом.
4. Retrieval отрабатывает на синтетическом графе и возвращает содержательно осмысленную окрестность.
5. Replay event log воспроизводит ту же графовую проекцию.
6. Хотя бы один базовый failure mode из батареи отказов — например, концентрация π на хабах или точное удаление — обнаруживается и воспроизводимо чинится на стенде.

Как должен выглядеть типичный успех Stage 1

На вход подаётся контролируемое событие. Затем в inspector’е видно: сам event, candidate, причину commit или skip, созданный узел, связанные с ним рёбра, найденные retrieval-якоря, собранный packer’ом контекст и, если нужно, support-skeleton. После этого тот же сценарий прогоняется

на synthetic graph и даёт воспроизводимый результат. Именно такая наблюдаемость и есть настоящее достижение первой стадии.

Принцип 20.2 (Первый успех — не «Рин помнит», а «физика графа наблюдаема»)**.** Первым продуктом этой стадии должна быть не “память v0” в рекламном смысле, а *memory inspector v0*. Если первым мерилом сделать “Рин умно пользуется памятью в диалоге”, система почти сразу утонет в шуме языковой модели. Поэтому успех Stage 1 состоит в том, что можно руками создать событие, увидеть candidate, увидеть commit или skip, увидеть узел, увидеть retrieval, увидеть provenance или support-skeleton и затем воспроизвести тот же путь на synthetic graph.

20.8. Роль Stage 1 в архитектуре всей книги

Stage 1 не является “маленькой технической вставкой” в конце курса. Он играет методологическую роль для всей архитектуры памяти.

- Он заставляет различать красивые понятия и реально наблюдаемые механизмы.
- Он принуждает к дисциплине типов, provenance и gate decisions.
- Он отделяет дефекты памяти от дефектов языковой модели.
- Он создаёт первую твёрдую площадку, на которой дальнейшие главы можно не только обсуждать, но и проверять.

Поэтому с точки зрения разработки Stage 1 — не скромное начало, а первый момент, когда архитектура памяти вообще начинает иметь право на дальнейшее усложнение.

Принцип 20.3 (Дальше — строить и ломать, а не расширять)**.** После фиксации этого среза следующий лучший ход — не писать ещё одну красивую главу, а собирать систему, прогонять synthetic tests и позволять стенду править гипотезы. Каждый следующий красивый модуль имеет право войти в курс только тогда, когда нижний срез уже выдерживает нагрузку.

21. Сквозные разборы: как память работает на цельном сценарии

21.1. Зачем нужны сквозные разборы

Когда архитектура описана по главам, возникает полезная, но опасная иллюзия: будто субстрат, retrieval, настроение, эпистемика, рантайм и ревизия существуют по отдельности. В реальном ходе работы памяти они вступают в действие одновременно. Поэтому здесь полезно пройти несколько сценариев от начала до конца.

21.2. Сценарий 1: обещание как факт, обязательство и будущий триггер

Событие

В разговоре появляется фраза: «нужно завтра отправить Маше черновик».

Что возникает в памяти

1. observation-узел о произнесённом обязательстве;
2. сущность “Маша” либо связывается с существующим узлом, либо создаётся;
3. task-like узел “отправить черновик”;
4. темпоральная привязка “завтра”;
5. отношение типа `commitment_to` и/или `deadline`.

Почему это не просто эпизод

Если система запишет только буквальный эпизод разговора, она будет помнить, что что-то когда-то обсуждалось, но не сможет вовремя действовать. Если же она сразу запишет только сухую задачу, потеряется provenance и будет трудно понять, откуда обязательство взялось и можно ли его позже снять.

Принцип 21.1 (У обязательства минимум два лица)**.** Для памяти обязательство должно существовать и как событие происхождения, и как операциональный объект будущего действия.

Какие слои включаются

- write-gate пропускает запись как task-critical;
- runtime кладёт краткоживущий foreground-след;
- timescale-слой создаёт будущий временной триггер;
- retrieval later поднимет задачу не потому, что она семантически похожа на текущий диалог, а потому что наступает её окно активации.

21.3. Сценарий 2: ложная гипотеза и каскад ревизии

Событие

Рин несколько раз видит человека Z рядом с университетом и рано заключает: “Z учится здесь”.

Как рождается ошибка

1. эпизоды наблюдений проходят write-gate;
2. внутренний walkthrough находит паттерн совместной встречаемости;
3. fusion предлагает гипотезу “Z связан с университетом”;
4. на слабом материале коммиттер принимает её как *гипотезу с умеренной уверенностью*, а не как факт.

Почему это хороший, а не плохой путь

Память, которая никогда не рождает таких гипотез, стерильна. Ошибка начинается не в самом рождении гипотезы, а в преждевременном коллапсе её статуса в “установленный факт”.

Контрсвидетельство

Позже приходит наблюдение: Z не студент, а курьер, часто бывающий в этом районе.

Что должно произойти

1. прежняя гипотеза понижается;
2. потомки, опиравшиеся на неё, получают `need_to_revise`;
3. provenance сохраняет след того, почему система раньше думала иначе;
4. если таких ошибок много, evidence-aware revision понижает доверие к данному типу “соседство $\rightarrow$ роль”-выводов.

21.4. Сценарий 3: настроение меняет маршрут, но не право на ложь

Событие

У Рин понижен фон настроения. Она вспоминает события последних дней и чаще тянется к неприятным эпизодам.

Нормальная часть этого эффекта

Mood-congruent recall действительно должен смещать walk. В противном случае система станет не похожей на живую память, а на бесплотный индекс.

Опасная часть

Если смещение не ограничено, retrieval начинает собирать контекст почти целиком из негативно валентных узлов. Тогда:

1. разговорные ответы становятся систематически мрачнее;
2. метакогниция ошибочно выводит “у меня всё ухудшается”;
3. возникает риск самосбывающейся гипотезы.

Как работает предохранитель

1. настроение смещает walk, но не отменяет epistemic gates;
2. self-facts остаются провизорными, пока не подтверждены независимыми узлами;
3. фоновые detector’ы оценивают автокорреляцию и глубину руминативной петли;
4. при превышении порога включается Reflecting.

21.5. Сценарий 4: точное удаление и судьба производных следов

Событие

Поступает команда забыть конкретный чувствительный фрагмент недавнего разговора.

Чего недостаточно

Недостаточно удалить сырой текст или зашифрованный raw-log. Если содержание уже успело породить узлы, summary, embeddings, pending-candidates и рабочие ассоциации, память останется загрязнённой производными следами.

Что должна сделать система

1. удалить или крипто-шреднуть исходный первичный носитель;
2. найти все производные артефакты по provenance и event lineage;
3. пересчитать affected context indices;
4. проверить, не остались ли осиротевшие потомки без объяснимой опоры.

Удаление без ревизии осиротевших потомков

Самая тихая ошибка — не то, что удалённый факт остался лежать в одном месте, а то, что его производные потомки продолжают жить как будто ничего не произошло. Тогда формально система “забыла”, а содержательно — нет.

21.6. Сценарий 5: от вопроса пользователя к ответу с provenance

Вопрос

“Почему система решила, что этому человеку важна музыка?”

Нормальный плохой ответ

Система выдаёт гладкую словесную рационализацию без доступа к реальному пути возникновения убеждения.

Нормальный хороший ответ

Система может сказать примерно следующее по структуре:

1. есть два наблюдения;
2. есть один эпизодический паттерн;
3. есть одна осторожная гипотеза;
4. уверенность умеренная;
5. вот что могло бы её поднять или опровергнуть.

Почему это важно

Такой ответ делает память не только продуктивной, но и проверяемой. Читатель видит не просто результат, а локальное доказательство, и тем самым память начинает быть ближе к научной записи, чем к черному ящику.

21.7. Вывод

Сквозные сценарии показывают общий закон курса: память хороша не тогда, когда у каждого её модуля красивое локальное описание, а тогда, когда между записью, обходом, ревизией, действием и объяснением нет немых провалов.

22. Эксперимент: как проверять архитектуру памяти

22.1. Что именно проверяется

Эксперимент в такой системе не должен отвечать на вопрос “похожа ли Рин на живого собеседника”. Это поздняя, интегральная и слишком мутная проверка. Сначала нужно проверять более узкие вещи:

1. правильно ли retrieval находит и соединяет материал;
2. не теряется ли provenance;
3. умеет ли память ревизоваться, а не только накапливаться;
4. не порождает ли топология систематические искажения.

22.2. Три яруса стенда

Ярус 1 — синтетическая топология. Графы с контролируемой структурой: SBM, BA, small-world, деревья, экспандерные варианты. Здесь проверяется не содержание, а физика потока.

Ярус 2 — семантический граф. Эмбриональный subgraph из ConceptNet, ATOMIC и позднейших доращиваний. Здесь уже проверяются fusion, bridge-узлы, влияние типов рёбер и базовая объяснимость.

Ярус 3 — поток жизни. Синтетически порождённые эпизоды, диалоги, обязательства, коррекции, ложные гипотезы и их последующие опровержения. Здесь проверяется уже полная петля: запись, retrieval, ответ, ревизия, сон.

22.3. Четыре типа ground truth

Чтобы тест не стал расплывчатым, нужно различать четыре разных ground truth.

1. **Топологический**: какие области графа должны быть связаны или труднодостижимы.
2. **Эпистемический**: какие утверждения истинны, ложны или должны остаться в зоне незнания.
3. **Происхождённческий**: откуда именно возник узел и какие посылки должны его поддерживать.
4. **Операциональный**: что система должна была сделать, например поднять дедлайн или снять обязательство.

22.4. Новая добавка: retrieval-аудит как кортеж

В расширенной версии вводится retrieval-аудит

$$\mathcal{R} = \big(R_a,\, R_b,\, R_c,\, R_p\big),$$

где:

$$\begin{aligned} R_a &= \text{anchor recall},\\ R_b &= \text{bridge rate},\\ R_c &= \text{contradiction hit rate},\\ R_p &= \text{provenance coverage}. \end{aligned}$$

У этой схемы есть практическое достоинство: она разрезает плохой ответ на четыре совершенно разных класса причин.

22.5. Батарея ключевых тестов

#	Гипотеза	Наблюдаемая метрика
1	хабы перетягивают внимание	распределение посещений vs степень, доля ответов через hub-shortcut
2	узкий мост душит контрсвидетельства	contradiction hit rate vs ширина межкластерного сечения
3	fusion переоценивает слабые со-активации	precision принятой гипотезы после отложенной ревизии
4	provenance деградирует под нагрузкой	доля узлов без support-skeleton после серии консолидаций
5	deletion неполно очищает след	число осиротевших потомков и residual retrieval hits
6	mood-lock искажает ответный контекст	сдвиг валентности retrieval-context относительно нейтральной базы

22.6. Отдельно про deletion tests

Тесты удаления в таких системах особенно важны, потому что они проверяют не декларацию, а дисциплину данных.

Минимальный честный deletion test

В стенд вводится чувствительный факт x. Он должен:

1. попасть в event-log;
2. породить как минимум один выводной или индексный след;
3. затем быть удалён по команде;
4. перестать появляться в retrieval;
5. перестать поддерживать потомков;
6. оставить audit-trace самого факта удаления.

Если хотя бы один из пунктов не выполняется, удаление нельзя считать реализованным.

22.7. Как измерять ревизуемость, а не только точность

Память хороша не только тогда, когда она попадает в истину с первой попытки. Она хороша тогда, когда умеет *дёшево возвращаться* из ошибки. Поэтому рядом с обычными precision/recall нужны динамические метрики:

1. средняя задержка до понижения ложного узла после прихода контрсвидетельства;
2. средняя глубина каскада ревизии;
3. доля потомков, переживших понижение родителя без альтернативной опоры;
4. число ложных повторных resurrect после удаления или ретракции.

22.8. Чего нельзя проверять в самом конце

Есть соблазн сначала построить всё, а потом задать большой разговорный сценарий и посмотреть, «ведёт ли себя память разумно». Это полезно как финальная демонстрация, но почти бесполезно как инструмент разработки. Если ответ не удался, непонятно, сломался ли якорь, топология, fusion, контекст-пакер или голос.

Принцип 22.1 (Большой сценарий завершает, а не заменяет аналитику)**.** Интегральный end-to-end тест нужен, но его ценность появляется только после того, как у системы есть локальная измеримость отдельных механизмов.

22.9. Что должно выйти из хорошего стенда

Хороший экспериментальный контур даёт не только зелёные/красные лампы, но и *карту слабых мест*. После серии прогонов разработчик должен уметь сказать:

1. память теряет якоря, но хорошо держит bridges;
2. или память находит материал, но плохо объясняет происхождение;
3. или память богата на гипотезы, но слишком медленно их ревизует;
4. или память чисто эпистемически здорова, но топологически застревает в хабах.

Только тогда стенд действительно служит архитектуре, а не просто производит числа.

23. Failure Modes: как память ломается

23.1. Зачем нужен отдельный каталог поломок

У сложной памяти не бывает одного режима отказа. Она может ошибаться в содержании, в топологии, в темпоральной динамике, в provenance, в контекстной упаковке и в модальности ответа. Если всё это смешать, любая ошибка выглядит одинаково: “система ответила странно”. Отдельный каталог нужен, чтобы странность стала диагностируемой.

23.2. Топологические поломки

Hub collapse

Слишком высокая концентрация π на хабах приводит к тому, что память отвечает через общие, часто посещаемые узлы, а не через редкие релевантные.

Симптомы: однообразные мосты, частое использование общих концептов, потеря частного контекста.

Наблюдаемые переменные: корреляция между частотой включения узла в ответ и его степенью; распределение visits.

Предохранители: штраф за сверхцентральные промежуточные узлы, рестарт на якорях, отдельный glue budget.

Bottleneck starvation

Нужные контрсвидетельства сидят за узким сечением графа и не попадают в retrieval.

Симптомы: система упорно повторяет прежнюю картину мира, хотя в графе уже есть коррекция.

Предохранители: телепорт-скраб, явные bridge-edge policies, targeted forensic retrieval.

23.3. Эпистемические поломки

Premature certainty

Гипотеза слишком рано превращается в “почти факт”.

Симптомы: мало узлов в зоне незнания, низкая доля осторожных формулировок, частая болезненная ретракция.

Предохранители: priors по типам выводов, penalty за зависимые источники, запрет на $|\mathrm{conf}| = 1$ у выводных узлов.

Provenance erosion

После нескольких циклов консолидации память ещё хранит утверждения, но уже не может объяснить, откуда они взялись.

Симптомы: ответы звучат уверенно, но support-skeleton отсутствует.

Предохранители: отдельная метрика provenance coverage, обязательная передача support-skeleton при prune и summarize.

23.4. Динамические поломки

Mood lock-in

Настроение слишком сильно переопределяет маршруты retrieval, и память начинает самоусиливать текущее состояние.

Симптомы: контекст ответа систематически смещён в одну валентность, возникают руминационные петли.

Предохранители: saturation на mood-congruence, аварийный reflective mode, ограничение числа self-facts, требование независимого подтверждения.

Over-consolidation

Память слишком активно сворачивает частные эпизоды в обобщения и теряет материал для будущей ревизии.

Симптомы: красивые summaries, но пустые forensic trails; исчезновение конкретных различающих эпизодов.

Предохранители: хранение support-skeleton, delayed prune, тесты на ability-to-disconfirm.

23.5. Инженерные поломки

Context bloat

Retrieval находит слишком много всего, а context packer не умеет экономить.

Симптомы: длинный, тяжёлый, но рыхлый контекст; LLM тратит слот на чтение мусора вместо решения.

Предохранители: dual budget packer, quota на provenance, лимиты по типам узлов.

Pending avalanche

В thought-inbox копится больше кандидатов, чем коммиттер может осмысленно судить.

Симптомы: растущий хвост необработанных гипотез, устаревание pending-материала, деградация explainability.

Предохранители: aging pending-кандидатов, priority queues, дешёвый triage, quota на генерацию новых fusion-предложений.

23.6. Таблица быстрой диагностики

Наблюдаемый симптом	Вероятный класс поломки	Первая проверка
ответы слишком общие	hub collapse	visits vs degree, top bridge nodes
контрпример лежит в графе, но не всплыл	bottleneck starvation	contradiction hit rate, conductance
много красивых выводов, мало объяснений	provenance erosion	provenance coverage, support-skeleton audit
часто уверена, потом резко переобувается	premature certainty	distribution of conf, class priors
мрачные циклические ответы	mood lock-in	autocorrelation of mood, valence skew
контекст раздут и дорог	context bloat	evidence/glue ratio, token budget usage

23.7. Главный вывод

Принцип 23.1 (Failure mode важнее красивой общей метафоры)**.** Пока у памяти нет таблицы собственных поломок, она остаётся литературным образом. Как только у неё появляется каталог типичных сбоев, наблюдаемые симптомы и предохранители, она становится инженерным объектом.

24. Архитектурные развилки и исследовательские продолжения

24.1. Зачем обсуждать альтернативы

Хороший курс не только излагает выбранную архитектуру, но и показывает, от чего она отказывается. Иначе решения начинают казаться естественными, как будто другой дороги не было. У архитектуры памяти дорога всегда была.

24.2. Граф против плоского векторного хранилища

Что выигрывает плоское хранилище. Дешёвый semantic lookup, простая эксплуатация, понятная масштабируемость.

Что оно проигрывает. Явные отношения, адресную ревизию, точное удаление потомков, provenance и структурный forensic retrieval.

Принцип 24.1 (Векторный индекс — хороший вход, плохая конституция)**.** Векторное хранилище полезно как инструмент поиска якорей и похожих эпизодов, но плохо подходит на роль самой памяти, если память должна поддерживать объяснение, ретракцию и точную адресность.

24.3. Граф против чисто символической базы

Символическая система с жёсткими правилами и полноценным TMS превосходит графовую память в прозрачности и строгости выводов. Но она проигрывает там, где память должна жить на неполном, шумном и ассоциативном материале.

Выигрыш символики. Строгая ретракция, богатая дедукция, точный контроль зависимостей.

Потеря символики. Дорогая ручная типизация, слабая терпимость к неполным cue, плохая естественность ассоциативного вспоминания.

24.4. Граф против динамической памяти

В приложении о динамической памяти уже обсуждаются резервуары, аттракторы и квантовые ветви. Здесь важно зафиксировать архитектурный смысл сравнения.

Сильная сторона динамики. Pattern completion, богатые нелинейные траектории, естественная работа с частичным cue.

Слабая сторона динамики. Почти полное разрушение того, ради чего граф строился как юридически и инженерно адресуемая память: точного forgetting, provenance, явного удаления, объяснимой ретракции.

Самый опасный соблазн

Соблазн состоит в том, чтобы принять красивую динамику за замену всей памяти. Но значительная часть требований к Рин идёт не из любви к биологическим метафорам, а из необходимости точной инженерии: кто сказал, когда сказал, что было удалено, какая гипотеза опровергнута, какие потомки нужно пересчитать.

24.5. Новая добавка: правило чистого интерфейса для дальних ветвей

Это расширение полезно зафиксировать явно.

Принцип 24.2 (Новая ветвь имеет право жить только за чистым интерфейсом)**.** Если в систему добавляется резервуарный, аттракторный или квантовый слой, он не должен получать право произвольно переписывать явный граф. Иначе выигрыши в ассоциативной динамике будут куплены ценой потери адресности и аудируемости.

Практически это означает:

1. динамический слой может предлагать кандидатов;
2. может модулировать retrieval;
3. может служить дополнительным feature-generator;
4. но финальный write-path остаётся у явной памяти.

24.6. Почему Stage 1 всё ещё главнее дальних ветвей

Легко влюбиться в дальнюю ветку раньше, чем построен рабочий позвоночник системы. Но без Stage 1 невозможно даже понять, что именно новая ветвь улучшает.

Как выглядит плохой порядок разработки

Сначала вводится красивая динамическая подсистема. Затем оказывается, что у памяти до сих пор нет чистого удаления, provenance и нормального Inspector. Тогда любое улучшение retrieval невозможно честно измерить: неясно, что изменилось из-за новой динамики, а что ломалось изначально.

24.7. Порог входа новой ветви

Дальняя ветвь заслуживает интеграции только если выполняются три условия.

1. На базовом стенде сформулирована измеримая слабость текущего графового решения.
2. Новая ветвь атакует именно эту слабость, а не просто выглядит богато.
3. Существует протокол, который сравнивает ветви на одном и том же стенде.

24.8. Вывод

Архитектура памяти не обязана навсегда остаться ровно такой, как в первой рабочей версии. Но она обязана знать, что в себе является сердцевиной, а что — будущим расширением. Сердцевина здесь проста: явная адресуемая память, управляемый retrieval, ревизуемая эпистемика и инженерная проверяемость. Всё остальное должно усиливать это ядро, а не растворять его.

25. Обучение без «обучения»

25.1. Мы шлифуем веса, не занимаясь глубоким обучением

Возникает верное ощущение: мы будто учим модель без «учить» и без «модели». Вроде бы это не глубокое обучение и не графовая нейросеть, но тем не менее мы шлифуем веса и параметры графа — по-своему. У этого ощущения есть точное содержание.

Система, которую мы строим, — параметрическая: у неё есть веса рёбер, уровни уверенности, активации, и они обновляются опытом. Это *обучение* в широком смысле, даже без обратного распространения. Конкретно это два хорошо известных паттерна, наложенных друг на друга:

- **Хеббовское/ассоциативное обучение** (семейство сетей Хопфилда; принцип Хебба «что вместе активируется, вместе укрепляется»). Правило Хебба — локальное усиление связи по со-активации:

  $$\Delta w_{ij} \;\propto\; a_i\, a_j,$$

  а ассоциативная память Хопфилда хранит образы в минимумах энергии $E = -\frac{1}{2}\sum_{ij} w_{ij} a_i a_j$ и извлекает их спуском по E. Наши fusion и консолидация, усиливающие со-активированные рёбра, — это буквально хеббовское обучение на графе; граф памяти со spreading activation и уверенностью — современная ассоциативная память, где извлечение есть спуск к аттрактору.

- **Байесовское обновление уверенности** (глава 14): локальные апдейты Δconf со знаком, гейтом источника и дисконтом независимости.

25.2. Это message passing — но с локальными правилами

Формально наша система — это *передача сообщений* по графу (как в графовой нейросети), но «параметры» (веса рёбер, уверенности) обновляются *локальными* правилами (хеббовскими, байесовскими, распадом), а не глобальным градиентным спуском по функции потерь. Иначе говоря, это message passing *без обратного распространения*. Где мы сидим на карте методов — *между* классической ассоциативной памятью (Хопфилд) и графовыми нейросетями.

Два способа настроить пианино

Глубокое обучение настраивает все струны сразу по глобальному критерию («чтобы аккорд звучал верно»), продвигаясь градиентом. Хеббовская система настраивает каждую струну по локальному правилу («подтяни там, где чаще играют вместе»). Обе «учат» инструмент, но первая знает про весь аккорд, а вторая — только про соседние струны. Наша память —

второго рода.

25.3. Мост к «настоящей» графовой сети

Может ли это стать графовой нейросетью в полном смысле? Да, и мост поучителен: можно *мета-обучать* сами правила обновления — учить оптимальное правило Δconf, оптимальный распад, оптимальное смещение обхода градиентным спуском вместо ручного дизайна. Это «обучение тому, как помнить» (learning to remember), мета-обучение операторов памяти — реальное исследовательское направление.

Но не на старте

Мета-обучение правил меняет интерпретируемость на чёрный ящик, которому нужна разметка и который заново вносит проблему непрозрачности субстрата. На старте — ручные локальные правила: они интерпретируемы, отлаживаемы и не требуют обучающих данных. «Может, и незачем» — верно для текущего этапа. Но мост реален, и полезно знать, что стоишь ровно на нём: сегодняшняя система — предельный, «нулевой» случай обучаемой графовой сети, где правила заданы руками, а не выучены.

25.4. Связь с обучаемыми графовыми моделями

Это же наблюдение позволяет увидеть место памяти Рин на общей карте методов. Обучаемые графовые модели изучают операторы передачи сообщений, параметры которых подбираются данными; память Рин использует тот же общий класс локальных переносов, но пока с правилами, заданными вручную. Если однажды правила памяти начнут мета-обучаться, различие между «графовой моделью памяти» и «обучаемой графовой системой» станет не принципиальным, а предметным: один и тот же класс операторов, применённый к разным видам графов и разным целям.

26. Заключение: единая картина

26.1. Один субстрат, много операторов

Если из монографии стоит унести одну мысль, то вот она: память Рин — это *один типизированный темпоральный граф с метаданными*, а всё богатство поведения — *операторы* над ним. Retrieval, пропагация уверенности, забывание и консолидация, ночные обходы, настроение и слой личности — не отдельные подсистемы со своими моделями памяти, а разные способы читать и писать одни и те же поля (`confidence`, `activation`, `valence`, `provenance`, `need_to_discover`, временной слой) на одном графе.

26.2. Четыре кита связаны

Отсюда — практический вывод о разработке. Четыре кита (правильный retrieval, правильная пропагация уверенности, правильное забывание/консолидация, правильные ночные обходы) сцеплены через общее состояние: забывание меняет топологию, топология — retrieval, retrieval — что пропагируется, пропагация — что найдёт скраб. Их нельзя тюнить порознь. Именно поэтому нужен стенд с ground truth (глава 22): без него взаимодействие операторов не измерить, и отладка «на живой Рин» найдёт проблемы слишком поздно.

26.3. Сквозные принципы

Несколько идей проходят через всю монографию и заслуживают повторения:

- **Гипотеза, а не факт.** Всё, что порождено выводом (fusion, реакции), пишется провизорно, с низкой уверенностью и провенансом; управление — на границе коммита. Это защищает от превращения обаятельной ассоциативности в фабрику ложных убеждений.
- **Внутрь, а не наружу.** Значимость (гейт консолидации характера) берётся из само-оценивания, а не из внешнего одобрения; иначе строится угодник, что противоречит ядру личности.
- **Ингредиенты, а не реакции.** Граф хранит материал; реакция композируется под настроение и мотивацию, а личность — смещение во время обхода, а не таблица весов.
- **Континуум, а не бинарность.** Врождённое/приобретённое, семантическое/эпизодическое, стабильное/пластичное — это оси глубины консолидации, а не выключатели.

- **Топология незрима, но правит.** Любой локальный процесс на графе наследует спектрально-топологический транспорт; телепорт-скраб — не украшение, а необходимая эпистемическая гигиена с доказуемой гарантией покрытия.

26.4. Единая спина курса

И последнее. Консолидация есть огрубление графа; тот же вопрос «что выживает при переходе fine $\to$ coarse» возникает во всякой сложной графовой системе памяти. Over-squashing, спектр лапласиана, эффективное сопротивление, персистентный лапласиан, обучаемые операторы передачи сообщений — это не декоративный набор слов, а общий математический язык, на котором можно обсуждать и retrieval, и забывание, и перенос структуры между уровнями абстракции. Отрефлексировать это явно значит увидеть память не как набор эвристик, а как частный случай более широкой науки о структуре и потоке на графах.

26.5. Что делать дальше

Порядок разработки, вытекающий из монографии: сначала прибить *схему субстрата* (узел/ребро, метаданные, что значит коммит и кто его гейтит); затем собрать Ярус-1 стенд и погнать батарею тестов, начиная с самого дешёвого (концентрация π на хабах — сразу показывает, что диагностика жива); затем локальное правило Δconf (знак, гейт источника, дисконт независимости), поверх которого пороги и ветвящийся коэффициент навешиваются тривиально; параллельно — построить φ-скорер как измерительный прибор для отбора голоса и фильтрации выходов. Настроение, сон и слой личности — усилители поверх субстрата, которому сначала надо стать доверенным.

A. Нотация

Сводка обозначений, используемых по тексту.

Символ	Смысл
$G = (V, E)$	граф памяти: узлы V, направленные типизированные знаковые рёбра E
v, u	узлы (воспоминания, факты, сущности, концепты)
$\mathcal{N}(A)$	окрестность узла A (соседи, фронт распространения)
$\mathbf{x}_v \in \mathbb{R}^d$	векторное представление содержания узла (эмбеддинг)
$c_v = \text{conf}(v) \in [-1, 1]$	знаковая уверенность: $+$да, $-$нет, 0 незнание
a_v	транзиентная активация узла при обходе
$\text{importance}(v)$	важность узла $\in [0, 1]$: salience при кодировании + центральность; гейтит прунинг
r_{uv}	тип отношения на ребре $u \to v$; знак $\pm$
w_{uv}	вес (эвиденциальная сила) ребра
$m_{u \to v}$	сообщение, передаваемое по ребру: $m_{u \to v} = \phi_{r_{uv}}(h_u, h_v, e_{uv})$
τ	постоянная времени слоя (интегратора)
B_v	базовая активация ACT-R узла v
π	стационарное распределение случайного блуждания
P	матрица переходов блуждающего агента; P, Q — операторы проекции/поднятия при огрублении
α	доля телепортов (рестартов) в блуждании
L, L^+	лапласиан графа и его псевдообратный
$R_{\text{eff}}(i, j)$	эффективное сопротивление между узлами i, j
σ	ветвящийся коэффициент лавины пропагации
$\theta_{\text{emergent}}, \theta_{\text{temp}}$	верхний и нижний пороги распространения (гистерезис)
$\Delta\text{conf}(B \mid A)$	локальное изменение уверенности соседа B под влиянием факта A
$\text{mood}_t \in \mathbb{R}^k$	вектор настроения в момент t (аффективное пространство)
$\varphi(\cdot) \in \mathbb{R}^d$	скорер по осям характера
$\mathbf{t}$	целевые координаты Рин в пространстве характера
$\text{Beta}(\alpha, \beta)$	апостериорное распределение успешности реакции
η	темп обновления (learning rate) локального правила

B. Глоссарий внешних идей

Краткие описания инженерных и научных идей, на которые опирается монография, чтобы читатель не был вынужден держать в голове внешний контекст. Порядок — тематический.

Память и активация

Базовая активация ACT-R. В когнитивной архитектуре ACT-R (Anderson) сила следа в памяти есть функция *частоты* и *свежести* обращений со степенным затуханием: $B_v = \ln \sum_k t_k^{-\alpha}$, где t_k — давность k-го обращения. Следствие: частые и *разнесённые во времени* обращения укрепляют след сильнее массовых (эффект распределённого повторения).

Spreading activation. Модель ассоциативного припоминания (Collins–Loftus): активация растекается от активированного узла по рёбрам, затухая с расстоянием. Формальная реализация — случайные блуждания и персонализированный PageRank.

Сети Хопфилда, обучение Хебба. Ассоциативная память как граф со взвешенными связями и динамикой активации, сходящейся к аттрактору (энергетический минимум). Правило Хебба: связь между совместно активными узлами усиливается («что вместе активируется, вместе укрепляется»). Наши fusion/консолидация — это хеббовское обучение на графе.

Убеждения и истинность

Система поддержки истинности (TMS/ATMS). Механизм (Doyle; de Kleer), хранящий *обоснования* выводов: для каждого производного убеждения записано, из каких посылок оно получено. При ретракции посылки её обоснованные следствия автоматически помечаются на пересмотр. Лечит «ползучую веру».

Недоопределённость (underdetermination). Тезис Дюэма–Куайна: данные не определяют теорию однозначно — одни и те же наблюдения совместимы с разными гипотезами. Когнитивное проявление — преждевременный коллапс: ум схлопывает суперпозицию совместимых гипотез в одну слишком рано.

Belief perseverance. Устойчивость убеждения к опровержению его основания: даже после опровержения посылки её следствия продолжают влиять.

Петлевое распространение (loopy belief propagation). Распространение уверенности по циклическому графу может привести к тому, что убеждение подтверждает само себя по петле (эхо-камера), искусственно раздувая уверенность. Лечится дисконтом по общей родословной свидетельств.

Динамика и критичность

Самоорганизованная критичность (SOC). Класс систем (Bak–Tang–Wiesenfeld; образ — куча песка), самонастраивающихся к критической точке, где отклики следуют степенному закону: почти все малы, изредка случаются крупные лавины. Нейронные лавины (Beggs–Plenz) — проявление SOC в мозге.

Ветвящийся коэффициент σ. Среднее число узлов, активируемых одним активным. $\sigma < 1$ — субкритично (затухает), $\sigma = 1$ — критично, $\sigma > 1$ — суперкритично (runaway). Управляющий параметр лавины пропагации.

Гистерезис (триггер Шмитта). Схема с двумя порогами (включения и выключения), не дающая системе «дребезжать» у единственного порога.

Графы: спектр, поток, топология

Стационарное распределение π. Долгосрочная частота посещений узлов случайным блужданием; глобальный объект (перронов собственный вектор матрицы переходов), а не локальные вероятности.

Время перемешивания (mixing time). Скорость сходимости блуждания к π; управляется спектральным зазором матрицы переходов.

PageRank / персонализированный PageRank / RWR. Случайное блуждание с телепортом (рестартом): с вероятностью α прыжок в узел из распределения рестарта. Телепорт делает блуждание эргодическим — гарантирует посещение всех узлов.

Эффективное сопротивление R_{eff}. Мера «электрической» удалённости узлов, если рёбра — резисторы: $R_{\text{eff}}(i,j) = (e_i - e_j)^\top L^+ (e_i - e_j)$. Большое R_{eff} между узлами ограничивает передачу влияния между ними.

Проводимость / константа Чигера. Мера «узости» перешейков графа; неравенство Чигера связывает её со спектральным зазором. Малая проводимость = узкое место.

Over-squashing. Удушение дальней связи: информация от экспоненциально растущей окрестности сжимается в вектор фиксированной размерности, проходя через узкое место графа; влияние дальних узлов затухает.

Over-smoothing. Пересглаживание: при многократной диффузии представления узлов сходятся к низкоразмерному консенсусу, теряя различимость.

Кривизна графа (Оливье, Форман). Дискретные аналоги кривизны; отрицательно искривлённые рёбра — мосты между сообществами, места over-squashing. Полезны как локатор узких мест.

Анти-энтропийный скраб. Приём из распределённых хранилищ (ZFS scrub; Merkle-based anti-entropy gossip): фоновая случайная перепроверка всего хранилища, поскольку неизвестно, какая запись повреждена.

Огрубление графа (graph coarsening), coarse-to-local. Замена мелкого графа крупным с сохранением релевантных свойств (спектра, разрезов, потоков). Центральный вопрос — какой сигнал выживает при переходе от мелкого масштаба к крупному; в памяти это консолидация.

Персистентный лапласиан, TDA. Топологический анализ данных: как меняются топологические признаки (числа Бетти, спектр лапласиана) при изменении масштаба (фильтрации). Персистентный лапласиан — спектральное обобщение персистентной гомологии.

Выразительность и обучение

WL-тест, GIN. Тест Вейсфейлера–Лемана — предел различающей способности обычной передачи сообщений; GIN (Xu и др.) достигает этого предела инъективной агрегацией. PNA — многостатистическая агрегация (сумма/среднее/макс/дисперсия).

Передача сообщений (GNN). Параметрическая функция на графе, обучаемая градиентным спуском: узел обновляется агрегированными сообщениями соседей. Наша память — та же передача сообщений, но с локальными (не обучаемыми градиентом) правилами.

LoRA. Дешёвое дообучение: вместо всех весов обучается низкоранговая добавка $\Delta W = BA$ (ранг $r \ll$ размерности), малая по памяти.

DPO (Direct Preference Optimization). Дообучение под предпочтения без отдельной модели вознаграждения: оптимизация на тройках (промпт, выбранный ответ, отвергнутый ответ).

Эмоции и данные

Теория оценивания (appraisal): OCC, Шерер. Эмоция как результат оценки события по измерениям: релевантность/новизна (ошибка предсказания), конгруэнтность целям, валентность, интенсивность, атрибуция причины (agency).

Предиктивная обработка, ошибка предсказания. Взгляд (Friston и др.), по которому мозг минимизирует ошибку предсказания; величина ошибки — естественная мера значимости события.

ConceptNet / Numberbatch. Ассоциативно-таксономический граф здравого смысла (взвешенные типизированные рёбра) и его родной эмбеддинг Numberbatch.

ATOMIC / ATOMIC10x. Граф причинно-социального здравого смысла (если субъект делает Y, то намерение/реакция/эффект). ATOMIC10x — крупная версия, дистиллированная языковой моделью и отфильтрованная критиком.

Wikidata. Фактологическая база с моделью утверждения: квалификаторы (время), references (провенанс), ранг (недеструктивная ревизия). Ценна схемой, не содержанием.

Generative Agents. Симуляция агентов с потоком памяти, retrieval по свежести×важности×релевантности и рефлексией (порождением обобщений); прообраз субстрата.

Распределение Бета, сэмплирование Томпсона. Сопряжённый апостериор для доли успехов; удобно моделирует «успешность реакции» (успехи α, провалы β) и баланс исследования/эксплуатации.

C. Сводка алгоритмов

Псевдокод ключевых операторов, собранный в одном месте для удобства. Это скелеты, а не финальные реализации: они фиксируют логику, обсуждённую в главах, но опускают инженерные детали (батчинг мутаций, кэширование π, индексы эмбеддингов).

C.1. Единый блуждающий агент (три политики)

```
def walk(G, start_nodes, mode, steps, mood, query=None):
    # mode задаёт политику через параметры: bias перехода, alpha телепорт(),
    # bias_uncertainty. Один движок — три режима глава( про обход).
    if mode == ”retrieval”:   alpha, bu = 0.0, 0.0
    if mode == ”curiosity”:   alpha, bu = 0.05, 1.0     # тянет к need_to_discover
    if mode == ”sleep”:       alpha, bu = 0.30, 0.0     # телепортскраб–

    active = set(start_nodes); visited = Counter()
    for _ in range(steps):
        A = sample(active)
        if random() < alpha:                 # телепорт рестарт()
            B = random_node(G)
        else:
            # P(A → B) ∝ w_AB · rel(B|query) · moodcong(B|mood) · unc(B)^bu
            B = sample_neighbor(G, A, weights=transition_scores(
                    G, A, query, mood, bias_uncertainty=bu))
        visited[B] += 1
        maybe_fire_operations(G, B, mode)    # retrieval / fusion / inspect
        active = frontier(active, B)
    return visited
```

C.2. Локальное правило и лавинная пропагация

```
def propagate(G, A, sigma_target=0.9):
    # Пропагация нового факта A как затухающей лавины с контролем ветвления.
    frontier = link_initial(G, A)            # A -> N(A), напр. последние top-k
    while frontier:
        next_frontier = []
        # нормировка под целевой ветвящийся коэффициент sigma < 1
        gain = branching_normalizer(frontier, sigma_target)
        for B in frontier:
            d = delta_conf(G, A, B) * gain    # см. ниже: знак, источник, независимость
            G.confidence[B] = clip(G.confidence[B] + d, −1, 1)  # знаковая уверенность
            th_e = theta_emergent(G, B)       # растёт с глубиной консолидации
            if abs(d) > th_e:                 # крупная переоценка -> в ширину
                next_frontier += neighbors(G, B)
            elif abs(d) > THETA_TEMP:         # промежуточная зона -> пометить
```

```
                mark(G, B, ”need_to_revise”)
            # else: глохнет здесь
        frontier = next_frontier

def delta_conf(G, A, B):
    sign      = edge_sign(G, A, B)            # supports (+) / contradicts (−)
    strength  = edge_weight(G, A, B)
    gate      = G.confidence[A] ∗ source_trust(G, A)   # карантин нового факта
    indep     = independence_discount(G, A, B)         # антиэхо− по провенансу
    return ETA ∗ sign ∗ strength ∗ gate ∗ indep ∗ G.confidence[A]
```

C.3. Забывание, прунинг, консолидация

```
def nightly_maintenance(G):
    for v in G.nodes:
        G.strength[v] = act_r_base_level(G.access_log[v])   # частота + свежесть
    for v in low_activation(G):
        if importance(G, v) low and confidence(G, v) low:
            drop(G, v)                                       # редкоеиневажное−−
    for cluster in redundant_episodic_clusters(G):
        merge_to_semantic(G, cluster)   # огрубление: эпизоды −> обобщение
    # эмбриональным узлам − меньшая скорость распада континуум( глубины)
```

C.4. Телепорт-скраб (анти-энтропия)

```
def scrub(G, budget):
    # Случайная инспекция: гарантия покрытия за счёт эргодичности телепорта.
    for _ in range(budget):
        v = random_node(G)                    # рестарт куда угодно
        for w in local_walk(G, v, depth=k):
            if contradicts_any(G, w):         # столкновение −> ревизия
                demote(G, w); mark(G, justificands(G, w), ”need_to_revise”)
            if flagged(G, w, ”need_to_discover”):
                maybe_fetch_external(G, w)     # любопытствовнешний/ вход
```

C.5. Настроение и монитор руминации

```
def mood_update(mood_layers, event):
    # Стек протекающих интеграторов с разными tau; актуальное = суперпозиция.
    for L in mood_layers:                     # session, day, ..., life
        mood_L = (1 − 1/L.tau) ∗ mood_L + (1/L.tau) ∗ appraise(event)
    mood_now = sum(w_L ∗ mood_L for L in mood_layers)
    if autocorr(history(mood_now)) > RUMINATION_TH:   # залипла валентность
        trigger(one_of=[”reframe”, ”attention_shift”, ”invoke_sleep”])
    return mood_now
```

C.6. Оценивание, значимость и консолидация реакции

```
def encode_event(G, event, mood_prev, mood_now):
    ap = appraisal(event)                    # surprise, goal_congruence, valence...
    salience = ap.surprise * ap.goal_relevance
    #  дешёвый прокси значимости, почти бесплатный:
    salience_cheap = abs(mood_now - mood_prev)
    consolidate(G, event, weight=salience)    # гейт значимости, НЕ одобрения
    if ap.surprise high: mark(G, event, "need_to_discover")

def update_reaction(G, situation, user, reaction, received_ok):
    e = reaction_edge(G, situation, user, reaction)   # условь по ситуация(, отношение)
    if received_ok: e.beta.alpha += 1
    else:           e.beta.beta  += 1                 # угасание, не второй коммит
    e.confidence = e.beta.alpha / (e.beta.alpha + e.beta.beta)
    if e.confidence > CONSOLIDATE_TH: promote_to_slow_layer(G, e)
```

С.7. Скорер характера и отбор голоса

```
def phi(text, axes):                         # LLMсудья– по рубрикам осей
    return [judge(text, rubric_a) for rubric_a in axes]   # вектор в R^d

def select_voice_corpus(candidates, target_t, axes):
    scored = [(c, phi(c, axes)) for c in candidates]
    # отборперевзвешивание/, чтобы среднее выборки приблизилось к target_t
    return greedy_match_mean(scored, target_t)   # не усреднять муз, а фильтровать
```

С.8. Стенд эксперимента

```
def run_experiment(build_graph, plant, operator, metric):
    G = build_graph()          # ярус 1: SBM/BA/WS + подсаженные сценарии
    truth = plant(G)           # ground truth: что должно произойти
    operator(G)                # walker / propagate / scrub / decay ...
    return metric(G, truth)    # precision/recall починки, сходимость, покрытие
```

D. Будущее: стоит ли переходить на гиперграфы

Это приложение — рассуждение о структурном направлении, а не готовая рекомендация к внедрению. Вопрос: логично ли в будущем сделать память Рин *гиперграфом*, а не обычным графом.

D.1. Проблема, которая это мотивирует

Обычный граф связывает узлы *попарно*. Но вывод часто *n-арен*: он рождается только при одновременном присутствии нескольких посылок, и ни одно подмножество не достаточно. Канонический пример (глава 10):

$$A = \text{«есть коты»}, \quad B = \text{«живёт в Португалии»}, \quad C = \text{«ездит на пары»},$$

$X = f(A, B, C) =$ «купи котам корм на выходных (в будни по пути с пар не успеть)».

Каждая посылка ещё и подтягивает свой контекст (коты→нужен корм; Португалия→ магазины рано закрываются; универ→поздний возврат; см. главу 10), и только их совмещение даёт X. Ни $A \wedge B$, ни $B \wedge C$, ни $A \wedge C$ не дают X. Попарные рёбра между посылками могут вообще отсутствовать: «Португалия» и «коты» напрямую не связаны.

D.2. Почему симплексы — плохой ответ

Первое, что приходит в голову, — симплициальные комплексы: тройка $\{A, B, C\}$ как 2-симплекс (треугольник). Но у симплициального комплекса есть жёсткое требование *downward closure*: если есть 2-симплекс $\{A, B, C\}$, то обязаны быть и все его грани — рёбра $A-B$, $A-C$, $B-C$. А в нашем случае этих попарных связей *нет и не должно быть*: «Португалия» и «коты» не связаны. Симплициальная память вынудила бы нас проводить *ложные* попарные рёбра ради формальной корректности структуры. Это плохой обмен.

D.3. Гиперребро — прямой ответ

Гиперребро связывает *множество* узлов сразу: $e = \{A, B, C\}$, и оно *не требует* существования попарных рёбер между ними. Это ровно та семантика, что нужна n-арному выводу: «эти три вместе порождают это», без обязательства, что они связаны попарно.

Гиперграф против графа

В графе «дружба» — всегда пара. В гиперграфе можно сказать «эти пятеро были в одной поездке» одним ребром, не утверждая, что каждый дружит с каждым. Для памяти это разница между «связь между двумя воспоминаниями» и «условие из нескольких воспоминаний, порождающее

вывод».

D.4. Ключевое наблюдение: мы уже наполовину гиперграф

Важно не пропустить: **текущая схема Рин уже кодирует гиперрёбра** — просто не называя их так. Когда fusion порождает узел X с провенансом $\{A, B, C\}$ (глава 10), это *factor-graph*-кодировка гиперребра $\{A, B, C\} \to X$: сам вывод X становится узлом, а его рёбра-обоснования указывают на посылки. Попарных рёбер между A, B, C при этом не требуется. Формально это просто **двудольный граф** (узлы-посылки $\leftrightarrow$ узлы-факторы), и его *полностью достаточно* для n-арного вывода — никакой высшей размерности не нужно.

Значит «переход на гиперграфы» — это не «переписать всё», а более узкий вопрос: *делать ли гиперрёбра первоклассными объектами* (со своим обходом и спектральной теорией), или достаточно узла-фактора.

D.5. Первоклассные гиперрёбра: за и против

За. Появляются *гиперграф-нативные* операции: обход/spreading activation, уважающий *совместность* срабатывания (гиперребро «зажигается», когда активны *все* его узлы, а не любой один); гиперграфовые лапласианы и спектральная диагностика (аналоги главы 8 для n-арной структуры). Это может точнее моделировать «условие из нескольких посылок».

Против. Алгоритмика гиперграфов менее зрелая и дороже графовой. На практике гиперграф часто *сводят* к графу — clique-expansion (заменить гиперребро кликой, что как раз возвращает ложные попарные рёбра!) или star-expansion (ввести узел-фактор — то есть ровно наша factor-кодировка). Есть спектральная теория гиперграфов (нормализованный гиперграфовый лапласиан; тензорные методы), но она сложнее и вычислительно тяжелее, а динамический мутирующий гиперграф — тем более.

D.6. Рекомендация

Принцип D.1 (Гиперграф — да, но постепенно)**.** Ближний горизонт: оставить *factor-узлы* (вывод X с провенансом-множеством). Это уже даёт n-арную семантику без гиперграфовой машинерии и без ложных попарных рёбер. Средний горизонт: сделать гиперрёбра первоклассными *только там*, где нужны гиперграф-нативные операции — совместное срабатывание при обходе и n-арная спектральная диагностика. Симплициальные комплексы обходить стороной: их downward closure противоречит разреженности реальных выводов.

Заметим, что star-expansion (факторные узлы) и первоклассные гиперрёбра — это два взгляда на одно и то же; переход между ними инкрементален, а не революционен. Полное же объединение узлов, рёбер и выводов в объекты одного рода разной размерности — это уже клеточные комплексы (приложение E).

E. Будущее: клеточные комплексы как язык для составных актов

E.1. Зачем вообще думать о клетках

Гиперграфовая ветвь уже показывает, что попарных рёбер иногда недостаточно. Но и гиперребро — ещё не вся картина. У памяти есть не только набор совместно действующих посылок, но и *структура их совместности*: границы, циклы, составные акты, которые хочется видеть не как мешок из входов, а как объект более высокой размерности.

E.2. Базовая идея

Клеточный комплекс позволяет держать объекты разных размерностей:

1. 0-клетки как узлы памяти;
2. 1-клетки как обычные рёбра;
3. 2-клетки как составные акты, выводы или сцены;
4. при желании и более высокие клетки для сложных конфигураций.

Это полезно тогда, когда память должна различать не только “что с чем связано”, но и “что из чего составлено как единый акт”.

E.3. Что это даёт памяти

Во-первых, можно хранить составную ситуацию как один объект, не раскладывая её полностью на попарные связи.

Во-вторых, появляются естественные границы: если 2-клетка кодирует составной вывод или эпизодический акт, её boundary прямо указывает на набор посылок или компонент.

В-третьих, становится богаче язык локальной ревизии: можно понижать не только узел или ребро, но и целую составную конфигурацию.

E.4. Чего это не решает

Новая структура не заменяет дисциплину provenance

Сам по себе переход к клеткам не делает память ни истиннее, ни объяснимее. Если у системы нет хорошего provenance, gates, deletion-path и ревизии, клеточный комплекс лишь переносит старые проблемы в более красивую геометрию.

E.5. Правильный статус этой ветви

Для текущего проекта клеточные комплексы — это язык *будущего структурного обобщения*, а не ближайшая реализация. Их стоит читать так:

1. как концептуальный мост между графом и n-арными актами;
2. как способ точнее думать о границах составных выводов;
3. как материал для дальнего сравнения с гиперграфами и динамической памятью.

Именно в таком статусе эта ветвь совместима с нынешним roadmap: не как срочная замена субстрата, а как более богатая геометрия для будущего поколения памяти.

F. Будущее: память как динамика (резервуар, ассоциативные аттракторы, кванты)

Вердикт вперёд: это смена субстрата, а не смена топологии

Приложения D и E спрашивали «какой *формы* должен быть граф». Это приложение спрашивает радикальнее: *а нужен ли символический граф вообще*, или память стоит держать как *динамику* — состояние большой рекуррентной системы, где факт не хранится объектом, а «протоптан» в связях. Идея красивая, глубоко укоренённая в нейронауке, и **мы её сейчас не строим**. Но не потому, что она наивна — наоборот, разберём её всерьёз. Причина в том, что для *компаньона* с требованиями редактируемости, забывания-по-команде, провенанса и поддержки истинности распределённая память в весах ломает ровно те свойства, ради которых мы и завели типизированный граф. Ниже — честный разбор: что здесь ценно (и что мы, оказывается, уже делаем), что дорого, и как оставить дверь открытой, не переписывая мозг.

F.1. Постановка: память в весах, воспоминание как траектория

Наш субстрат (гл. 5) — типизированные узлы и явные рёбра: у «узла Helena» есть содержание, у ребра — тип и вес. Альтернатива, которую предлагает эта ветка, меняет само определение:

$$\text{память} = (G, W, \theta), \qquad \text{воспоминание} = \text{траектория в пространстве состояний}, \qquad \text{обучение}$$

Здесь G — топология (какие пути вообще есть), W — веса связей (насколько они протоптаны), а θ — параметры узлов-вычислителей. Узел #48372 не значит «Лиссабон»; смысл распределён по тысячам узлов и существует только как *паттерн активации + связность + траектория*. Запрос не «ищет chunk», а *возмущает* систему: бросаем заряд, сеть эволюционирует, из неё выныривает активный подграф, и уже его вербализует LLM.

Бассейн, камни, датчики

Классический образ резервуарных вычислений: бассейн воды, в который бросают камни (входной поток). Каждый камень поднимает волны; следующий приходит, пока прежние не улеглись, — и поверхность в момент t несёт отпечаток нескольких последних бросков. Датчики по краям считывают состояние, и простой линейный слой учится по ним отвечать. Сам бассейн обучать не надо: его физика уже превращает историю входов в богатое состояние. Reservoir computing — это *случайное нелинейное динамическое отображение признаков для последовательностей*, аналог kernel trick, но во времени.

Важная оговорка сразу: *классический* резервуар (Echo State Network, Liquid State Machine) держит W_{res} *замороженным* и обучает только считывающий слой. Он обладает *затухающей* памятью (fading memory) на конечный горизонт — прекрасно для временных рядов, но не для «узнала в 2026-м, вспомнила в 2031-м». Для памяти длиной в годы нужен *пластичный* резервуар (W медленно меняется опытом) плюс механизм устойчивого хранения из мира *ассоциативных аттракторов* (Hopfield/CA3): воспоминание — это *бассейн притяжения*, частичный cue скатывается в него (pattern completion). То есть это не «reservoir computing» в чистом виде, а гибрид на пересечении резервуара, ассоциативной памяти, аттракторных сетей и пластичной динамики.

F.2. Математическая ось: один лапласиан — три динамики

Самое элегантное в этой ветке — то, что *одна и та же* топология допускает три принципиально разные динамики. Для графа с лапласианом $L = D - A$:

$$\text{диффузия:} \quad \dot{x} = -Lx \quad \Rightarrow \quad x(t) = e^{-tL}x(0), \tag{F.1}$$

$$\text{волна:} \quad \ddot{x} = -Lx \quad \Rightarrow \quad x(t) = \cos(t\sqrt{L})\,x(0), \tag{F.2}$$

$$\text{квантовое блуждание:} \quad i\dot{\psi} = L\psi \quad \Rightarrow \quad \psi(t) = e^{-itL}\psi(0). \tag{F.3}$$

Один L порождает e^{-tL} (рассеивающую память), $\cos(t\sqrt{L})$ (волновую, с интерференцией и без затухания) и e^{-itL} (унитарную квантовую). Это не словесная аналогия, а линейная алгебра: «бросить заряд в узел и смотреть, куда потечёт» имеет буквальный волновой и квантовый аналоги на той же сети.

Мы уже живём в углу e^{-tL}

Полезно осознать: наш ретрив (гл. 6) — это *и есть* диффузионный угол этой оси. ‘diffuse()‘ считает усечённое e^{-tL}-подобное распространение из seed-якорей; «вспышки, слабеющие к окраинам» — это спад e^{-tL}. То есть значительная часть красоты этого приложения у нас уже реализована, просто в скромной, редактируемой форме. Волновой и квантовый углы — это *другие операторы над той же топологией*, и их честно можно сравнить на одном графе (см. открытые вопросы в конце).

F.3. Нейронаучная опора (принцип, а не крысиный мозг)

Ветка опирается на реальные результаты, и стоит назвать их аккуратно. Обучение оставляет устойчивые следы в самой сети (engram-клетки: клетки, участвовавшие в формировании воспоминания, можно искусственно возбудить и вызвать его; хранение связывают с синаптическим усилением engram-ансамблей). Гиппокамп работает не как «жёсткий диск», а как система связывания/индексации и реактивации распределённых корковых представлений (hippocampal indexing theory), причём со временем память дрейфует от гиппокампа к коре (systems consolidation, replay). Complementary Learning Systems (CLS) прямо противопоставляет *быстро* обучающийся гиппокамп *медленно* обучающейся коре — и это вычислительно мотивированная защита от катастрофического забывания. Аттракторная память (Hopfield; CA3 как autoassociative network) хранит паттерн как бассейн притяжения. Мы заимствуем *принцип* (быстрый индекс + медленное распределённое хранилище + восстановление по частичному cue), не пытаясь эмулировать биологию до деталей.

F.4. Предлагаемая гибридная архитектура

Синтез, к которому сходится обсуждение, — не «резервуар вместо графа», а шестислойный объект плюс архив сбоку:

1. **Event encoder.** Приходит не «факт», а *событие* целиком — контент, говорящий, тон, контекст, время, настроение — и кодируется в вектор x_t. Источник знания становится *частью паттерна*, а не строкой БД (значит, память сможет и *ошибиться* в источнике — по-человечески).
2. **Hippocampal pattern separator / index.** Из x_t строится разреженный код $h_t = \text{kWTA}(Px_t)$ (активны немногие units из многих); похожие события разводятся в непохожие коды (pattern separation). Это «умный стартер»: куда бросить заряд.
3. **Fast associative memory.** Быстрая Hebbian-запись «эти вещи произошли вместе» ($H \leftarrow H + \eta_h h_t h_t^\top$), связывающая индекс с распределённым состоянием.
4. **Slow plastic cortical reservoir.** Большая разреженная рекуррентная сеть
$$r(t{+}1) = (1-\alpha)\, r(t) + \alpha\, \sigma\big(W r(t) + U x(t) + B h(t)\big),$$
где W *медленно* пластичен ($\eta_c \ll \eta_h$). Быстрый гиппокамп + медленная кора — это и есть CLS.
5. **Continuous activation field.** Живое поле $r(t)$ — «о чём Рин сейчас думает».
6. **Latent-to-LLM interface.** Мост от $r(t)$ к языковой модели.

Плюс **replay/consolidation** в простое (ночью гиппокамп многократно проигрывает след, медленно перестраивая W — «переваривание дня» не через `summarize()`, а через проигрывание), и **граф-архив** как «чёрный ящик самолёта» сбоку.

Принцип F.1 (Граф не выбрасывается — он становится архивом и учителем)**.** Ключевой примирительный ход: наш явный граф перестаёт быть *моделью* человеческой памяти и становится (а) *эпизодическим архивом* (что реально произошло, когда, откуда знаем) и (б) *учителем* для пластичной памяти (материал для replay-консолидации). Тогда специализация чистая: *граф* — «что действительно было записано», *резервуар* — «что Рин из этого сейчас помнит» (неточно, ассоциативно, с интерференцией). Это не дублирование, а разделение ролей.

F.5. Настроение как нейромодулятор и «пирожки Алисы»

Отдельная и, на мой взгляд, самая *сразу применимая* идея ветки: mood перестаёт быть вектором, который кладут в промпт, и становится *нейромодулятором* — он меняет *геометрию* recall:

$$W_{\text{effective}} = \gamma(m)\, W, \qquad A_{\text{effective}} = A \odot M(\text{mood}).$$

Тот же cue при разном настроении уходит по разным траекториям («помнишь Тбилиси?» $\to$ переезд $\to$ Ада; или $\to$ ОИЯИ $\to$ смена жизни). Инерция настроения тут — не недостаток, а *нужное* свойство: медленный low-pass-контроллер задаёт устойчивый ландшафт для быстрых процессов и не «дёргает руль» на каждый токен.

Гомеостаз как гормоны: заманчиво, но чинит болезнь, которую само же создаёт

Предложение «мониторить энергию поля $E(t) = \frac{1}{N}\sum r_i^2$ и впрыскивать седатив (поднять утечку λ, снизить крутизну β) при перевозбуждении или кофеин при апатии» — это настоящая идея из neuromodulated networks, и она красива. Но будем честны: это *лекарство от нестабильности, которую вносит сам пластичный резервуар*. На нашем текущем субстрате диффузия ограничена по построению (decay + floor + max_steps + max-CRDT активация, гл. 6) — «эпилептического шторма» просто негде взяться, и гормональный контроллер не нужен. Более того, он добавляет ровно то, чего мы весь проект избегали: свободные параметры ($E_{\min}, E_{\max}$, коэффициенты впрыска), которые нечем калибровать без downstream-сигнала. Вывод сбалансированный: *mood-как-нейромодулятор* — берём хоть сейчас (он и так входит в правило перехода через $\mathrm{mc}(B \mid \mathrm{mood})$); *гормональный гомеостаз* — осмыслен только если уже ушли в распределённый пластичный резервуар, то есть это инструмент *той* ветки, а не нашей.

F.6. Честный разбор: что покупаем и чем платим

Что действительно ценно. Ассоциативная *реконструкция* вместо поиска chunk'а; человекоподобная память (нечёткое время, ассоциативные ошибки источника, интерференция как feature); mood как геометрия recall; CLS как защита от катастрофического забывания; и красивая ось $e^{-tL}/\cos(t\sqrt{L})/e^{-itL}$, дающая единый язык для diffusion/wave/quantum памяти.

Чем платим — и это надо назвать прямо. Три технических риска и один принципиальный.

- **Readout-бутылочное горлышко.** Перевод $r(t) \in \mathbb{R}^{10^5}$ обратно в токены — самая хрупкая часть. Тупой декодер (MLP) может не извлечь смысл; умный — *сам* сделает всю работу и замаскирует бесполезность динамики (нужен экспериментальный контроль: сначала нарочно *тупой* D). А чтобы декодер обучить, нужны пары «состояние $\rightarrow$ правильный ответ» — которых для автобиографии *нет*. Откуда берётся обучающий сигнал — главный незакрытый вопрос. Элегантный вариант (резервуар модулирует hidden states LLM через cross-attention/prefix) концептуально безупречен, но требует глубокого fine-tuning локальной модели и почти исключает API-бэкенд; дешёвый вариант (декодировать в текст) — это, по сути, RAG с лишними шагами, теряющий главную приманку «память остаётся неязыковой».
- **Пластичность против гомеостаза (край хаоса).** Держать $\rho(W) \lesssim 1$ на *непрерывно меняющемся* W — нерешённая в вычислительной нейронауке задача: пластичность воюет с устойчивостью. Плюс аттракторная память имеет малую ёмкость и *ложные* аттракторы; «плотные» версии Хопфилда чинят ёмкость, но математически превращаются в attention — и тогда закономерен вопрос «а почему не взять память самого трансформера».
- **Цена inference.** Прогонять N шагов динамики по графу в сотни тысяч узлов на *каждый* запрос, да ещё фоном для replay — тяжело; нужна серьёзная оптимизация разреженных умножений. Это не смертельно, но и не бесплатно (наш же принцип «граф дёшев только при чистой обвязке»).

- **Принципиальное: теряются свойства, ради которых мы строили граф.** Редактируемость, забывание-по-команде (крипто-шред), провенанс, поточечная ревизия (TMS), метки contested, семантика node_kind — всё это распределённый резервуар *уничтожает*. Нельзя крипто-шреднуть факт из W; нельзя показать провенанс; нельзя гарантированно забыть конкретное («Рин, забудь последние 10 минут» — у компаньона это *UX/приватность*, а не когнитивная поэзия). Возражение «человек тоже не забывает по команде» верно *когнитивно*, но игнорирует, что у Рин есть *юридическо-продуктовые* требования, которых у человека нет. Архив-«чёрный ящик» спасает сырое удаление, но *не* отменяет того, что резервуар уже *выучил* факт в W — распределённо и не-разучиваемо.

Принцип F.2 (Ключевая развилка: «не хранить строки» $\neq$ «не иметь адресуемых узлов»)**.** Здесь легко проскочить подмену. Есть два разных смысла «памяти без фактов»: *(а)* не хранить факт как *человекочитаемую строку* — узлы могут быть эмбеддингами; это *дёшево и совместимо* с нашим субстратом; *(б)* вообще не иметь *адресуемых объектов памяти* — всё размазано в W; это и покупает человекоподобность, и *ломает* редактируемость/провенанс/забывание. Ценное — это (а) + ассоциативная динамика; дорогая переоценка — (б). А наш ретрив уже делает (а)+динамику: «возмути узел и смотри, что загорится» — это диффузия по типизированному графу. Значит львиную долю пользы этой ветки мы *уже* имеем, не платя за (б). По-настоящему *новых* адоптируемых кусков три: пластичные рёбра (Hebbian-усиление от со-активации — по сути формализованная консолидация), mood-как-нейромодулятор, и CLS-разделение быстрый/медленный (у нас грубо есть как слои session→life).

F.7. Ближняя инженерная форма ветки: что именно стоило бы пробовать

Если отбросить соблазн «сразу заменить память распределённой динамикой целиком», то у этой ветки остаётся очень сильная и вполне рабочая ближняя форма. Её разумно формулировать так:

explicit graph archive + hippocampal associative index + small plastic reservoir + replay

То есть не «убираем граф», а вводим рядом с ним *вторую форму памяти*: граф отвечает за адресность, provenance, forgetting и аудит, а динамический слой — за pattern completion, ассоциативный recall и временную интерференцию.

Как это должно выглядеть конструктивно.

1. **Архив остаётся явным.** Все наблюдения, коммиты, ретракции и удаления по-прежнему живут в event-log и типизированном графе.
2. **Быстрый индекс остаётся разреженным.** Cue сначала поднимает не весь мир, а небольшой набор ассоциативных seed-узлов — «гиппокампальный» стартер для дальнейшей динамики.
3. **Динамика живёт не на всём графе, а на активном подграфе.** Практически это самый важный инженерный ход: первый прототип не обязан гонять пластику по сотням тысяч узлов. Куда разумнее взять подграф, уже поднятый retrieval'ом (якоря + локальное расширение + bridge-узлы), и крутить reservoir-подобную динамику *только на нём*. Тогда и цена inference падает, и диагностика становится обозримой.

4. **Replay идёт от архива к динамике, а не наоборот.** Ночью или в idle режимах архив проигрывает важные эпизоды, а пластичный слой медленно меняет веса и режимы ассоциации.
5. **Write-path остаётся у графа.** Динамический слой не пишет факты напрямую; он поднимает кандидатов, контркандидатов, bridge-конфигурации, а коммиттер судит их обычным путём.

Самая опасная ошибка прототипа

Если первый же прототип будет строиться как «огромный свободно пульсирующий резервуар поверх всей памяти», проект почти наверняка утонет не в идеях, а в инфраструктурной боли: дорого, трудно стабилизировать, трудно понимать, трудно проверять. Активный подграф — не уступка, а правильная ступень масштабирования.

Где здесь место mood и гомеостазу. Настроение в этой ветке полезно не как ещё один факт, а как *режимный контроллер*: медленный параметр, меняющий допустимую глубину, усиление и ассоциативную «температуру» динамики. Это почти прямой low-pass-контроллер для границы между апатией и перевозбуждением. Идея «гормонального» гомеостаза — мониторить энергию поля $E(t) = \frac{1}{N}\sum r_i(t)^2$ и скорость его изменения $V(t) = \frac{1}{N}\sum (r_i(t) - r_i(t-1))^2$ — имеет смысл *только* если динамический слой уже реально способен входить в плохие режимы. Тогда можно вводить быстрые broadcast-модуляторы: «седатив» поднимает утечку и снижает усиление, «кофеин» делает систему чувствительнее, «дофамин» временно повышает пластичность. Но на сегодняшнем графовом ядре это лекарство от болезни, которой ещё нет.

Что проверять на стенде. У этой ветки есть вполне конкретные, не философские тесты.

1. **Pattern completion:** восстанавливает ли слой правильную окрестность по частичному cue лучше, чем чистый graph-retrieval.
2. **Интерференция:** не начинает ли новый опыт разрушать старые устойчивые ответы.
3. **Source confusion:** не растёт ли доля случаев, где ассоциативный recall красив, но provenance расползается.
4. **Readout control:** сохраняется ли смысл в самой динамике, если посадить нарочно тупой декодер.
5. **Стабильность:** не уходит ли слой в тривиальное затухание или в хаотический шторм.

Именно в таком виде ветка выглядит не фантазией, а сильной исследовательской программой: сначала маленький пластичный слой на активном подграфе, потом replay, потом режимные контроллеры, и только после этого — разговор о более радикальных носителях.

F.8. Квантовый слой: честно

Соблазн прямой: ассоциативная память = волновая динамика, кванты = волновая механика, значит кванты идеальны. Не совсем. *Чистая* квантовая дина-

мика унитарна ($|\psi(t)\rangle = U(t)|\psi(0)\rangle$): обратима, сохраняет норму, *не забывает* и *не* схлопывается в аттрактор ($\|Ux - Uy\| = \|x - y\|$ — нет сжатия). А ассоциативной памяти нужно ровно сжатие: много близких состояний → одно воспоминание. Поэтому квантовая ассоциативная память неизбежно тащит *диссипацию*, измерения, связь со средой, open-system-динамику. Второй подвох: 2^n амплитуд $\neq 2^n$ *доступных* чисел — измерение выдаёт ограниченную классическую информацию, а классический вход ещё надо *загрузить* в квантовое состояние. Поэтому экспоненциальное гильбертово пространство не даёт автоматически экспоненциальной полезной памяти.

Оценка кванта: исследовательски высоко, практически сейчас низко

Область quantum reservoir computing и quantum associative memory реально существует и активно развивается (спиновые сети, фотоника на сотнях мод, quantum Hopfield на малых NISQ-устройствах). *Дисклеймер о верифицируемости*: конкретные свежие эксперименты и цифры «превосходства», всплывшие в обсуждении, лежат за горизонтом надёжной проверки этого текста — их стоит держать как *иллюстрацию направления*, а не как установленный quantum advantage; сам advantage для памяти пока не доказан. Разумная ближняя форма — *не* «память Рин = квантовый компьютер», а гибрид: классический гиппокамп → *маленький* квантовый резервуар как чрезвычайно богатый нелинейный feature-generator → классическое пластичное хранилище. Квант тут — не хранилище на двадцать лет жизни, а мощный возбудитель одного оператора.

F.9. Вердикт и как оставить дверь открытой

Для Рин с её требованиями (редактировать, забывать, объяснять источник, поддерживать истинность) **символический типизированный граф остаётся хребтом**, а динамический слой — это *дополняющая* ассоциативная надстройка («что я сейчас помню») и отношения *учитель–ученик* (граф = архив/учитель). Радикальную версию «убрать семантический граф» для наших требований отклоняем — с конкретными причинами (потеря забывания/провенанса/ TMS), а не из робости. Но инженерно правильно **держать динамический слой за чистым интерфейсом**: тогда классический решатель $R_{\text{classical}}$ можно однажды заменить на $R_{\text{oscillator}}/R_{\text{photonic}}/R_{\text{quantum}}$, не переписывая остальное.

Фальсифицируемые эксперименты, если идти в эту ветку

Три чистых вопроса, которые можно поставить на стенде, не ломая текущую систему: *(1)* сравнить e^{-tL}, $\cos(t\sqrt{L})$ и e^{-itL} на *одной* topologии memory-графа — какой профиль лучше поднимает правильную окрестность (diffusion vs wave vs quantum-walk memory); *(2)* даёт ли *пластичная* надстройка над типизированным графом (Hebbian-усиление соактивированных рёбер) измеримо лучший recall, чем статические веса, — и не ценой ли катастрофической интерференции; *(3)* readout-контроль: посадить нарочно тупой декодер и проверить, есть ли вообще содержательная информация в динамике, прежде чем строить сложный мост к LLM. Все три — на golden-микромире (гл. 22), где есть ground truth.

G. Будущие ветви: как расширять систему, не теряя её

G.1. Две обещанные дальние ветви

У расширенного курса есть ещё одна задача: не просто завершить сегодняшнюю версию, но и подготовить язык, на котором будут обсуждаться следующие большие ветви. На горизонте уже видны по меньшей мере две:

1. резервуарные вычисления как дополнительная ассоциативно-динамическая среда;

2. ассоциативная память на кубитах как исследовательская ветвь квантового слоя.

G.2. Что нужно требовать от любой новой ветви

Новая ветвь не должна приходить в систему как эстетический жест. Для неё нужен одинаковый протокол допуска.

1. Какая конкретная слабость базовой архитектуры атакуется?

2. Как будет измеряться улучшение?

3. Как ветвь взаимодействует с provenance, deletion и revision?

4. Что остаётся в явном графе, а что уходит в скрытую динамику?

G.3. Резервуарная ветвь: что в ней действительно интересно

Резервуарная схема интересна не тем, что “она нейроподобна”, а тем, что даёт богатый нелинейный feature space для частичных cue и сложных временных контекстов. Если использовать её осторожно, она может:

1. улучшать pattern completion;

2. давать признаки для сложных mood/history-dependent retrieval policies;

3. служить дополнительным предложителем bridge-кандидатов.

Но её нельзя пускать напрямую в write-path без жёсткого интерфейса.

G.4. Квантовая ветвь: чего нельзя обещать заранее

Квантовая ассоциативная память соблазнительна по двум причинам: волновая интуиция и естественная работа с суперпозициями. Но сама по себе эта красота ничего не доказывает. До стенда допустимы только такие утверждения:

1. ветвь может дать иной режим диффузии и поиска;
2. ветвь может дать иной bias к pattern completion;
3. ветвь *не считается полезной*, пока не показала измеримый выигрыш на общем стенде против классических альтернатив.

G.5. Правильная амбиция

Принцип G.1 (Большая ветвь должна быть сравнима с малой системой)**.** Чем радикальнее новая ветвь, тем строже она обязана сравниваться с уже работающей графовой памятью. Иначе вместо расширения курса получается побег из проверяемости.

Это правило и будет держать будущие главы честными.

H. Расширение: гибридный резервуарный слой для памяти Рин

H.1. Почему эта ветка отличается от прочих будущих ветвей

Не всякая дальняя ветвь одинаково близка к инженерной практике. Гиперграфы, клеточные комплексы и квантовые носители полезны как сильные структурные и исследовательские идеи, но резервуарно-динамическая ветвь выделяется на их фоне по трём причинам.

1. Она **многое объясняет** уже на языке текущей архитектуры: почему retrieval можно понимать как возмущение системы, почему mood естественно трактовать как режим динамики, почему replay похож на настоящую консолидацию, а не на summarization.
2. Она **интерпретируема**: здесь нет требования отдать весь смысл чёрному ящику. Напротив, явный граф, быстрый индекс, динамический слой и write-path можно развести по ролям.
3. Она **doable**: первый прототип можно собрать, не переписывая весь мозг Рин, а добавив рядом с текущим графовым позвоночником новый слой с чистым интерфейсом.

H.2. Ключевая идея: две формы памяти вместо одной

Главный выигрыш этой ветки начинается не с замены графа, а с отказа от ложной альтернативы “или архив, или динамика”. Плодотворная формула другая:

$$\boxed{\text{память Рин} = \text{адресуемый архив} + \text{динамическая ассоциативная среда}}$$

Архив отвечает на вопросы:

- что действительно было записано;
- когда это произошло;
- откуда это известно;
- что было удалено;
- какие потомки надо ревизовать.

Динамическая среда отвечает на другие вопросы:

- что всплывает по частичному cue;
- какие далёкие мосты сейчас собираются в одну конфигурацию;

- насколько контекст “живой”, а не просто найденный;
- в какой ассоциативный бассейн скатывается текущее состояние.

Принцип Н.1 (Архив и динамика решают разные классы задач)**.** Если архив пытается быть живой памятью, он становится тяжёлой базой данных с плохой ассоциативностью. Если динамика пытается быть архивом, она теряет provenance, deletion, ревизию и юридическую адресность. Сильная архитектура не выбирает одно из двух, а разводит их роли.

Н.3. Краткая теория: что такое резервуар в этом контексте

В классическом reservoir computing есть большая рекуррентная система с фиксированными случайными связями:

$$h_t = \sigma(W_{\text{res}} h_{t-1} + W_{\text{in}} x_t),$$

и обучается только считывающий слой:

$$y_t = W_{\text{out}} h_t.$$

Это даёт богатое нелинейное отображение истории входов в пространство состояний.

Но для памяти Рин классический резервуар в чистом виде недостаточен по двум причинам.

1. Он даёт в основном *fading memory*, то есть естественную память на конечный горизонт.
2. Он не решает задачу устойчивого хранения многолетнего опыта без изменения самого хранилища.

Поэтому здесь уместнее не “reservoir computing как есть”, а гибрид:

быстрый ассоциативный индекс + медленный пластичный резервуар + явный архив

То есть:

- быстрый индекс обеспечивает pattern separation и запуск recall;
- пластичный слой даёт богатую ассоциативную динамику;
- архив страхует долгую память, provenance и точную ревизию.

Н.4. Архитектурный скелет

1. Event encoder

На вход идёт не “голый факт”, а событие целиком: текст, время, настроение, источник, социальный контекст, task-status, salience. Результатом становится вектор события x_t.

2. Быстрый разреженный индекс

Из x_t строится sparse code:

$$z_t = \mathrm{kWTA}(Px_t),$$

где активна малая доля компонент. Это ближайший аналог “гиппокампального” индекса: не хранение всей памяти, а адресное зажигание стартовых площадок.

3. Быстрая ассоциативная связка

На этом слое действует быстрая запись совместности:

$$H \leftarrow H + \eta_h z_t z_t^\top, \qquad \eta_h \text{ велика.}$$

Он нужен для свежих связей, которые должны появляться почти немедленно.

4. Медленный пластичный резервуар

Дальше живёт собственно динамический слой:

$$r_{t+1} = (1-\lambda)\, r_t + \lambda\, \sigma(W_t r_t + U x_t + B z_t).$$

Пластичность медленная:

$$W_{t+1} = W_t + \eta_c \Delta W_t, \qquad \eta_c \ll \eta_h.$$

Именно это разделение временных шкал делает идею похожей на CLS, а не на хаотическую самопереписывающуюся сеть.

5. Replay

В idle и sleep режимах архивные события снова проигрываются через индекс и резервуар. Так пластичный слой учится не от разрозненных единичных вспышек, а от повторяемых, структурно отобранных паттернов.

6. Readout

Readout не должен быть ни тупым до бесполезности, ни настолько умным, чтобы съесть всю работу динамики. Идеологически здесь есть три уровня:

1. простое считывание диагностик и классов;
2. считывание набора кандидатов / активных подграфов;
3. интерфейс в hidden-state или cross-attention слой LLM.

7. Архив-учитель

Явный граф остаётся не просто “запасным вариантом”, а учителем и судьёй: он поставляет материал для replay, хранит provenance и принимает окончательные записи.

Н.5. Новая инженерная рекомендация: начинать не со всего графа

Здесь я специально добавляю предложение, которого не было в исходном обсуждении в такой форме, но которое следует из него как наиболее разумный путь prototyping.

Принцип Н.2 (Первый резервуар должен жить на активном подграфе)**.** Самый практичный MVP — не “пластичный океан поверх всей памяти”, а динамический слой на активном подграфе, который уже поднят retrieval’ом: якоря, их окрестность, bridge-узлы, несколько важных provenance-носителей и контрсвидетельства.

Пусть retrieval поднял подграф $G_t = (V_t, E_t)$. Тогда резервуарный шаг идёт не на всём V, а только на V_t:

$$r_{k+1}^{(t)} = (1-\lambda)\, r_k^{(t)} + \lambda\, \sigma(W^{(t)} r_k^{(t)} + u^{(t)}),$$

где $W^{(t)}$ индуцирован текущим активным подграфом.

Преимущества этого решения:

1. вычислительная цена резко падает;
2. появляется хороший стенд для диагностики;
3. динамический слой автоматически привязан к интерпретируемому фрагменту памяти;
4. replay можно запускать локально по тематическим областям, а не по всему миру сразу.

Недостаток только один: это уже не “чистая распределённая память всего мозга”, а слой локальной ассоциативной динамики. Но именно это и делает ветку инженерно подъёмной.

Н.6. Mood как контроллер режима

Диалог очень точно попадает в одно важное место: mood в такой архитектуре полезно понимать не как ещё один текстовый факт и не как косметический тег в промпте, а как *медленный регулятор режима динамики*.

Пусть $\gamma(m)$ — функция настроения. Тогда она может модифицировать эффективный усилитель рекуррентной матрицы:

$$W_{\text{eff}} = \gamma(m)\, W.$$

Или, более осторожно, влиять на leak, gain и bias перехода:

$$r_{t+1} = (1-\lambda(m))r_t + \lambda(m)\, \sigma\big(\beta(m) W r_t + U x_t + b(m)\big).$$

Почему именно mood подходит на эту роль

У настроения есть свойство, которое в инженерии систем управления необычайно удобно: оно инертно. Оно не дёргается на каждый токен, а меняется медленно. Значит, оно может быть low-pass-контроллером, за-

дающим общий ландшафт для быстрых когнитивных колебаний.

Интерпретация проста:

- спокойный режим делает динамику более диссипативной;
- сфокусированный или тревожный режим подводит её ближе к зоне длинных ассоциаций;
- творческий поиск может временно расширять допустимую область лёгкого хаоса.

H.7. Гомеостатический контроллер: «пирожки Алисы»

Вторая идея из диалога — ещё интереснее: не только медленный режимный сдвиг, но и быстрый контур искусственного “химического” гомеостаза.

Градусник системы

Полезно следить по крайней мере за двумя агрегатами:

$$E(t) = \frac{1}{N}\sum_{i=1}^{N} r_i(t)^2, \qquad V(t) = \frac{1}{N}\sum_{i=1}^{N} (r_i(t) - r_i(t-1))^2.$$

Первый измеряет “энергию” поля, второй — скорость его изменения.

Быстрые модуляторы

Если система выходит из рабочего диапазона, можно временно впрыскивать глобальные модуляторы.

1. **Седатив**: увеличивает утечку и уменьшает gain нелинейности.
2. **Кофеин**: уменьшает затухание и повышает возбудимость.
3. **Дофаминоподобный модулятор**: временно поднимает learning rate пластичности, когда пришёл важный или неожиданный опыт.

Это не подарок бесплатно

Гомеостатический контур очень красив, но он вносит собственные параметры, собственные границы, свои ошибки калибровки и свой новый способ сломать систему. Поэтому он не должен появляться раньше, чем реально возникнет болезнь, которую он лечит.

H.8. Как объединяются mood и гомеостаз

Лучший способ совместить эти две идеи — не путать их.

- **Mood** — медленный режимный слой. Он сдвигает целевые границы, в которых система считает себя “нормальной”.

- **Гомеостаз** — быстрый автоматический контур. Он следит, чтобы текущая динамика не вышла за границы, заданные более медленным режимом.

Например, в спокойном режиме $E_{\max}$ ниже и система быстрее “успокаивается”. В творческом режиме $E_{\max}$ выше: допускается более длинная и смелая ассоциативная траектория, прежде чем включится торможение.

H.9. Что строить сначала

У этой ветви очень легко потерять масштаб и начать “с будущего будущего”. Поэтому полезен строгий порядок прототипирования.

1. **Шаг 1.** Пластичность на типизированном графе без отдельного резервуара: Hebbian-like усиление со-активированных рёбер и replay от архива.
2. **Шаг 2.** Малый динамический слой на активном подграфе: проверка, даёт ли он реальный выигрыш в pattern completion и bridge formation.
3. **Шаг 3.** Mood как параметр режима динамики.
4. **Шаг 4.** Только при реальной нестабильности — гомеостатические “гормоны”.
5. **Шаг 5.** Более глубокий readout-интерфейс к LLM.

Как выглядит хороший MVP

Хороший MVP этой ветки — это не “Рин теперь вся reservoir”. Хороший MVP — это следующее: есть golden-микромир, есть текущий graph-retrieval, есть активный подграф, есть малый пластичный слой поверх него, и можно строго измерить, улучшились ли partial-cue recall, bridge rate и устойчивость к интерференции.

H.10. Как проверять

Для этой ветки нужны отдельные тесты, отличные от обычного RAG-like retrieval.

1. Partial cue completion

Подать не полный ключ, а его фрагмент, и измерить, дотягивается ли система до правильной конфигурации лучше базовой версии.

2. Delay under interference

Записать событие, потом зашумить систему похожими событиями, затем проверить, что именно всплывает спустя время и после replay.

3. Provenance stress

Проверить, не выросла ли способность вспоминать ценой потери объяснимости.

4. Stability envelope

Смотреть на $E(t)$, $V(t)$, прокси для spectral radius и длину ассоциативных хвостов.

5. Ablation of controllers

Сравнить: без mood-модуляции, с mood-модуляцией, с грубым homeostasis-контуром, чтобы отделить полезную режимность от чисто декоративной.

Н.11. Главные риски

1. **Readout bottleneck.** Динамика может быть богата, но недоступна для хорошего извлечения.
2. **Пластичность против устойчивости.** Слишком живая система быстро учится и быстро разрушается.
3. **Вычислительная цена.** Даже локальный резервуар нельзя бездумно крутить на каждый запрос и каждый фоновой цикл.
4. **Конфликт с продуктовой адресностью.** Чем больше знания живёт в скрытой динамике, тем тяжелее deletion, provenance и точная юридическая ревизия.

Н.12. Вердикт

Из всех “будущих” ветвей именно эта выглядит как самая содержательная промежуточная ступень между сегодняшней графовой памятью и более радикальными субстратами.

Принцип Н.3 (Самая сильная версия ветки не радикальна, а гибридна)**.** Наиболее перспективная форма резервуарной ветви для Рин — это не отказ от графа, а гибрид: явный архив, быстрый ассоциативный индекс, локально или тематически ограниченный пластичный резервуар, replay и режимные контроллеры. Именно в этой форме она и интерпретируема, и измерима, и инженерно правдоподобна.

Если когда-нибудь эта ветвь действительно выстрелит, она даст Рин не “ещё один поисковый слой”, а новую форму живой памяти: менее буквальную, более ассоциативную, но всё ещё связанную с архивом, который позволяет оставаться честной, редактируемой и проверяемой.

Литература

Ниже — работы, на которые опирается монография, в согласованном формате. Для препринтов и внутренних материалов проекта (библиографии волтов RIN и RIN–DTI) в конце записи в скобках указан локатор во внутренней библиографии; это указание на местоположение источника, а не замена выходных данных.

Литература

[1] J. R. Anderson, D. Bothell, M. D. Byrne и др. An Integrated Theory of the Mind. *Psychological Review*, 111(4), 2004. (Базовая активация ACT-R; см. также Anderson & Schooler, 1991.)

[2] A. M. Collins, E. F. Loftus. A Spreading-Activation Theory of Semantic Processing. *Psychological Review*, 82(6), 1975.

[3] D. O. Hebb. *The Organization of Behavior*. Wiley, 1949.

[4] J. J. Hopfield. Neural Networks and Physical Systems with Emergent Collective Computational Abilities. *PNAS*, 79(8), 1982.

[5] J. S. Park, J. C. O’Brien, C. J. Cai и др. Generative Agents: Interactive Simulacra of Human Behavior. *UIST*, 2023.

[6] J. Toy, J. MacAdam, P. Tabor. Metacognition is all you need? Using Introspection in Generative Agents to Improve Goal-directed Behavior. arXiv:2401.10910, 2024.

[7] D. Kahneman. *Thinking, Fast and Slow*. Macmillan, 2011. (System 1 / System 2.)

[8] R. E. Nisbett, T. D. Wilson. Telling More Than We Can Know: Verbal Reports on Mental Processes. *Psychological Review*, 84(3), 1977. (Конфабуляция при самообъяснении.)

[9] J. Doyle. A Truth Maintenance System. *Artificial Intelligence*, 12(3), 1979.

[10] J. de Kleer. An Assumption-Based TMS. *Artificial Intelligence*, 28(2), 1986.

[11] W. V. O. Quine. Two Dogmas of Empiricism. *The Philosophical Review*, 60(1), 1951. (Тезис недоопределённости Дюэма–Куайна.)

[12] L. Ross, M. R. Lepper, M. Hubbard. Perseverance in Self-Perception and Social Perception. *J. Personality and Social Psychology*, 32(5), 1975. (Belief perseverance.)

[13] P. Bak, C. Tang, K. Wiesenfeld. Self-Organized Criticality. *Physical Review Letters*, 59(4), 1987.

[14] J. M. Beggs, D. Plenz. Neuronal Avalanches in Neocortical Circuits. *J. Neuroscience*, 23(35), 2003.

[15] L. Page, S. Brin, R. Motwani, T. Winograd. The PageRank Citation Ranking: Bringing Order to the Web. Stanford InfoLab Technical Report, 1999.

[16] H. Tong, C. Faloutsos, J.-Y. Pan. Fast Random Walk with Restart and Its Applications. *ICDM*, 2006.

[17] D. A. Spielman. *Spectral and Algebraic Graph Theory* (лекционные заметки), Yale University.

[18] A. Ortega, P. Frossard, J. Kovačević и др. Graph Signal Processing: Overview, Challenges, and Applications. *Proceedings of the IEEE*, 106(5), 2018.

[19] J. Topping, F. Di Giovanni, B. P. Chamberlain и др. Understanding Over-Squashing and Bottlenecks on Graphs via Curvature. *ICLR*, 2022. (Волт RIN–DTI, поз. 148.)

[20] M. Black, Z. Wan, A. Nayyeri, Y. Wang. Understanding Oversquashing in GNNs through the Lens of Effective Resistance. *ICML*, 2023. (Волт RIN–DTI, поз. 183.)

[21] T. K. Rusch, M. M. Bronstein, S. Mishra. A Survey on Oversmoothing in Graph Neural Networks. arXiv:2303.10993, 2023. (Волт RIN–DTI, поз. 170.)

[22] C. Bodnar, F. Frasca, N. Otter и др. Weisfeiler and Lehman Go Cellular: CW Networks. *NeurIPS*, 2021. (Волт RIN–DTI, поз. 185.)

[23] C. Bodnar, F. Frasca, Y. Wang и др. Weisfeiler and Lehman Go Topological: Message Passing Simplicial Networks. *ICML*, 2021. (Волт RIN–DTI, поз. 186.)

[24] C. Bodnar, F. Di Giovanni, B. P. Chamberlain и др. Neural Sheaf Diffusion. *NeurIPS*, 2022.

[25] R. Wang, D. D. Nguyen, G.-W. Wei. Persistent Spectral Graph. *Int. J. Numerical Methods in Biomedical Engineering*, 2020. (TDA/персистентный лапласиан для биомолекул; ср. TopologyNet, Cang & Wei, 2017. Волт RIN–DTI, поз. 100, 103, 104, 144.)

[26] A. Ortony, G. L. Clore, A. Collins. *The Cognitive Structure of Emotions*. Cambridge University Press, 1988. (OCC-модель оценивания; ср. компонентная модель K. Scherer.)

[27] K. Friston. The Free-Energy Principle: A Unified Brain Theory? *Nature Reviews Neuroscience*, 11(2), 2010. (Ошибка предсказания как сигнал значимости.)

[28] R. Speer, J. Chin, C. Havasi. ConceptNet 5.5: An Open Multilingual Graph of General Knowledge. *AAAI*, 2017. (Включая эмбеддинги Numberbatch.)

[29] M. Sap, R. Le Bras, E. Allaway и др. ATOMIC: An Atlas of Machine Commonsense for If-Then Reasoning. *AAAI*, 2019.

[30] J. D. Hwang, C. Bhagavatula, R. Le Bras и др. (Comet-)ATOMIC 2020: On Symbolic and Neural Commonsense Knowledge Graphs. *AAAI*, 2021.

[31] P. West, C. Bhagavatula, J. Hessel и др. Symbolic Knowledge Distillation: from General Language Models to Commonsense Models. *NAACL*, 2022. (ATOMIC10x.)

[32] D. Vrandečić, M. Krötzsch. Wikidata: A Free Collaborative Knowledgebase. *Communications of the ACM*, 57(10), 2014.

[33] K. Xu, W. Hu, J. Leskovec, S. Jegelka. How Powerful are Graph Neural Networks? *ICLR*, 2019. (GIN; волт RIN–DTI, поз. 68.)

[34] G. Corso, L. Cavalleri, D. Beaini и др. Principal Neighbourhood Aggregation for Graph Nets. *NeurIPS*, 2020. (Волт RIN–DTI, поз. 178.)

[35] E. J. Hu, Y. Shen, P. Wallis и др. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021.

[36] R. Rafailov, A. Sharma, E. Mitchell и др. Direct Preference Optimization. *NeurIPS*, 2023.

[37] D. Zhou, J. Huang, B. Schölkopf. Learning with Hypergraphs: Clustering, Classification, and Embedding. *NeurIPS*, 2006. (Гиперграфовый лапласиан.)

[38] L.-H. Lim. Hodge Laplacians on Graphs. *SIAM Review*, 62(3), 2020. (Ходж-лапласианы и высшие размерности.)

[39] J. Zhu, J. Li, C. Zhang и др. HeLa-Mem: Hebbian Learning and Associative Memory for LLM Agents. Препринт, 2026. (Внутренняя библиография волта RIN, поз. 53.)

[40] Библиография памяти LLM-агентов из волта RIN: APEX-MEM, TRACE, SYNAPSE, GAAMA, MRMS, Zep/Graphiti, Persistent Sycophancy, LifeSide, LoCoMo и др. (Внутренние материалы проекта; полные выходные данные — в `99 Sources/1. Bibliography (RIN)`.)

[41] Материалы проекта RIN: модульные системные промпты (`prompts/global_core/*`), persona-тесты и логи Stage 0 (`logs/rin_stage0.jsonl`), Ринциклопедии (`docs/rincyclopedia_*`).